



Hantro VC9000E Video Encoder

API User Manual

Document Revision 1.11

19 September 2024

This document is compatible with the following product:

Hantro VC9000E software release version 2.4.123

VERISILICON

LEVEL B: CONFIDENTIAL - DISTRIBUTION RESTRICTED

Legal Notices

COPYRIGHT INFORMATION

This document contains proprietary information of VeriSilicon Holdings Co., Ltd. VeriSilicon reserves the right to make changes to any products herein at any time without notice. VeriSilicon does not assume any responsibility or liability arising out of the application or use of any product described herein, except as expressly agreed to in writing by VeriSilicon; nor does the purchase or use of a product from VeriSilicon convey a license under any patent rights, copyrights, trademark rights, or any other of the intellectual property rights of VeriSilicon or third parties.

DISCLOSURE/RE-DISTRIBUTION LIMITATIONS

The information contained herein may be directly re-distributed or may be adapted for re-distribution by SoC vendors who have licensed the related Hantro core IP, with the understanding that the re-distribution is limited to those customers of the SoC vendor who have active and valid NDA or licenses applicable for the SoC which contains the related Hantro core IP. Otherwise, the information contained herein is not to be used by or disclosed to the third parties without the express written permission of an officer of Hantro Corporation or VeriSilicon Holdings Co., Ltd.

(VeriSilicon Distribution **LEVEL B: Confidential - Distribution Restricted**)

Upon request VeriSilicon provides this document in Word format for adaptive inclusion in documentation intended for the SoC vendor's customers. VeriSilicon requests a pre-release copy of the adaptation for review and approval. In the adapted document please identify the technology and features described in this document using the Hantro trademark and include a reference to the copyright owner, for example: "This section describes the Hantro® VPU and contains copyright material disclosed with permission of VeriSilicon Holdings Co., Ltd, who has authorized re-distribution of this material restricted to those NDA partners and licensees of *[the SoC vendor]* who are engineering *[SoC vendor's product]* which includes the discussed Hantro IP."

TRADEMARK ACKNOWLEDGMENT

VeriSilicon, the VeriSilicon logo, Hantro and the Hantro logo are the trademarks of VeriSilicon

Holdings Co., Ltd. in the United States and/or other jurisdictions. All other trademarks are the property of their respective holders.

For our current distributors, sales offices, design resource centers, and product information, visit our web site located at <http://www.verisilicon.com>.

VeriSilicon Confidential, Copyright © 2024 by VeriSilicon Holdings Co., Ltd. All rights reserved worldwide.

Preface

This chapter introduces the *Hantro VC9000E Video Encoder API User Manual*.

About This Document

This document introduces the application programming interface (API) of Hantro VC9000E video encoder IP. It describes the data types, enumerations, structures, and functions in the API, which is written in C code. It also provides a programming overview and examples for your reference.

Additional Reading

- Hantro VC9000E Video Encoder Hardware Features
- Hantro VC9000E Video Encoder Memory Buffer and Format Organization
- Hantro VC9000E Video Encoder Test Bench User Manual
- Hantro VC9000E Video Encoder Wrapper Layer API User Manual
- **ITU-T Rec. H.265**: High-efficiency video coding
- **ITU-T Rec. H.264**: Advanced video coding for generic audiovisual services
- **ITU-R REC. BT.601**: Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios
- **ITU-R REC. BT.709**: Parameter values for the HDTV standards for production and international programme exchange
- **ITU-R REC. BT.2020**: Parameter values for ultra-high-definition television systems for production and international programme exchange

Table of Contents

Legal Notices	1
Preface	3
1 Overview	21
1.1 IP Introduction	21
1.2 Supported Codec Standards and Profiles	21
2 Programming Overview	23
2.1 Overall Workflow of the Encoder	23
2.2 Encoder Instance States	24
2.3 Encoder Limitations	26
2.4 Initialization of the Encoder	26
2.4.1 Multi-Stream Support	27
2.4.2 Stream Profiles, Tiers, and Levels.	27
2.4.3 Stream Formats.	28
2.4.4 Encoded Picture Size	28
2.4.5 Frame Rate Descriptor	29
2.4.6 Default Encoder Initialization Settings	30

2.5	Configuration of the Encoder	30
2.5.1	Bit Rate Control Configuration	30
2.5.2	Coding Control Settings	33
2.5.3	Video Pre-Processing Blocks	34
2.6	Stream Production	35
2.6.1	Stream Starting	35
2.6.2	Frame Encoding	36
2.6.3	Stream Flushing	37
2.6.4	Stream Ending	38
2.7	Release of the Encoder	38
3	Coding Control Features	39
3.1	Coding Formats and Profiles	39
3.1.1	Monochrome Support	39
3.1.2	Automatic Level Detection	40
3.1.3	High Efficiency Image File Format	40
3.1.3.1	Feature Support	40
3.1.3.2	Building Video Encoder with HEIF Enabled	42
3.1.3.3	Saving an Image or an Image Sequence to an HEIF File	43
3.1.3.4	Saving Image Grids to an HEIF File	43

3.2	Quality Tuning	44
3.2.1	RDO Level Selection	44
3.2.2	Look-Ahead Encoding	45
3.2.3	Rate-Distortion Optimized Quantization (RDOQ)	46
3.2.4	Tuning for Various Quality Targets.	46
3.2.5	Adaptive Quantization	46
3.2.6	Anti Intra Flicker	46
3.3	Metadata Programming	47
3.3.1	External SEI Information	47
3.3.2	Interlaced Coding for HEVC	47
3.3.3	VUI Video Signal Type	48
3.3.4	HDR Information Programming.	48
3.3.5	HDR10+ or Dolby Vision Information Insertion.	49
3.4	Frame-Level Encoding	49
3.4.1	Skipped-Frame Insertion	49
3.4.2	Reference Customization	49
3.4.3	ROI-Based Coding Control	50
3.4.3.1	Rectangular ROI	50
3.4.3.2	ROI Map	52
3.4.3.3	QP/CU Information Formats	53
3.4.4	Cyclic Intra Refresh	53
3.4.5	Intra Pulse Code Modulation.	54
3.4.6	Stream-Level Encoding Control	54

3.5	Error Concealment Encoding Tools	55
3.5.1	Multi-Slice Encoding	55
3.5.2	VPS/SPS/PPS Re-Sending	55
3.5.3	Frame Re-Encoding	55
3.6	Low-Latency Encoding	55
3.6.1	Low-Latency System Requirements	56
3.6.2	Low-Latency Synchronization	56
3.6.3	Low-Latency Encoding Flows	57
3.7	Encoding Statistics Output	58
3.7.1	CU/MB Statistics Output	58
3.7.2	Bit Statistics Output	59
3.7.3	Frame Statistics Output	59
3.8	Miscellaneous Features	60
3.8.1	Reference Frame Compression	60
3.8.2	Reference Ring Buffer	60
3.8.3	On Scene Change	61
3.8.3.1	Scene Change Detection	61
3.8.3.2	External Scene Change Indication	61
3.8.4	Security Encoding	62
3.8.5	Batch Mode Encoding	62
4	Bit Rate Control Features	64

4.1	Picture-Level Rate Control	64
4.1.1	Constrained Variable Bit Rate	64
4.1.2	Average Bit Rate	65
4.1.3	Constant Bit Rate	65
4.1.4	Variable Bit Rate	66
4.1.5	Constant Rate Factor	66
4.1.6	Constant Quantization Parameter	66
4.2	Block-Level Rate Control	67
4.3	Relationship Between ROI-Based Coding Control and Block-Level Rate Control	67
5	Pre-Processing Features	68
5.1	RGB-to-YUV Conversion	68
5.2	Picture Rotation	69
5.3	Picture Cropping	69
5.4	Offline Down-scaling	70
5.5	In-Loop Down-Scaling	71
5.6	On-the-Fly OSD Mixer	71
5.7	Video Stabilization	72
5.8	Osdmap	72
6	GOP Structure Configuration	74
6.1	Basic GOP Structures	74
6.1.1	Default GOP Structure of Size 1	75
6.1.2	Default GOP Structure of Size 4	76
6.1.3	Example GOP Structure with Temporal Scalable Video Coding (SVC-T)	77

6.2	GOP Structures with Long-Term Reference	77
6.2.1	Example 1: One Special Frame	79
6.2.2	Example 2: One Long-Term Reference Frame	80
6.2.3	Example 3: One Long-Term Reference Frame and One Short-Term Reference Frame	81
6.2.4	Example 4: Two Long-Term Reference Frames	82
6.2.5	Example 5: Combination of Long-Term Reference Frame and Special Frame	83
6.3	Adaptive GOP Structure Size Decision	83
7	Application Examples	84
7.1	Encoding 300 HEVC Frames	84
7.2	Encoding 300 H.264 Frames	90
7.3	Optional Encoder Settings	95
8	Build and Test in Linux	97
8.1	Building the Video Encoder	97
8.1.1	Build Example	98
8.2	Debugging the Video Encoder	98
9	Module Documentation	100
9.1	Numeric Data Types	100
9.1.1	Enumeration Type	100
9.1.1.1	bool_e	101
9.1.1.2	true_e	101

9.2	Basic Encoder API	101
9.2.1	Enumeration Type	103
9.2.1.1	VCEncRet	103
9.2.1.2	VCEncVideoCodecFormat	105
9.2.1.3	VCEncStreamType	105
9.2.1.4	VCEncLevel	106
9.2.1.5	VCEncProfile	108
9.2.1.6	VCEncTier	110
9.2.1.7	VCEncPictureType	110
9.2.1.8	VCEncPictureScanType	113
9.2.1.9	VCEncPictureRotation	114
9.2.1.10	VCEncPictureMirror	114
9.2.1.11	VCEncColorConversionType	114
9.2.1.12	VCEncPictureCodingType	115
9.2.1.13	VCENC_AV1_FRAME_TYPE	116
9.2.1.14	VCEncHDRTransferType	116
9.2.1.15	VCEncChromaIdcType	117
9.2.1.16	VCENC_TuneType	117
9.2.1.17	ReEncodeStrategy	118
9.2.2	Functions	118
9.2.2.1	VCEncGetApiVersion()	118
9.2.2.2	VCEncGetBuild()	119

9.2.2.3	VCEncGetRoiMapVersion()	119
9.2.2.4	VCEncGetBitsPerPixel()	120
9.2.2.5	VCEncGetAlignedStride()	120
9.2.2.6	VCEncInit()	121
9.2.2.7	VCEncRelease()	122
9.2.2.8	VCEncGetPerformance()	122
9.2.2.9	VCEncSetStrmCtrl()	123
9.2.2.10	VCEncSetSeiUserData()	123
9.2.2.11	VCEncStrmGetOutput()	124
9.2.2.12	VCEncSetTestId()	125
9.2.2.13	VCEncSetInputMblines()	125
9.2.2.14	VCEncGetEncodedMblines()	126
9.2.2.15	VCEncSetError()	126
9.2.2.16	VCEncGetCulInfo()	127
9.2.2.17	VCEncGetEncodeMaxDelay()	127
9.2.2.18	VCEncGetClientType()	128
9.3	Coding Control API	129
9.3.1	Functions	129
9.3.1.1	VCEncSetCodingCtrl()	129
9.3.1.2	VCEncGetCodingCtrl()	130

9.4	Bit Rate Control API	130
9.4.1	Enumeration Type	130
9.4.1.1	rcMode_e	130
9.4.2	Functions	131
9.4.2.1	VCEncSetRateCtrl()	131
9.4.2.2	VCEncGetRateCtrl()	131
9.5	Pre-Processing API	132
9.5.1	Functions	132
9.5.1.1	VCEncSetPreProcessing()	132
9.5.1.2	VCEncGetPreProcessing()	133
9.6	PPS Configuration API	133
9.6.1	Functions	133
9.6.1.1	VCEncCreateNewPPS()	133
9.6.1.2	VCEncModifyOldPPS()	134
9.6.1.3	VCEncGetPPSData()	134
9.6.1.4	VCEncActiveAnotherPPS()	135
9.6.1.5	VCEncGetActivePPSId()	135
9.7	Stream Production API	136
9.7.1	Functions	136
9.7.1.1	VCEncStrmStart()	136
9.7.1.2	VCEncStrmEncode()	137
9.7.1.3	VCEncStrmEncodeExt()	137
9.7.1.4	VCEncStrmEnd()	138
9.7.1.5	VCEncFlush()	139

9.8	Helper API	140
9.8.1	Enumeration Type	140
9.8.1.1	anonymous enum	140
9.8.1.2	RoiQpType_e	140
9.8.1.3	RoiCuMode_e	141
9.8.1.4	RoiIntraPartType_e	141
9.8.1.5	RoiInterPartType_e	141
9.8.2	Functions	142
9.8.2.1	EncGetDec400TsBufferSize()	142
9.8.2.2	EncAsicGetCuInfoBufferSize()	143
9.8.2.3	EncUfbcGetSize()	144
9.8.2.4	GetConstantVal()	145
9.8.2.5	EncParseDEC400HeaderData()	145
9.8.2.6	VCencPutHDR10Info()	146
9.8.2.7	VCencFillRoiMap()	146
9.9	HEIF Writer API	147
9.9.1	Enumeration Type	147
9.9.1.1	VCencHeifRet	147
9.9.1.2	VCencHeifDecInfoType	147
9.9.1.3	VCencHeifAuxiliaryType	148
9.9.1.4	VCencHeifMirrorType	148
9.9.1.5	VCENCHeifGridModeType	149
9.9.2	Functions	149
9.9.2.1	VCENCHeifParseConfig()	149
9.9.2.2	VCencHeifInit()	150
9.9.2.3	VCencHeifSetPrpForGrid()	150
9.9.2.4	VCencHeifSetHevcDecConfig()	151
9.9.2.5	VCencHeifWriteStream()	151
9.9.2.6	VCencHeifCloseWriter()	152

9.10	Log System API	152
9.10.1	Marco Definition	154
9.10.1.1	APITRACEERR	154
9.10.1.2	APITRACEWRN	155
9.10.1.3	APITRACE_ERR	155
9.10.1.4	CMLTRACE	155
9.10.1.5	RC_LOG_E	155
9.10.2	Enumeration Type	156
9.10.2.1	VCEncLogEnvControl	156
9.10.2.2	VCEncLogOutput	156
9.10.2.3	VCEncLogFileType	157
9.10.2.4	VCEncLogTraceType	157
9.10.2.5	VCEncLogCheckType	158
9.10.2.6	VCEncLogLevel	158
9.10.3	Functions	158
9.10.3.1	VCEncLogInit()	159
9.10.3.2	VCEncTraceMsg()	159
9.10.3.3	VCEncCheckMsg()	160
9.10.3.4	VCEncLogDestory()	160
9.10.3.5	EncTraceRegs()	161
9.10.3.6	VCEncLogGetEnvSetting()	161

10 Data Structure Documentation	162
10.1 BezierDict	162
10.2 DistDict	163
10.3 dolby_vision_attr	163
10.4 ExtDmDataBlock	164
10.5 ExternalSEI	165
10.6 h264FramInfo	166
10.7 hdr10_plus_attr	168
10.8 Hdr10DisplaySei	169
10.9 HDR10Info	171
10.10 Hdr10LightLevelSei	171
10.11 HeifOffset	171
10.12 hevcFramInfo	172
10.13 ITU_T_T35	174
10.14 MasteringDict	175
10.15 MetaData	175
10.16 NewEncodeParams	175
10.17 PeakLum	176
10.18 ProcWindows	176
10.19 RoiRect.	177
10.20 ufbc_afbc_param	179
10.21 ufbc_param	180
10.22 ufbc_pvrlic_param.	180
10.23 VceApiVersion	181
10.24 VceBuildInfo	181
10.25 VCEncAGopInfo	181
10.26 VCEncAv1Para.	182
10.26.1 Detailed Description	182

10.27	VCEncCodingCtrl	182
10.27.1	Field Documentation	195
10.27.1.1	blockOverride	195
10.27.1.2	enable_slice_irq	195
10.27.1.3	visualBitRateTolerance	195
10.27.1.4	intraReconEnable	195
10.28	VCEncColorConversion	196
10.29	VCEncConfig	196
10.30	VCEncConsumedAddr	205
10.31	VCEncCuInfo	205
10.32	VCEncCuOutData	207
10.33	VCEncCuStatis	207
10.34	VCEncExtParaIn	208
10.35	VCEncGopConfig	209
10.36	VCEncGopPicConfig	210
10.37	VCEncGopPicRps	211
10.38	VCEncGopPicSpecialConfig	211
10.39	VCEncH264Para	212
10.40	VCEncHeifConfig	213
10.41	VCEncHeifCropCfg	215
10.42	VCEncHeifEditListCfg	216
10.43	VCEncHeifGridCfg	216

10.44	VCEncHeifInst	216
10.45	VCEncHeifOSDConfig	217
10.46	VCEncHeifOverlayCfg	217
10.47	VCEncHeifSamplecCfg	218
10.48	VCEncHeifStreamData	218
10.49	VCEncHevcPara	218
10.50	VCEncln	219
10.50.1	Field Documentation	226
10.50.1.1	dec400Enable	226
10.50.1.2	extension	227
10.51	VCEnclnTileExtra	227
10.52	VCEncLogSetting	228
10.53	VCEncMode	229
10.54	VCEncMv	229
10.55	VCEncOut	230
10.56	VCEncOutFrameInfo	232
10.57	VCEncOutTileExtra	232
10.58	VCEncOverlayArea	233
10.59	VCEncPictureArea	234
10.60	VCEncPPSCfg	235
10.61	VCEncPreProcessingCfg	235
10.62	VCEncPrpTileExtra	238
10.63	VCEncRateCtrl	239
10.63.1	Field Documentation	243
10.63.1.1	tolCtbRcIntra	243

10.64	VCEncReconPara	243
10.65	VCEncSliceReady	244
10.66	VCEncStatisticOut	245
10.67	VCEncStrmBufs	247
10.68	VCEncStrmCtrl.	247
10.69	VuiColorDescription	248

Appendix: Glossary	249
---------------------------	------------

Document Revision History	252
----------------------------------	------------

List of Figures

2.1 Typical Encoding Process	23
2.2 Encoder Instance State Transition	24
2.3 Limitations on Input Picture Width and Encoded Picture Width	29
3.1 Saving Images or Image Sequences to an HEIF File	43
3.2 Saving Image Grids to an HEIF File	44
3.3 Two-Pass Look-Ahead Encoding	45
5.1 Picture Dimensions After Rotation	69
5.2 Input Picture and Crop Area	70
6.1 GOP Structure of Size 1	75
6.2 GOP Structure of Size 4	76
6.3 GOP Structure with SVC-T	77
6.4 GOP Structure with One Special Frame	79
6.5 GOP Structure with One Long-Term Reference Frame	80
6.6 GOP Structure with One Long-Term Reference Frame and One Short-Term Reference Frame	81
6.7 GOP Structure with Two Long-Term Reference Frames	82
6.8 GOP Structure with Long-Term Reference Frame and Special Frame	83

List of Tables

6.1 VCEnc GopPicConfig Configurations for GOP Structure of Size 1	75
6.2 VCEnc GopPicConfig Configurations for GOP Structure of Size 4	76
6.3 VCEnc GopPicConfig Configurations in the SVC-T Example	77
6.4 VCEnc GopPicConfig Configurations in Example 1	79
6.5 VCEnc GopPicSpecialConfig Configurations for Special Frame in Example 1	80
6.6 VCEnc GopPicConfig Configurations in Example 2	80
6.7 VCEnc GopPicSpecialConfig Configurations in Example 2	80
6.8 VCEnc GopPicConfig Configurations in Example 3	81
6.9 VCEnc GopPicSpecialConfig Configurations in Example 3	81
6.10 VCEnc GopPicConfig Configurations in Example 4	82
6.11 VCEnc GopPicSpecialConfig Configurations in Example 4	82
6.12 VCEnc GopPicConfig Configurations in Example 5	83
6.13 VCEnc GopPicSpecialConfig Configurations in Example 5	83
9.6 VCEnc Level Enumeration Values Defined for HEVC (H.265)	106
9.7 VCEnc Level Enumeration Values Defined for H.264 (AVC)	107

1 Overview

This chapter gives an overview of the IP core.

1.1 IP Introduction

Designed for easy integration onto the SoC, VeriSilicon's Hantro VC9000E multi-format video encoder is based on silicon-proven designs deployed in millions of smartphones, tablets, set-top boxes, televisions, IP cameras, data centers, server ASIC, and AIoT devices worldwide.

The VC9000E video encoder provides up to 8192x4096 encoding, featured by small silicon footprint, lower power consumption, and negligible load on the host CPU. The integrated reference frame compression (RFC) provides significant savings on system bandwidth, and extensive use of multi-level hierarchical clock gating minimizes the dynamic power consumption.

1.2 Supported Codec Standards and Profiles

The APIs discussed in this version of the document are compatible with the following video codec standards and profiles. Additional levels may be supported via software.

- HEVC - ITU-T Rec. H.265 (04/2013), ISO/IEC 23008-2
 - Main Profile, up to Level 6, High Tier
 - Main 10 Profile, up to Level 6, High Tier
 - Main Still Picture Profile, up to Level 6, High Tier
- AVC - Spec Version 12: ISO/IEC 14496-10 / ITU-T Rec. H.264 (03/2010)
 - Baseline Profile, up to level 6

- Main Profile, up to level 6
- High Profile, up to level 6
- High 10 Profile, level 6
- AV1 - AV1 Bitstream & Decoding Process Specification Version 1.0.0 with Errata 1
 - Main Profile, Level 5.1
- VP9 - VP9 Bitstream & Decoding Process Specification v0.6
 - Profile 0 (8-bit), Level 5.1
 - Profile 1 (10-bit), Level 5.1

2 Programming Overview

2.1 Overall Workflow of the Encoder

The following figure shows the main steps to use the encoder, from initialization, to configuration (optional), stream production, until release of the encoder.

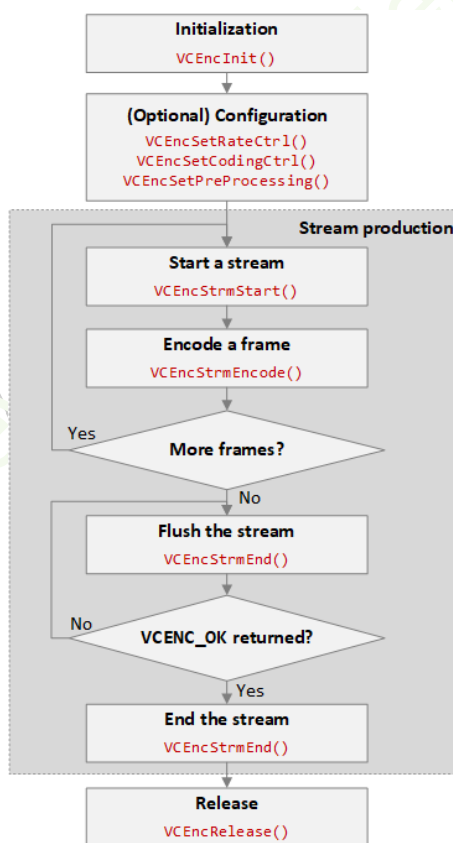


Figure 2.1 Typical Encoding Process

2.2 Encoder Instance States

The following figure shows the state transition of an encoder instance (`VCEncInst`).

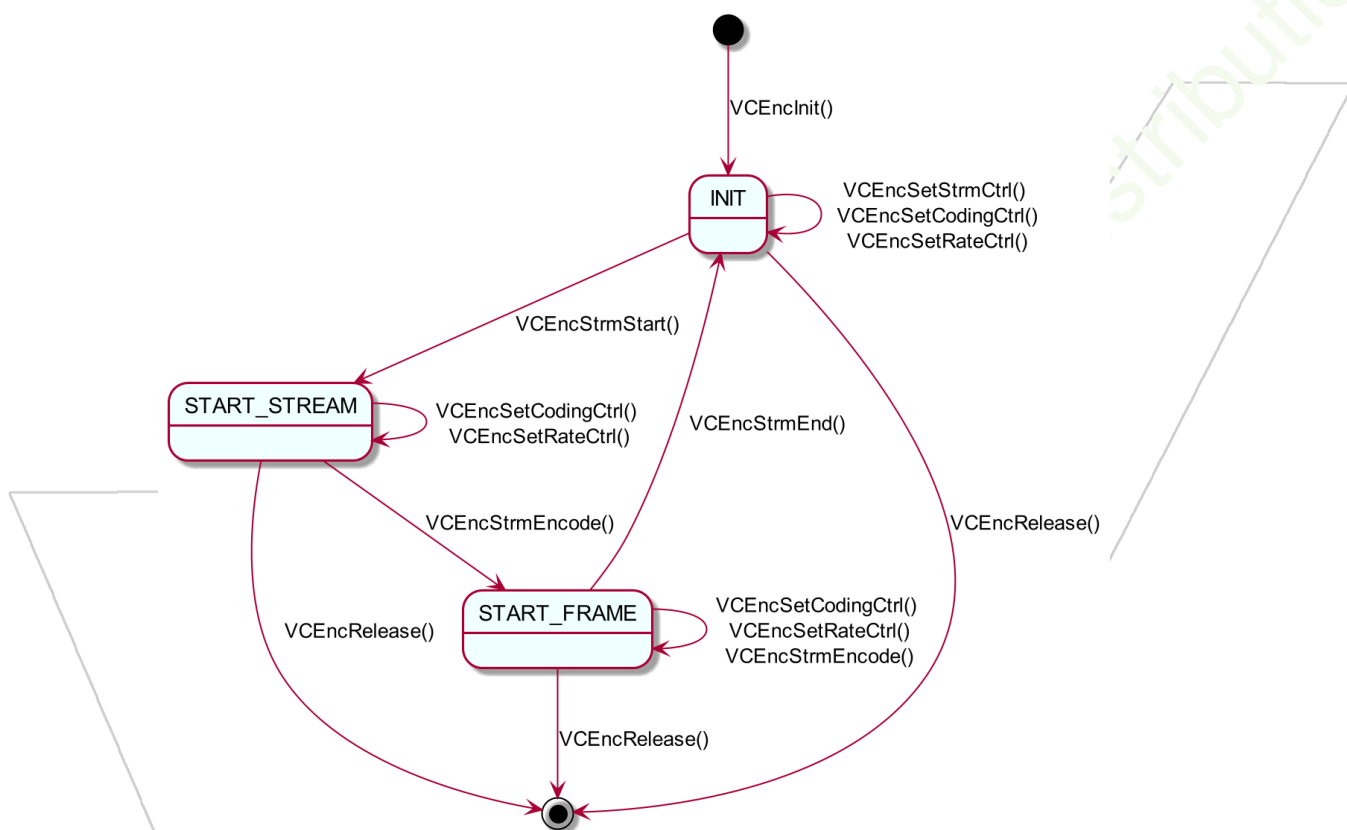


Figure 2.2 Encoder Instance State Transition

The encoder instance states are described as follows:

INIT

- The following API control is applicable on instances in INIT state:
 - Stream-level encoding control through `VCEncSetStrmCtrl()`
 - Coding control through `VCEncSetCodingCtrl()`
 - Bit rate control through `VCEncSetRateCtrl()`

Some parameters for coding control and bit rate control are modifiable only when instances are in INIT state.

- An instance enters the INIT state when any of the following conditions is met:

- The instance is initialized with `VCEncInit()`.
- `VCEncStrmEnd()` is called when the instance is originally in `START_FRAME` state.
- An instance exits the `INIT` state after `VCEncStrmStart()` or `VCEncRelease()` is called.

START_STREAM (SPS and PPS are set when instances are in this state.)

- The following API control is applicable on instances in `START_STREAM` state:
 - Coding control through `VCEncSetCodingCtrl()`
 - Bit rate control through `VCEncSetRateCtrl()`

Only some of the parameters for coding control and bit rate control are modifiable when instances are in `START_STREAM` state.

- An instance enters the `START_STREAM` state after `VCEncStrmStart()` is called when the instance is originally in `INIT` state.
- An instance exits the `START_STREAM` state after `VCEncStrmEncode()` or `VCEncRelease()` is called.

START_FRAME

- The following API control is applicable on instances in `START_FRAME` state:
 - Coding control through `VCEncSetCodingCtrl()`
 - Bit rate control through `VCEncSetRateCtrl()`

Only some of the parameters for coding control and bit rate control are modifiable when instances are in `START_FRAME` state.

- `VCEncStrmEncode()` can be called to encode a new frame when instances are in `START_FRAME` state.

After this API is called, the instances remain in `START_FRAME` state.

- An instance enters the `START_FRAME` state after `VCEncStrmEncode()` is called when the instance is originally in `START_STREAM` state.
- An instance exits the `START_FRAME` state after `VCEncStrmEnd()` or `VCEncRelease()` is called.

2.3 Encoder Limitations

Limitations on Resolution

The encoder hardware supports resolutions such as 8K, 4K, or 1080p. The resolution is configured with the `VCEncInit()` function. During initialization, the encoder software checks the configured resolution against the maximum resolution supported. If the picture is larger than the maximum, the initialization fails.

Limitations on Input Picture Size

The pre-processing block has some limitations on the input picture size. The pre-processor returns failure if the application does not conform to these limitations.

For the limitations, see Section *Video Pre-Processing Blocks*.

Limitations on Feature Support

Feature support varies among different hardware versions and configuration. If the application attempts to use an encoding feature that the hardware does not support, the initialization fails.

For the hardware feature support, check the `EVLHwConfig_t` structure.

2.4 Initialization of the Encoder

The encoding process is started with a `VCEncInit()` call, which initializes the encoder instance through allocation of the encoder-required resources and setup of the encoder. Most parameter settings in the `VCEncInit()` call cannot be changed after initialization.

A successful `VCEncInit()` call returns `VCENC_OK` and creates an encoder instance, which is used as a parameter in subsequent encoder API calls. If the initialization fails, the `VCEncInit()` call returns an error code through `VCEncRet`.

2.4.1 Multi-Stream Support

The encoder hardware supports multiple streams, which is achieved by initializing multiple encoder instances in software and using each to encode a different stream. However, the hardware can encode only one frame at a time. Therefore, the encoder instances share the hardware in time-division mode.

The application can manage the multiple encoder instances and streams by using either of the following methods:

- The application uses a single process to manage all the encoder instances. After one frame is encoded, the application switches to another frame of either the same instance or a different instance.
- The application uses multiple processes, each managing one encoder instance. After one frame is encoded, hardware resources are released for other processes to use.

NOTE: The encoder driver adopts a hardware resource protection mechanism to avoid conflicts across processes or across instances in the same process.

2.4.2 Stream Profiles, Tiers, and Levels

ITU-T Rec. H.265 defines profiles, tiers, and levels for imposing restrictions on some encoder parameters.

You need to manually specify the stream profile and tier based on your tools. A typical configuration is Main profile and Main tier.

The level of encoded stream can be automatically set according to the picture size and bit rate. For details, see Section [Automatic Level Detection](#). You can also manually select a level by using `VCEncConfig.level` before encoder initialization. After the encoder is initialized, the profile, tier, and level cannot be changed.

For tier and level limits for profiles, see Table 9.6 and Table 9.7.

2.4.3 Stream Formats

ITU-T Rec. H.265 and H.264 define the following stream formats:

- NAL unit stream format: consists of plain NAL units. To allow the application to parse individual NAL units, the encoder returns the size of each NAL unit in a separate user-allocated buffer.

This format is commonly used by streaming applications.

- Byte stream format: separates each NAL unit with zero values and a prefix, which makes it easy to separate NAL units.

This is the default stream format.

You can select a stream format by using `VCEncConfig.streamType` based on application needs.

2.4.4 Encoded Picture Size

When initializing the encoder, you need to specify the maximum width and height of the encoded picture in pixels by using the `VCEncConfig` structure. The maximum width and height settings are used as the initial width and height of the encoded picture.

You can update the encoded picture size, only when the encoder instance is in INIT state. The updated picture size must be less than or equal to that specified during initialization.

Video encoding imposes the following limitations on the encoded picture size:

- The width must be a multiple of 2.
- The height must be a multiple of 2.
- The minimum size is 130x130 pixels.
- The maximum size is 8192x8640 pixels for HEVC (H.265) and H.264 (AVC) and 4096x4096 pixels for AV1 and VP9.

NOTE: The input picture size can be different from the encoded one, as shown in the following figure. This occurs if the input picture is cropped before being encoded. For the input picture width (horizontal scanline in the figure), a multiple of 16 is recommended.

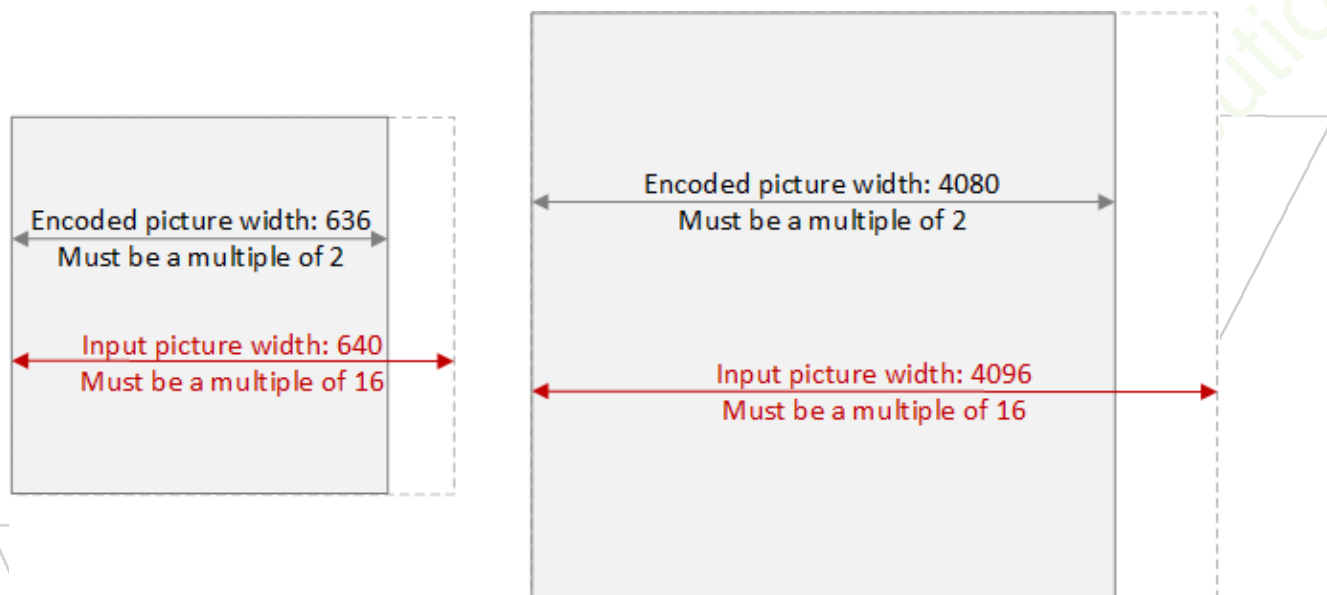


Figure 2.3 Limitations on Input Picture Width and Encoded Picture Width

2.4.5 Frame Rate Descriptor

To achieve efficient control over the bit rate, you need to specify a target frame rate through a numerator and a denominator in the `VCEncConfig` structure.

- Frame rate numerator: the numerator for calculating the target number of frames to be encoded per second.
- Frame rate denominator: the denominator for calculating the target number of frames to be encoded per second.

The target frame rate defines the stream time scale, which indicates the number of equal sub-intervals, namely ticks, within a second. Tick is the base unit for the time increment of each individual frame.

For more information about frame timing, see Section *Stream Production*.

NOTE: The frame rate specified using the numerator and denominator is the target frame rate. The real frame rate is determined by the time increments of encoded video frames. In other words, when the time between two consecutive frames is always two ticks, the real frame rate is 15 FPS even if a 30 FPS frame rate (numerator = 30 and denominator = 1) is targeted.

2.4.6 Default Encoder Initialization Settings

- Picture-level rate control: disabled
- Picture skipping: disabled
- HRD: disabled
- Pre-processing: varies according to the input picture format
- Multi-slice encoding: disabled

2.5 Configuration of the Encoder

After the encoder is initialized, you can update some parameter settings for rate control and encoding control. You can also enable pre-processing blocks to process input pictures before encoding.

2.5.1 Bit Rate Control Configuration

During encoder initialization, the initial bit rate is set to a default value. After the encoder instance is initialized, the bit rate can be controlled based on a specified target. Different rate control modes are available and can be achieved by updating the quantization parameter (QP) according to different strategies. In extreme cases, entire frames can be skipped in order to achieve the target bit rate.

NOTE: A QP controls the step size for quantization which compresses video frames. A lower QP means a smaller step, which lowers the compression rate and provides higher visual quality.

You can use `VCEncSetRateCtrl()` to modify the rate control parameters. For encoder instances in INIT state, all the rate control parameters are configurable. When an encoder instance is in START_STREAM or START_FRAME state:

- If hypothetical reference decoder (HRD) is disabled:
 - Rate control parameters except parameters `ctbRc`, `hrd`, `frameRateNum`, and `frameRateDenom` can be altered between frames.
- It is recommended that you update rate control parameters at the end of a group of pictures (GOP) when needed.

- To modify the setting of `ctbRc`, `hrd`, `frameRateNum`, or `frameRateDenom`, you need to restart the stream as follows to update whole-sequence syntax such as SPS or PPS:
 1. End the stream by calling `VCEncStrmEnd()`.
 2. Modify the parameter settings by calling `VCEncSetRateCtrl()`.
 3. Start the stream by calling `VCEncStrmStart()`.
 4. Encode subsequent frames by calling `VCEncStrmEncode()`.
- If HRD is enabled, you need to follow the preceding procedure to restart the stream for altering most parameters.

Before you call the `VCEncSetRateCtrl()` function to modify parameter settings, it is recommended that you query the current settings by using `VCEncGetRateCtrl()`.

The rate control settings available in the `VCEncSetRateCtrl()` function are described as follows:

- **Picture-level rate control:** a frame based rate control algorithm, which adjusts the QP between frames.
- **Block-level rate control:** a CTB/MB based rate control algorithm, which adjusts the QP inside a frame.
- **Picture skipping:** the picture skipping feature, which skips frames when needed.

You can enable this feature when the target bit rate is so low that it cannot be achieved through adjusting the QP. If this feature is enabled, the output frame rate may be different from the input frame rate.

NOTE: If HRD is enabled, frames may be skipped even if picture skipping is disabled.

- **Default QP:** the default or initial QP value used by the encoder. To enable the encoder to calculate a QP value based on resolution, frame rate, and target bit rate, you can set the default QP to -1.
 - When picture-level rate control is enabled, the encoder uses the specified value as the initial QP for the first encoded picture.
 - When picture-level rate control is disabled, the encoder uses the specified value as the default QP for every encoded picture.

For block-level rate control, the specified value affects picture-level QP, based on which block-level rate control automatically adjusts the QP value inside each frame.

- **Min QP** and **Max QP**: the minimum and maximum QP values, valid only if picture or block-level rate control is enabled.

You need to set the minimum and maximum QP values in variable bit rate (VBR) scenarios where constant quality is preferred over constant bit rate. However, the minimum and maximum QP values limit the possibilities for rate control to achieve the target bit rate.

- **Output bit rate**: the target bit rate in bits per second for the output stream, valid if picture-level rate control is enabled.

The initial bit rate is set to a default value. You must update it to the target value before the stream encoding process starts. If HRD is enabled, the updated bit rate must be within the limits set by the stream tier and level selected during encoder initialization. For detailed limitations, see ITU-T Rec. H.265.

- **HRD**: an algorithm defined in ITU-T Rec. H.265 and H.264 to check an output stream with its bit rate.
 - When HRD is enabled, the encoder runs a coded picture buffer (CPB) model with a specified size to check whether the CPB can accommodate the stream data for the decoder to decode each picture.
 - When HRD is disabled, the encoder does not check the bit rate of the output stream. The encoder may produce streams whose size exceeds the capacity of the CPB.

The HRD algorithm is useful in verifying that the H.265 and H.264 requirements on the decoder input stream buffer (namely, CPB) can be met. By default, the HRD feature is disabled.

- **HRD CPB size**: the size of the CPB used in the HRD model.

The CPB size is usually set to double the size of the target bit rate. The specified CPB size must be less than or equal to the maximum one defined in ITU-T Rec. H.265 according to the tier and level. Modifying this parameter is not recommended if you are unfamiliar with the HRD feature.

- **Bit rate window**: the number of frames within which the rate control algorithm tries to meet the target bit rate.

The bit rate window is used for matching the average bit rate across the specified number of encoded frames to the target bit rate. It affects how fast the encoder reacts to changes in the video sequence and encoded bits per frame. For example, a bit rate window of 30 frames on a 30 FPS stream matches the target bit rate every second. A bit rate window of 150 frames matches the target bit rate over five seconds.

- **GOP length**: the interval between two consecutive intra frames (I-frames).

GOP contains an I-frame and several inter frames (predictive frames). Intra frames allow for seekable and editable video streams. However, they occupy more bits, which decreases the overall stream quality. Whether an I-frame is inserted at the beginning of each bit rate window is determined by the application.

NOTE: gopSize parameters in the encoder API indicates the GOP structure size instead of GOP length.

- **Intra delta QP:** the QP delta for intra prediction.

Intra frames in HEVC (H.265) video streams may introduce noticeable flickering because of different prediction methods. This problem can be relieved by adjusting the quantization of intra frames compared to the surrounding inter frames.

2.5.2 Coding Control Settings

The encoder API provides you with `VCEncSetCodingCtrl()` for coding control.

- For encoder instances in INIT state, all the coding control parameters are configurable.
- For encoder instances in START_STREAM or START_FRAME state, some parameters are configurable between frames without the need to end the stream. However, to alter the parameters that impact whole-sequence syntax such as SPS and PPS, you need to restart the stream as follows:
 1. End the stream by calling `VCEncStrmEnd()`.
 2. Modify the parameter settings by calling `VCEncSetCodingCtrl()`.
 3. Start the stream by calling `VCEncStrmStart()`.
 4. Encode subsequent frames by calling `VCEncStrmEncode()`.

The coding control parameters that you can modify through `VCEncSetCodingCtrl()` are described as follows. Before you modify the parameter settings, it is recommended that you query the current settings by using `VCEncGetCodingCtrl()`.

- **Slice size:** the number of coding tree unit (CTU) rows in each slice.
Data of each encoded picture can be split into several independent packages, namely slices. Slicing is helpful in recovery of erroneous streams.
- **SEI messages:** the supplemental enhancement information (SEI) message feature.

- When this feature is enabled, a SEI message containing picture timing information is inserted into the stream before every encoded frame. If hypothetical reference decoder (HRD) is also enabled, buffering period information is inserted into the stream before every encoded instantaneous decoder refresh (IDR) frame.
- When this feature is disabled, the stream does not contain timing information. The timing information must be transferred some other way.

- **Video full range:** the input YUV data sample range, which is to be included in stream headers.

This parameter must be properly set. Otherwise, the video range may suffer degradation on the decoder side.

- **Disable de-blocking filter:** disables the de-blocking filter, thus lowering encoding and decoding complexity but also decreasing video quality.
- **Disable SAO filter:** disables the sample adaptive offset (SAO) filter, thus lowering encoding and decoding complexity but also decreasing video quality.
- **Sample aspect ratio width and height:** the aspect ratio of the input picture samples, to be included in stream headers.

The default aspect ratio is 1:1, which indicates square.

- **Field order** (for HEVC only): the field order, either bottom-first or top-first. The field order is valid only for the interlaced frame input format.

Some applications may assume a certain field order. You can set the field order according to the requirements of the application. The default value is 0, which indicates a bottom-first field order.

- **Region-of-interest (ROI):** specifies rectangular ROIs or ROI maps for differentiated encoding.

For details, see Section *Rectangular ROI*.

- **GDR:** the gradual decoder refresh (GDR) feature, which uses a series of intra areas in several frames instead of one I-frame as recovery points in the stream.

Intra areas gradually cover the whole frame area, which enables error recovery as an I-frame and spreads the transferring load of an I-frame over several frames.

2.5.3 Video Pre-Processing Blocks

The video encoder includes the following pre-processing blocks:

- **RGB-to-YUV conversion** (hardware configurable)

NOTE: This block is used to convert 30-, 24-, 16-, 15-, or 12-bit RGB to YUV420. Blocks after CSC may implement conversion from YUV422 to YUV420 or from I010, P010, planar YUV420 10-bit packed, or Y0L2 to YUV420.

- **Picture rotation**
- **Picture cropping**
- **Picture down-scaling**
- **In-loop down-scaling**
- **On-screen display (OSD)**

By default, the encoder expects input in planar YUV420 format and all other blocks are disabled. You can modify the settings of video pre-processing blocks by calling `VCEncSetPreProcessing()`. It is recommended that you query the current settings with `VCEncGetPreProcessing()` before updating the settings.

2.6 Stream Production

After a successful initialization and an optional configuration step, the encoder is ready to produce a stream with the following steps:

1. Start the stream by calling `VCEncStrmStart()`
2. Encode video frames by calling `VCEncStrmEncode()` for each frame
3. Flush unfinished encoding frames by calling `VCEncFlush()`
4. End the stream by calling `VCEncStrmEnd()`

2.6.1 Stream Starting

A stream must be started by a call to `VCEncStrmStart()`. This function generates stream headers with stream-level parameters as follows:

- For HEVC (H.265), video parameter set (VPS), sequence parameter set (SPS), and picture parameter set (PPS) NAL units are produced.

- For H.264 (AVC), SPS and PPS NAL units are produced.

The `VCEncStrmStart()` function requires an output buffer to which the software-generated stream header data is to be written. The buffer must be accessible by the CPU through its virtual address.

If the `VCEncStrmStart()` call is successful, it returns `VCENC_OK` and outputs the size of the generated stream in bytes through a specified `VCEncOut` structure.

2.6.2 Frame Encoding

A video sequence can be encoded after a stream is started. Then the `VCEncStrmEncode()` function is called once to feed the encoder with each frame.

- If the frame is successfully encoded, the function returns `VCENC_FRAME_READY`.
- If the frame is not immediately encoded due to re-ordering caused by a bi-directional predictive frame (B-frame), the function returns `VCENC_FRAME_ENQUEUE`.

In this case, after all input pictures are fed into the instance through `VCEncStrmEncode()`, you can flush the stream by calling `VCEncFlush()`. For details, see Section *Stream Flushing*.

The main inputs for `VCEncStrmEncode()` are described as follows:

- **Input buffer:** contains the current video frame to be encoded.

The input color format is specified using the `pRateCtrl` parameter in the `VCEncSetPreProcessing()` function. For the list of supported input color formats, see *Hantro VC9000E Series Memory Buffer and Format Organization*. Before selecting an input color format, you need to verify the hardware configuration through `EWLHwConfig_t`.

The input buffers must be linear. Proper alignment of buffer lines (for example, 128-bit alignment) is helpful in increasing the efficiency for the hardware fetching the input data.

Each input buffer must be hardware accessible. You need to provide the bus address of each input buffer for the hardware to read the input video frames.

NOTE: When MMU is selected in your sub-system, the memory allocated to the frame buffer is not required to be physically continuous. In this case, the MMU virtual address instead of the input buffer bus address needs to be provided.

- **Output buffer:** stores the generated stream.

The buffer size depends on the target bit rate and frame rate. In typical scenarios, a buffer size equal to the bit rate of the stream is recommended. However, a single encoded I-frame can take many times more space than the average. The maximum size of the encoded frame depends on the input picture and QP used to encode the picture.

If the encoder reaches the end of the output buffer, it discards the current frame and returns the error code `VCENC_OUTPUT_BUFFER_OVERFLOW`. You can implement your error handling logic by either re-encoding the current frame (only available for single-core configuration) or continuing with the next frame. This scenario is more likely to happen when the output buffer is excessively small.

- **Encoded picture count:** the index of each frame in the video sequence, relative to the first frame.

You can use the index to identify the encoded picture and maintain the timestamp of the encoded frame.

- **NAL unit size buffer:** receives the size of each NAL unit.

The encoder fills this buffer with the size of each NAL unit in bytes. After the last unit size, the encoder writes a zero value to the buffer.

The buffer must be large enough to hold all the size information for the maximum number of slices (NAL units) that the encoder can generate.

NOTE: The size information is needed for separating each NAL unit from the byte stream when the NAL unit stream format is used.

2.6.3 Stream Flushing

If `VCENC_FRAME_ENQUEUE` is returned for a `VCEncStrmEncode()` call, you can flush the stream by calling `VCEncFlush()`. `VCENC_FRAME_ENQUEUE` indicates that the encoding of some frames are not completed. The `VCEncFlush()` function drives the video encoder to finish encoding these frames.

You need to call `VCEncFlush()` once for each unfinished frame. If a `VCEncFlush()` call returns `VCENC_FRAME_READY`, there are still unfinished frames in the encoder queue. You need to call `VCEncFlush()` until `VCENC_OK` is returned, which means that no unfinished frame exists.

2.6.4 Stream Ending

A stream must be ended by a call to `VCEncStrmEnd()`. The stream then ends with a NAL unit of the end-of-sequence type.

If the `VCEncStrmEnd()` call is successful, it returns `VCENC_OK`.

2.7 Release of the Encoder

You need to release an encoder by calling `VCEncRelease()` at the end of the encoding process. This API safely releases all the resources allocated to the encoder when it is initialized.

3 Coding Control Features

This chapter describes the features of the VC9000E video encoder and the method of configuring each feature.

3.1 Coding Formats and Profiles

The video encoder supports YUV420 for HEVC/AVC/AV1/VP9 encoding and monochrome formats for HEVC/AVC encoding. For details about the encoder support for the monochrome format, see Section *Monochrome Support*.

You can select the coding format by using the `VCEncConfig.codedChromaIdc` field.

NOTE: The support for YUV422 as the coding format is under development.

3.1.1 Monochrome Support

HEVC (H.265) and H.264 (AVC) encoding supports monochrome, which encodes luma samples only without outputting chroma samples.

This feature is available only when the value of `EVLHwConfig_t.MonoChromeSupport` is 1.

You can apply monochrome encoding by setting `VCEncConfig.codedChromaIdc` to `VCENC_CHROMA_IDC_400`.

3.1.2 Automatic Level Detection

When `VCEncConfig.level` is set to 0, the smallest level that fits the current settings, including picture size, frame rate, initial bit rate, and CPB size, is automatically filled in the SPS. If no level fits the settings, the highest level (6.2) is selected and a warning is emitted. This is to support resolutions such as 8192x8192 and 8192x8640.

The automatic level detection is supported only for HEVC (H.265) and H.264 (AVC).

NOTE: The automatic level detection takes only the initial bit rate and CPB size set before `VCEncStrmStart()` into consideration.

3.1.3 High Efficiency Image File Format

This chapter describes how to save HEVC (H.265) or AV1 encoded images into files of High Efficiency Image File Format (HEIF), ISO/IEC 23008-12, with helper APIs. HEIF specifies the file format as a container to store individual images or image sequences.

To support the HEIF feature, the video encoder requires the third-party HEIF writer library for constructing the HEIF container. For information about how to build and use the library for the video encoder, see Section *Building Video Encoder with HEIF Enabled*.

3.1.3.1 Feature Support

Image Roles

Each image can be assigned one or more roles. The supported image roles are described as follows:

- **Coded image**

A coded representation of an image.

- **Derived image**

An image that is represented in a file by a given non-destructive operation on input images. The image can be obtained by performing the operation on the input images.

For more information, see Section *heifDerivedImage*.

- **Cover image**, also known as primary item

A representative image of the image items of the file. The cover image is displayed when no other information is available on the preference to display the image items of the file. The file can have only one cover image.

- **Thumbnail image**

A smaller-resolution representation of a master image. A master image can have multiple thumbnail images (for example, at different resolutions).

- **Auxiliary image**

An image that complements a master image. For example, an alpha plane or a depth map. An auxiliary image can assist in displaying the master image but is not typically displayed as such.

- **Master image**

An image that is not a thumbnail image or an auxiliary image. Typically represents a full-resolution displayable image.

- **Hidden image**

An image that is never displayed. For example, a hidden image can be present in the file as an input image for a derived image.

Derived Image Item Types

Derived images are categorized as follows based on the type of the given operation:

- **Identity derivation:**

The output image is derived from an input image with one or more of the following methods:

- Cropping the input image
- Rotating the input image by 90, 180, or 270 degrees
- Horizontally and/or vertically mirroring the input image

- **Image overlay:**

The output image is derived by overlaying a number of input images at given locations in a given order onto the canvas of the output image.

- **Image grid:**

The output image is derived by reconstructing grids of same-size input images in a given order onto the canvas of the output image.

Image Sequence Tracks

- **Coded image sequence** (only P-frames supported)
A coded representation of an image sequence.
- **Thumbnail image sequence**
A smaller-resolution representation of a master image sequence.
- **Master image sequence**
An image sequence that is not a thumbnail image sequence. A master image sequence typically contains full-resolution displayable images.

3.1.3.2 Building Video Encoder with HEIF Enabled

To use the HEIF feature, you need to build the video encoder with the HEIF writer library imported as follows:

1. Download the Nokia HEIF source code from <https://github.com/nokiatech/heif>.
2. Build the HEIF writer library as follows:

```
cd build
cmake ../srcs -G"Unix Makefiles"
make
cmake --build .
```

Note: This package supports HEVC only, if you want to save AVIF file, ask VeriSilicon to get a patch of it.

3. Set the following environment variable:

```
export EXT_HEIF_BASE=[heif_lib_path]
```

where `heif_lib_path` is the local path of the Nokia HEIF source code.

4. Build the video encoder as follows:

```
make [target] target=[configure] HEIF_SUPPORT=y
```

where `HEIF_SUPPORT=y` means to enable the HEIF writer.

3.1.3.3 Saving an Image or an Image Sequence to an HEIF File

The following figure shows the typical procedure for saving images or image sequences to an HEIF file. For details about the APIs used in the procedure, see [HEIF Writer API](#).

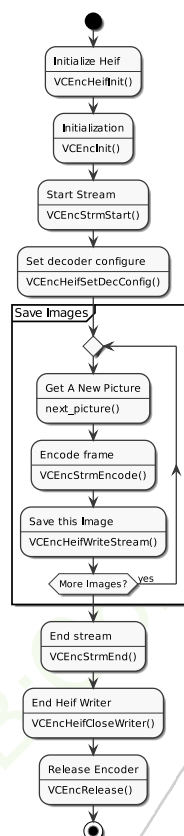


Figure 3.1 Saving Images or Image Sequences to an HEIF File

3.1.3.4 Saving Image Grids to an HEIF File

The HEIF feature supports splitting input images into smaller grids for HEVC (H.265) or AV1 encoding and saving the encoded grids into an HEIF file. The images can then be reconstructed from the grids based on HEIF information after each grid is decoded.

The following figure shows the procedure for saving image grids to an HEIF file. For details about the APIs used in the procedure, see [HEIF Writer API](#).

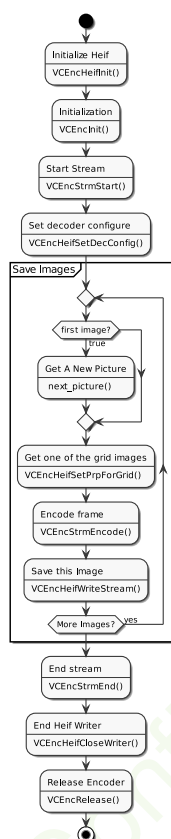


Figure 3.2 Saving Image Grids to an HEIF File

3.2 Quality Tuning

This section describes the quality tuning features that the video encoder supports for improving coding quality. However, quality improvement may decrease the IP performance.

3.2.1 RDO Level Selection

The video encoder allows for selecting a rate-distortion optimization (RDO) level to balance quality and throughput. The RDO level controls how many times the encoder runs to find the best partition and mode for encoding a CTB.

This feature is supported only for HEVC (H.265) and AV1. You can query whether RDO level selection is supported through the `EVLHwConfig_t.progRdoEnable` field.

You can either select a fixed RDO level or a dynamically selected RDO level.

- To use a fixed RDO level, you can set the `VCEncConfig.rdoLevel` field.
- To use a dynamically selected RDO level, you can set the `VCEncCodingCtrl.enableDynamicRdo` field.

The support for dynamic RDO level selection can be queried using `EVLHwConfig_t.dynamicRdoSupport`.

3.2.2 Look-Ahead Encoding

The video encoder supports look-ahead HEVC (H.265) and H.264 (AVC) encoding, which greatly improves video quality by analyzing the content ahead of real encoding, as shown in the following figure. To enable look-ahead encoding, you need to set `VCEncConfig.lookaheadDepth` in the `VCEncInit()` call to the number of look-ahead frames.

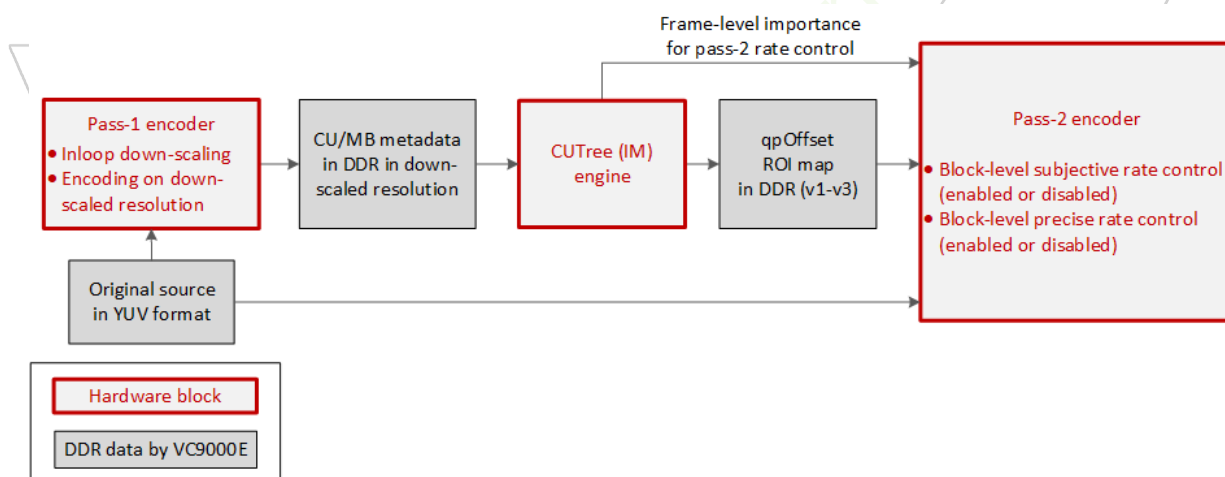


Figure 3.3 Two-Pass Look-Ahead Encoding

Look-ahead encoding contains three pipelined stages:

1. The pass-1 encoder encodes input frames in a down-sampled resolution as look-ahead frames, and generates CU- or MB-level metadata.
The video encoder supports a maximum of 40 look-ahead frames.
2. The CuTree analyzes the metadata in look-ahead frames, decides the importance of each CU or MB and the whole frame, and generates a QP delta for each CU or MB.
3. The pass-2 encoder encodes the input frames with the QP delta. The frame rate control is also impacted.

3.2.3 Rate-Distortion Optimized Quantization (RDOQ)

The video encoder supports rate-distortion optimized quantization (RDOQ) to improve the coding quality. However, the IP performance will decrease.

You can enable RDOQ by setting `VCEncCodingCtrl.enableRdoQuant` to 1. Before enabling this feature, you need to verify the feature support by querying the `RDOQSupportHEVC`, `RDOQSupportH264`, `RDOQSupportAV1`, or `RDOQSupportVP9` field of the `EWLHwConfig_t` structure, depending on the video stream format.

3.2.4 Tuning for Various Quality Targets

The video encoder supports quality tuning based on a PSNR, SSIM, visual, or sharpness-based visual target. The quality tuning is achieved by forcing some encoding parameters.

The target type is selected by using the `VCEncConfig.tune` field. Visual and sharpness-based visual targets are supported only if the value of `EWLHwConfig_t.encPsyTuneSupport` is 1.

3.2.5 Adaptive Quantization

The video encoder supports the adaptive quantization feature for improving subjective quality and SSIM.

This feature is enabled if `VCEncConfig.tune` is set to `VCENC_TUNE_SSIM`.

3.2.6 Anti Intra Flicker

Intra flicker is one kind of artifacts existing in video encoding system. The flicker is due to intra frame is inserted into the video sequence periodically. Since intra frame's prediction methods is different from P or B frame, such artifact is hard to remove. In hantro video encoding system, AIF(Anti Intra Flicker) feature is designed to reduce such kinds of artifact. In our design, the IDR frame is encoded twice. So when the feature is enable, the I frame will take double time to complete. Because IDR frame interval is quite large, e.g. one per 30 frames or one per 60 frames, the performance impact is small. AIF feature is useful to improve the subjective quality, especially when bit rate is low or middle and the scenario is full of slow-moving details.

To use this feature, hardware need to support tile 4x4 encoding. `EWLHwConfig_t.tile4x4FormatSupport` should be 1 to indicate it is supported.

This feature is conflicted with following features,

- 10 bit encoding: Due to hardware limitation, currently AIF cannot support 10-bit encoding.
- GDR(Gradual Decoder Refresh): GDR will change IDR frame to P frames. and current AIF only works for whole I frame.
- Parallel encoding, multi-tile encoding, lookahead: to implement AIF for parallel encoding, the two-pass encoding scheme need to adjust to avoid dependence change due to parallel jobs. Currently, it is not implemented.

To use this feature, you need to configure the video encoder as follows:

- Set `VCEncConfig.aifEnable` to 1.set when Call `VCEncInit()`;
- Set `VCEncCodingCtrl.aifQpDelta` to a proper value when Call `VCEncSetCodingCtrl()` before start encoding. This parameter specify the QP delta between the QP used to encode the IDR frame in first pass and the QP used to encode its previous P frame. Smaller delta value will get better quality for the IDR frame but cost more bits. suggest to use "-1" or "0" in general case.

3.3 Metadata Programming

3.3.1 External SEI Information

The video encoder allows you to specify external SEI information by using the `VCEncIn.externalSEICount` and `VCEncIn.pExternalSEI` fields. In order that the external SEI information can be written to a frame, you need to specify the two fields before calling `VCEncStrmEncode()` for the frame.

`VCEncIn.pExternalSEI` must be released after the `VCEncStrmEncode()` call is completed.

3.3.2 Interlaced Coding for HEVC

The video encoder supports interlaced coding for HEVC (H.265). To configure interlaced coding, you need to:

- Set `VCEncGopConfig.interlacedFrame` to 1 to enable interlaced coding.

- Set `VCEncCodingCtrl.seiMessages` to 1 to enable SEI output, because the interlace information is coded at picture timing SEI.
- Set `VCEncCodingCtrl.fieldOrder` to specify the order of top and bottom fields. The default value is 0, which indicates a bottom-first order.

Interlaced coding is supported for HEVC only if `VCEncIn.gopSize` is set to 1.

3.3.3 VUI Video Signal Type

When `VCEncCodingCtrl.vuiVideoSignalTypePresentFlag` is set to 1, the video signal type in the video usability information (VUI) is included in the VUI field in the sequence parameter set (SPS). By default, the video signal type is not included in the VUI field.

The VUI information includes the settings of `VCEncCodingCtrl.vuiVideoFormat`, `VCEncCodingCtrl.vuiVideoFullRange`, and `VCEncCodingCtrl.vuiColorDescription`.

NOTE: The VUI controls whether to display based on HDR parameters. For more HDR information, see Section *HDR Information Programming*.

3.3.4 HDR Information Programming

Two types of HDR information are available:

- Static HDR information: inserted into VUI or SEI of HEVC (H.265) and H.264 (AVC) streams.
- Dynamic HDR information: saved in T.35 packages of T.35 SEI packets.

You can use the following structures to configure HDR information:

- `Hdr10LightLevelSei`, which configures the fields for content light level information SEI messages.
- `Hdr10DisplaySei`, which configures the fields for mastering display color volume SEI messages.

For details about SEI messages, see ITU-T Rec. H.265.

3.3.5 HDR10+ or Dolby Vision Information Insertion

An HDR helper API `VCEncPutHDR10Info()` is designed to help write HDR information into the T.35 SEI packets of streams. The HDR information is parsed from a .json file compatible with HDR10+ and/or Dolby Vision. HDR10+ information for one frame in the JSON data can be identified by the frame ID. Such information is repeated until new information comes.

For definitions of HDR10+ and Dolby Vision, see the following documents:

- ST 2094-40:2020 - SMPTE Standard (HDR10+)
- ST 2094-10:2016 - SMPTE Standard (Dolby Vision)

3.4 Frame-Level Encoding

This section describes the coding syntax control features that the video encoder supports for frame-level encoding in order to adapt to various scenarios.

3.4.1 Skipped-Frame Insertion

You can enable the video encoder to encode all the CTBs and MBs in a frame as skipped blocks by setting `VCEncIn.bSkipFrame` to 1.

This feature is available only when the value of `EVLHwConfig_t.roiMapVersion` is greater than or equal to 2.

3.4.2 Reference Customization

The video encoder supports custom references for each frame. To customize the references, before calling `VCEncStrmEncode()`, you need to set fields in the `VCEncIn` structure as follows:

- Set `VCEncIn.flexRefsEnable` to 1 to enable external reference lists for frames.

- Set `VCEncIn.gopNextPicConfig` (for H.264) and `VCEncIn.gopCurrPicConfig` to custom the references.

These fields describe the reference management details and the method of selecting and generating reference frames.

For an example of how to set other `VCEncIn` fields, see the testbench function `FillInputOptionsForFlexRps()`.

Reference customization is supported only for HEVC (H.265) and H.264 (AVC) formats. It is available only if `VCEncCodingCtrl.gdrDuration` and `VCEncConfig.lookaheadDepth` are set to 0.

NOTE: It is recommended that you set `VCEncConfig.refFrameAmount` and `VCEncConfig.maxTLayers` to 4 (the maximum) to maximize the flexibility of the reference customization feature.

3.4.3 ROI-Based Coding Control

The video encoder supports coding control specific to two types of region of interest (ROI):

- Rectangular ROI
- ROI map

NOTE: For information about how the video encoder handles ROI-based coding control and block-level rate control, see Section *Relationship Between ROI-Based Coding Control and Block-Level Rate Control*.

3.4.3.1 Rectangular ROI

Description

The video encoder supports the following rectangular ROIs:

- A maximum of two or eight ROIs with specific QP delta or absolute QP values

The encoder supports a maximum of eight ROIs if the value of `EWLHwConfig_t.ROI8Support` is 1, and a maximum of two ROIs if the value is 0.

- A maximum of eight IPCM ROIs

In IPCM ROIs, no compression is applied to achieve no data loss. For details about IPCM, see Section *Intra Pulse Code Modulation*.

IPCM ROIs are available only if the value of `EVLHwConfig_t.IPCM8Support` is 1.

- An intra-mode ROI

Blocks inside the ROI are force encoded as intra blocks. When gradual decoder refresh (GDR) is enabled, the intra-mode ROI also serves as the GDR intra area for GDR refresh pictures.

NOTE: You can use the cyclic intra refresh (CIR) feature to specify the MBs or CTBs to be force encoded with the intra mode. For detail, see Section *sss_cir*.

Configurations

The fields used to configure ROIs of each type are listed as follows:

NOTE: If two areas of the same type overlap, the ROI with a greater ID has a higher priority. For example, `Roi2Area` takes precedence over `Roi1Area`.

- QP ROI
 - `VCEncCodingCtrl.roi1Area` to `VCEncCodingCtrl.roi8Area` define the eight ROIs.
If two areas overlaps, the ROI with a greater ID has a higher priority. For example, `Roi2Area` takes precedence over `Roi1Area`.
 - `VCEncCodingCtrl.roi1DeltaQp` to `VCEncCodingCtrl.roi8DeltaQp` define a QP delta value for each region.
You need to specify the QP delta based on the value of the slice QP.
 - `VCEncCodingCtrl.roi1Qp` to `VCEncCodingCtrl.roi8Qp` define an absolute QP value for each region.
You can assign a smaller QP value for ROIs and a larger QP value for other regions.
- IPCM ROI
 - `ipcm1Area` to `ipcm8Area` define the IPCM ROIs.
- Intra-mode ROI
 - `intraArea` defines the intra-mode ROI.

3.4.3.2 ROI Map

Description

The following two types of ROI maps are available:

- QP map used to specify QP ROIs, IPCM ROIs, and skip-mode ROIs
 - QP ROI, in which a QP delta or absolute QP value is forced
 - IPCM ROI, in which no compression is applied to achieve no data loss
IPCM ROIs are available only if the value of `EVLHwConfig_t.IPCM8Support` is 1.
 - Skip-mode ROI, in which encoding is skipped
The encoder encodes only the residual outside ROIs of this type.
- Coding unit (CU) control map used to specify CU control ROIs

Configurations

The fields used to configure ROI maps of each type are listed as follows:

- QP map
 - `VCEncCodingCtrl.roiMapDeltaQpEnable` controls whether to enable the QP map.
 - `VCEncIn.roiMapDeltaQpAddr` specifies the bus address of the QP map buffer through which the encoder accesses the buffer.
 - `VCEncIn.pRoiMapDelta` specifies the virtual address of the QP map buffer through which the CPU accesses the buffer of the QP map.
 - `VCEncIn.roiMapDeltaSize` specifies the size of the ROI map.
 - `VCEncCodingCtrl.roiMapDeltaQpBlockUnit` specifies the block unit size of the QP map.
 - `VCEncCodingCtrl.RoiQpDelta_ver` specifies the QP information format used in the QP map.
For format details, see Section *QP/CU Information Formats*.
- CU control map
 - `VCEncCodingCtrl.RoimapCuCtrl_enable` controls whether to enable the CU control map.

- `VCEncIn.RoimapCuCtrlAddr` specifies the bus address of the CU control map buffer.
- `VCEncCodingCtrl.roiMapDeltaQpBlockUnit` specifies the block unit size of the CU control map.
- `VCEncCodingCtrl.RoimapCuCtrl_ver` specifies the CU information format used in the CU control map.

For format details, see Section *QP/CU Information Formats*.

3.4.3.3 QP/CU Information Formats

For the video encoder, seven QP/CU information formats are defined for ROI maps:

- Formats 1 to 3 are QP information formats.
- Formats 4 to 7 are CU information formats.

The support for formats varies according to the value of `EVLHwConfig_t.roiMapVersion`. For more details about the formats, see the *ROI Map Buffer in External Memory* section in *Hantro VC9000E Series Memory Buffer and Format Organization*.

NOTE: A helper API `VCEncFillRoiMap()` is available for automatically filling the map buffer of formats 1 to 4 according to the configurations in `RoiRect`.

3.4.4 Cyclic Intra Refresh

The video encoder supports cyclic intra refresh feature to force encoding blocks (MBs or CTBs) with the intra mode at a given interval.

To configure the CIR feature, you need to set the following fields:

- `VCEncCodingCtrl.cirInterval`

If this field is set to 0, CIR is disabled.

If this field is set to a non-zero value, CIR is enabled and the field indicates the interval between two intra blocks.

- `VCEncCodingCtrl.cirStart`

This field specifies the index of the first block to be encoded as intra mode.

If CIR is enabled, the encoder encodes MBs or CTBs with the order count of $(VCEncCodingCtrl.cirStart + n * VCEncCodingCtrl.cirInterval)$ in intra mode. n is an integer in the range of $[0, N]$ where N is calculated by $[(\text{Number of blocks in each frame} - VCEncCodingCtrl.cirStart - 1) / VCEncCodingCtrl.cirInterval]$.

3.4.5 Intra Pulse Code Modulation

The video encoder supports intra pulse code modulation (IPCM) as a lossless encoding tool. Specifically, IPCM allows the video encoder to bypass intra-prediction, transform coding processes, and finally code input pixels into streams without transformation or quantization.

The encoder supports multiple IPCM ROIs in which all coding units (CUs) are coded as IPCM CUs for both HEVC (H.265) and H.264 (AVC). All IPCM ROIs must be aligned to 64x64 pixel block for HEVC or 16x16 pixel block for AVC. For details about IPCM ROIs, see Section *ROI-Based Coding Control*.

NOTE: IPCM is designed for cases where data expansion happens during the encoding process, for example, when the input picture contains a region full of random noise, while it is used by the video encoder as a tool for coding input data in a lossless way.

3.4.6 Stream-Level Encoding Control

The video encoder allows dynamic update of stream-level encoding control settings via the `VCEncSetStrmCtrl()` function. After this function is called, it inserts the new VPS, SPS, or PPS to update the settings.

Currently, `VCEncSetStrmCtrl()` can only update the resolution. The resolution of a new stream can be updated through `VCEncStrmCtrl.width` and `VCEncStrmCtrl.height`. The updated resolution must be lower than the resolution specified in the `VCEncConfig`.

The `VCEncSetStrmCtrl()` function is applicable only to encoder instances in INIT state. If an encoder instance is in START_STREAM or START_FRAME state, you need to restart the stream as follows to apply the new parameter settings:

1. End the stream by calling `VCEncStrmEnd()`.
2. Modify the parameter settings by calling `VCEncSetRateCtrl()`.
3. Start the stream by calling `VCEncStrmStart()`.
4. Encode subsequent frames by calling `VCEncStrmEncode()`.

NOTE:

- Do not change the allocated buffers while updating the resolution. This avoids memory fragments caused by frequent memory allocation and release of memory blocks.
- More stream-level settings defined in `VCEncSetCodingCtrl()` are planned to be moved to the `VCEncSetStrmCtrl()` function.

3.5 Error Concealment Encoding Tools

3.5.1 Multi-Slice Encoding

The video encoder enables multi-slice encoding when the `sliceSize` parameter in the `VCEncCodingCtrl()` call is set to a non-zero value. This parameter indicates the number of CTB or MB rows in each slice, while a slice may contain fewer rows.

In software, `VCEncSliceReadyCallbackFunc()` can be registered for processing slice data when one or more slices are generated. To facilitate the callback function, encoded slices can be collected through the `VCEncSliceReady` structure.

3.5.2 VPS/SPS/PPS Re-Sending

When `VCEncIn.resendVPS`, `VCEncIn.resendSPS`, or `VCEncIn.resendPPS` is set to 1, the corresponding parameter set is inserted before the slice data.

3.5.3 Frame Re-Encoding

A frame can be re-encoded when `VCEncConfig.cb_try_new_params` is set.

In software, `VCEncTryNewParamsCallbackFunc()` can be registered for obtaining the latest encoding parameter settings and accordingly re-encoding the frame when `VCENC_OUTPUT_BUFFER_OVERFLOW` is returned.

3.6 Low-Latency Encoding

The encoder hardware supports low-latency encoding with software synchronization by employing short-term buffering of video data in memory. Low-latency encoding is designed for real-time videos.

The video encoder provides two low-latency encoding modes:

- Conventional mode
- Segment buffer interface (SBI) mode

The SBI mode is available only if the value of `EWLHwConfig_t.prpSbiSupport` is 1. If the SBI mode is available, you can enable it by setting `VCEncCodingCtrl.inputLineBufEn` to 1 and set the initial segment ID for each frame through the test bench.

3.6.1 Low-Latency System Requirements

- The input is in any of the supported color format.
- The input picture buffer stores the entire input picture as it does for normal encoding.
- The output stream buffer stores the entire output stream of a picture as it does for normal encoding.
- The application software is aware of the number of pixels available in the input picture buffer.
- (For conventional mode) Synchronization and hardware handshake are supported.
- (For SBI mode) Synchronization and hardware handshake through Flexa SBI are supported.

3.6.2 Low-Latency Synchronization

Low-latency encoding requires synchronization between the encoder software and both its upstream and downstream.

Synchronization with Upstream

The encoder synchronizes with the upstream in steps of stripe. The software pushes one stripe at a time to the encoder. A stripe is defined as follows for HEVC (H.265) and H.264 (AVC):

- HEVC:
 - One stripe contains multiple 64x64 coding tree block (CTB) rows.
 - The CTB row width is the picture width in the unit of CTB.
 - The number of CTB rows is a positive integer.
 - The rightmost CTB in a CTB row can be a partial CTB.
There is no need for pixel padding in the input picture buffer for partial CTBs.
 - The stripe height is constant throughout the picture except for the last stripe of the picture. The last stripe at the bottom of the picture consists of the remaining CTB rows. This means that the height of the last stripe can be less than or equal to the constant stripe height.
- H.264:

- One stripe contains multiple macroblock (MB) rows.
- The MB row width is the picture width in the unit of MB.
- The number of MB rows is a positive integer.
- The stripe height is constant throughout the picture except for the last stripe of the picture. The last stripe at the bottom of the picture consists of the remaining MB rows. This means that the height of the last stripe can be less than or equal to the constant stripe height.

Synchronization with Downstream

The encoder synchronizes with the downstream in steps of chunk. The software reads the number of encoded chunks from the output buffer and the information of slice boundary. A chunk is defined as follows:

- Before the end of a picture, a chunk is 256 bytes of the encoded stream.
- At the end of a picture, a chunk contains the remaining part of the encoded stream, which can be 256 bytes or less.

At each slice end, there is an additional synchronization point within a normal chunk. This point indicates the slice boundary in the encoded stream.

3.6.3 Low-Latency Encoding Flows

Input Picture Encoding Flow

1. At the start of a picture, the encoder software sets up the stripe size and other picture-level settings in software registers.
2. The system provides the application software with the number of CTB/MB rows available in the current input picture buffer.
3. The application software sends the number of available CTB/MB rows to the encoder software by calling the encoder software API.
4. The encoder software updates the number of available CTB/MB rows in the appropriate register.
5. The encoder checks the number of available CTB/MB rows at the start of the picture and each time it finishes reading a stripe.

- If new stripes are available, the encoder encodes the new pixels.
 - If no new stripe is available, the encoder waits for new input data to arrive.
6. When the encoder finishes reading a stripe, it generates a stripe ready interrupt.
- The encoder continues encoding as long as new stripes are available, without waiting for responses from the software to the stripe ready interrupt.

Output Stream Encoding Flow

1. The encoder increments a software-readable counter each time it writes a chunk of the encoded stream to the output buffer.
The counter is reset at the start of a picture. At the end of the picture, the counter indicates the total amount of encoded data.
2. (Optional) If multi-slice encoding is enabled, the encoder generates a slice ready interrupt when it finishes encoding a slice and flushes the encoded bits to the DDR. It also updates the encoded slice size to a corresponding DDR buffer.
The encoder continues encoding the next slice without waiting for responses from the software to the slice ready interrupt.
3. When the encoder finishes encoding the entire picture and flushes the encoded bits to the DDR, it generates a frame ready interrupt.
The encoder detects the picture end based on the picture size information.

3.7 Encoding Statistics Output

3.7.1 CU/MB Statistics Output

The video encoder can output CU/MB statistics to external memory while encoding at the same time.

You can use the following fields for configuring the CU/MB statistics output:

- `VCEncConfig.enableOutputCuInfo` controls whether to enable the CU/MB statistics output feature.

- `VCEncConfig.cuInfoVersion` specifies the format of statistics data to be output. You can query the support for output CU/MB statistics formats through `EWLHwConfig_t.cuInforVersion`. For details about the formats, see *Hantro VC9000E Series Memory Buffer and Format Organization*.
- `VCEncConfig.dumpCuInfo` controls whether to dump CU/MB statistics into files `cuInfo.txt` and `cuInfo_p1.txt` for debugging use. The `cuInfo_p1.txt` file is generated only for the pass-1 encoder of the look-ahead feature.

3.7.2 Bit Statistics Output

The video encoder can output the number of bits in each CTB or MB to an external buffer.

Bit statistics output is supported only if the value of `EWLHwConfig_t.CtbBitsOutSupport` is 1.

When the bit statistics output feature is supported, you can use the following fields for feature configuration:

- `VCEncConfig.enableOutputCtbBits` controls whether to allocate buffers for storing bit statistics.
- `VCEncConfig.enableOutputCtbBits` controls whether to enable the bit statistics output feature for the current frame.
- `VCEncConfig.dumpCtbBits` controls whether to dump bit statistics into the `ctbBits.txt` file for debugging use.

3.7.3 Frame Statistics Output

The video encoder can output frame-level statistics to external memory. The feature is supported only if the value of `EWLHwConfig_t.frameInfoSupport` is 1.

When the frame statistics output feature is supported, you can use the following fields for feature configuration:

- `VCEncConfig.enableFrameInfo` controls whether to enable the frame statistics output feature.

- `VCEncCodingCtrl.rect0` to `VCEncCodingCtrl.rect7` defines a maximum of eight regions for SSE statistics.
- `VCEncOut.frameInfo` specifies the buffer to store the output.

For details about the output frame statistics, see *Hantro VC9000E Series Memory Buffer and Format Organization*.

3.8 Miscellaneous Features

3.8.1 Reference Frame Compression

Reference frame compression (RFC) can save bandwidth by storing reference frames in a lossless compressed format. However, in the worst case, the storage memory may not be reduced. Instead, additional small buffers are used to store the compression metadata.

You can enable RFC for all components by setting to `VCEncConfig.compressor` to 3, and disable the compression by setting `VCEncConfig.compressor` to 0. Other values are invalid.

3.8.2 Reference Ring Buffer

In some application, memory footprint is important for the system cost. For encoder, reference frame is the largest memory requirement in general. Normally, buffers for two reference frames are required to encode P frame. One is used to write current reconstruction frame, another is used as the reference to do motion estimation. Since motion estimation is limited in fixed vertical range, the reference data out of the search range can be overwritten if they are not used. So ring-buffer is designed to store reference frame to save reference frame buffer's memory footprint. Instead of two reference frames, one reference frame plus some reference rows memory is used to encode P frames.

Currently, we use extra 64 row buffer. So the total reference frame buffer is one whole frame and 64 rows. When reading or writing such buffer and accessing to the end of the buffers, the read and write address will roll-back to the start of the buffer. The ring-buffer includes both luma reference buffer (with its downsample data) and chroma reference buffer.

NOTE: When reference compression is enable, the table is not work as ring-buffer.

The feature is supported only if the value of `EVLHwConfig_t.refRingBufEnable` is 1.

You can enable reference ring buffer by setting to `VCEncConfig.refRingBufEnable` to 1, and disable the compression by setting `VCEncConfig.refRingBufEnable` to 0. Other values are invalid.

NOTE: Because the reference buffer is overwritten by current reconstruction frame, following features cannot be supported now.

- B Frame, P frame reference more than one frame and LTR
- lookahead
- re-encode which need previous reference frame.
- multi-core and parallel encoding

3.8.3 On Scene Change

3.8.3.1 Scene Change Detection

The video stabilization hardware module can be used to detect scene changes. When video stabilization is enabled and the input picture resolution is the same as the encoded picture resolution, the video stabilization module performs scene change detection. The result is written in `VCEncOut.nextSceneChanged`.

This feature is available only if the value of `EVLHwConfig_t.vsSupport` is 1.

For more information about video stabilization, see Section *Video Stabilization*.

3.8.3.2 External Scene Change Indication

When `VCEncIn.sceneChange` in an API is set to 1, the video encoder considers the current frame as a scene change frame. The API updates the rate control parameters, including the scene change frame flag.

3.8.4 Security Encoding

The video encoder supports security encoding as a system feature in the digital rights management (DRM) architecture.

When security encoding is enabled, memory is divided into the security region and a non-security region. The encoder hardware is allowed only to access data in the non-security region, and cannot access data in the security region.

NOTE: When security encoding is enabled, the encoder software still can properly run in the rich execution environment (REE) if it does not need to access data in the security mode.

At the encoder wrapper layer (EWL), the memory allocator API `EWLMallocLinear()` provides a flag to indicate whether the allocated buffer locates in the security or non-security region.

- When security encoding is enabled, the flag needs to be set according to the usage of the buffer.
- When security encoding is disabled, all buffers locate in non-security region.

You need to overwrite the memory allocator when integrating the encoder software to your system and perform memory allocation by using the flag.

The security encoding feature is available only if the hardware integrates this feature. To enable security encoding, you can set `VCEncConfig.secure_mode` to 1.

3.8.5 Batch Mode Encoding

Batch mode encoding will aggregate multiple encoding tasks in software and trigger hardware to start the encoding tasks after the aggregation achieve some threshold. In such way, the hardware can work on the tasks one by one without interrupt until the last one. User can use this feature to reduce hardware's power because the hardware can sleep longer if the tasks are excuted in short time. At the same time, the software can reduce interrupt process for only one interrupt is processed when a batch of tasks is done.

This feature has following limitation,

- It is only valid for VCMD because software must be involved to process frame done IRQ.

- To reduce software complexity, batch mode is invalid for lookahead
- It is disabled automatically if multi-instance is detected. In such condition, the hardware cannot save power because it is hard to find proper timing to power off the codec.

To use this feature, you need to configure the video encoder as follows:

- Set `VCEncConfig.enc_mode.batch_flag` to 1 when Call `VCEncInit()`.
- Set `VCEncConfig.parallelCoreNum` as the max tasks number that the software can aggregate.
- Set `VCEncCodingCtrl.batchCount` as the threshold to trigger the hardware to execute the aggregated tasks. the default threshold is `VCEncConfig.parallelCoreNum-1` and it is the maximum number the `batchCount` can be set. This parameter can be changed before call `VCEncStrmEncode`.

The return value of `VCEncStrmEncode` reports the status of task aggregation.

- `VCENC_FRAME_CACHE`: the task is cached and not send to hardware;
- `VCENC_FRAME_ENQUEUE`: the task(s) are send to hardware;
- `VCENC_FRAME_DONE`: there're task(s) done.

A new API `VCEncStrmGetOutput()` is added to get the result of task one by one after they are send to hardware. In batch mode, user should call this API when `VCEncStrmEncode` return `VCENC_FRAME_ENQUEUE`.

4 Bit Rate Control Features

The video encoder can control the bit rate at both picture and block (CTB or MB) levels.

- At picture level, the bit rate is controlled by the software adjusting the QP of each frame.
- At block level, the bit rate is controlled by the hardware adjusting the QP of each block inside frames.

4.1 Picture-Level Rate Control

The video encoder supports various rate control modes at the picture level.

The following sections list the recommended settings for each mode.

4.1.1 Constrained Variable Bit Rate

Constrained variable bit rate (CVBR) is the default rate control mode of the video encoder. In this mode, encoded bits in slow-motion and simple scenes are reduced, and the reduced bits are used for complex scenes to improve overall quality.

To select the CVBR mode, you need to configure the video encoder as follows:

- Set `VCEncRateCtrl.rcMode` to CVBR.
- Set `VCEncRateCtrl.bitPerSecond` to the target bit rate.

4.1.2 Average Bit Rate

In average bit rate (ABR) mode, the video encoder tunes QP based on the gap between the actual and target bit rates instead of scene complexity. This ensures the average bit rate approximates the target bit rate.

To select the ABR mode, you need to configure the video encoder as follows:

- Set `VCEncRateCtrl.rcMode` to ABR.
- Set `VCEncRateCtrl.bitPerSecond` to the target bit rate.

4.1.3 Constant Bit Rate

Constant bit rate (CBR) is a mode that ensures output at a constant bit rate to the transmission path.

NOTE: If HRD is also enabled, HRD information is inserted into stream SEI messages with other rate control behaviors unchanged.

To select the CBR mode, you need to configure the video encoder as follows:

- Set `VCEncRateCtrl.rcMode` to CBR.
- Set `VCEncRateCtrl.bitPerSecond` to the target bit rate.
- Set `VCEncRateCtrl.hrdCpbSize` to the maximum allowed size of an encoded frame in bits.
 - A larger size limit helps avoid data overflow.
In this case, to avoid data underflow, set `VCEncRateCtrl.hrd` to 1.
 - A smaller size limit generates a more stable bit rate, but increases the risk of data overflow.

A value of $(2 \times \text{VCEncRateCtrl.bitPerSecond})$ is recommended, which is also the default value. Values less than `VCEncRateCtrl.bitPerSecond` are not recommended.

4.1.4 Variable Bit Rate

The variable bit rate (VBR) mode allows for bit rate control with limitations on the minimum QP and maximum bit rate.

- If the real bit rate is less than the maximum bit rate, the video encoder decreases QP until the minimum QP is reached.
- If the real bit rate exceeds the maximum bit rate, the video encoder increases QP.

To select the VBR mode, you need to configure the video encoder as follows:

- Set `VCEncRateCtrl.rcMode` to VBR.
- Set `VCEncRateCtrl.qpMinPB` and `VCEncRateCtrl.qpMinI` to the minimum QP values at which video quality meets expectation for P/B- frames and I-frames, respectively.
- Set `VCEncRateCtrl.bitPerSecond` to the maximum bit rate.

4.1.5 Constant Rate Factor

The constant rate factor (CRF) mode ensures a certain level of quality with a constant slice QP derived from a given rate factor and frame rate.

The CRF mode is available only if look-ahead encoding is enabled.

To select the CRF mode, you need to configure the video encoder as follows:

- Set `VCEncRateCtrl.rcMode` to CRF.
- Set `VCEncRateCtrl.crf` to 22.
- Set `VCEncConfig.lookaheadDepth` to 20.

4.1.6 Constant Quantization Parameter

The video encoder supports the constant quantization parameter (CQP) mode where a constant QP value is used for all slices, with a separate QP delta for I slices.

To select the CQP mode, you need to configure the video encoder as follows:

- Set `VCEncRateCtrl.rcMode` to CVBR.
- Set `VCEncRateCtrl.qpHdr` to 22.

4.2 Block-Level Rate Control

Block-level rate control supports two types of targets: one for subjective rate control and the other for precise rate control. The two targets can be used together to achieve a balance between subjective and precise bit rates.

To control block-level rate control, you can:

- Set `VCEncRateCtrl.ctbRc` to control whether to enable block-level rate control.
- Set `VCEncConfig.ctbRcMode` to select the target for block-level rate control.
- Set `VCEncRateCtrl.tolCtbRcInter` and `VCEncRateCtrl.tolCtbRcIntra` to specify the tolerance of block-level rate control for inter and intra frames, respectively.
- Set `VCEncRateCtrl.ctbRcRowQpStep` to the maximum accumulated QP adjustment step per CTB/MB row.
- Set `VCEncRateCtrl.rcQpDeltaRange` to the maximum absolute delta between block-level QP delta and picture-level QP value.

4.3 Relationship Between ROI-Based Coding Control and Block-Level Rate Control

The video encoder handles ROI-based coding control and block-level rate control as follows:

1. Calculates the target QP of a block according to the ROI configurations if ROI-based coding control is enabled.
If ROI is disabled, the encoder loads the slice QP as the target QP.
2. Runs the block-level rate control algorithm to calculate the QP delta based on the slice QP if the algorithm is enabled.
If block-level rate control is disabled, the QP delta equals 0.
3. Adds the QP delta to the target QP, and performs clipping between the minimum QP and the maximum QP.
The clipping result is regarded as the final QP for this block.

NOTE: You can enable or disable ROI-based coding control and block-level rate control separately. For details, see Section *ROI-Based Coding Control* and Section *Block-Level Rate Control*.

5 Pre-Processing Features

5.1 RGB-to-YUV Conversion

The optional color space conversion (CSC) block reads the data of each picture in 30-, 24-, 16-, 15-, or 12-bit RGB. It then converts the picture data into planar YUV420 format by sub-sampling chrominance samples vertically and rearranging the samples.

The input RGB pixel is converted into YUV by using the following equations:

$$Y' = A \times r + B \times g + C \times b$$

$$Y = clip(Y' + LumaOffset)$$

$$U = clip(E \times b - G \times Y' + Const)$$

$$V = clip(F \times r - H \times Y' + Const)$$

where:

- A, B, C, E, F, G, and H are color conversion coefficients in Q0.16 fixed-point format.
 - If `VCEncColorConversion.type` is set to `VCENC_RGBTOYUV_USER_DEFINED`, you need to specify custom coefficients in `VCEncColorConversion`.
 - If `VCEncColorConversion.type` is set to another value, coefficients predefined based on ITU-T recommendations are used.
- LumaOffset is the color conversion offset, which is supported only if the value of `EWLHwConfig_t.cscExtendSupport` is 1.
- Const is the constant offset for chroma, which is 128 for 8-bit encoding and 512 for 10-bit encoding.
- clip() is the function that clips the result into the range of corresponding bit depth.

The CSC block is configured using the `colorConversion` parameter in the `VCEncSetPreProcessing()` function.

5.2 Picture Rotation

The picture rotation block rotates each picture 180 or 90 degrees clockwise or counterclockwise. The following figure illustrates the resultant encoded picture dimensions after rotation.

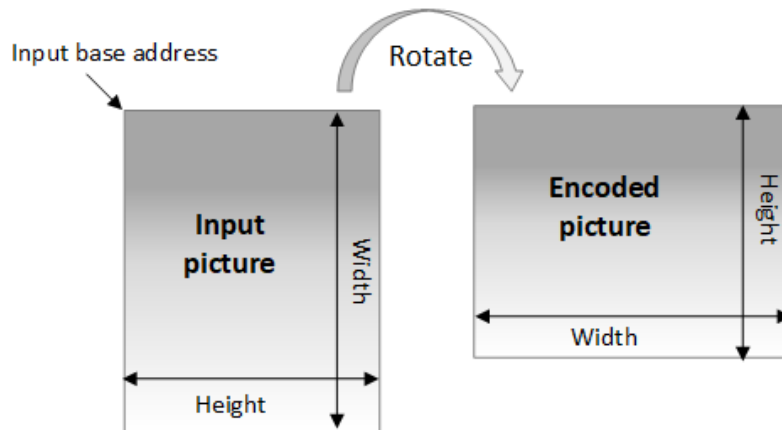


Figure 5.1 Picture Dimensions After Rotation

The picture rotation feature is supported only if the value of `EVLHwConfig_t.NonRotationSupport` is 0. When rotation is supported, you can control it by using the `rotation` parameter in the `VCEncSetPreProcessing()` function.

5.3 Picture Cropping

The picture cropping block allows reading and encoding a selected portion of a picture.

The crop area to be encoded is specified by setting the size of the input picture, the size of the crop area, and the top-left corner coordinates of the crop area relative to the top-left corner of the input picture, as shown in the following figure. The maximum size of the input picture is determined by the configurations through `VCEncPreProcessingCfg.origWidth` and `VCEncPreProcessingCfg.origHeight`.

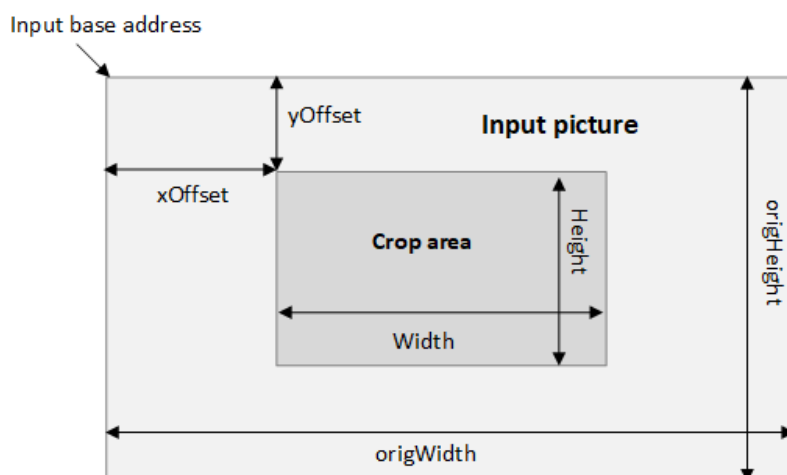


Figure 5.2 Input Picture and Crop Area

The picture cropping block requires the following conditions on the width, height, and their offsets of the crop area:

$$origWidth - xOffset - width \geq 0$$

$$origHeight - yOffset - height \geq 0$$

The input picture horizontal stride in pixels must be a multiple of 2. For example, when the input width is 639 pixels, the horizontal stride must be 640 pixels.

5.4 Offline Down-scaling

The picture scaler receives the same picture as the encoder and down-scales the picture. The down-scaled picture is then written to external memory so that it can be accessed by the software for low-resolution encoding and preview.

The down-scaling feature is supported only if the value of `EWLHwConfig_t.scalingEnabled` is 1.

When down-scaling is supported, you can configure the down-scaling feature by using the `VCEncSetPreProcessing()` function as follows:

- To enable the feature, set `VCEncPreProcessingCfg.scaledOutput` to 1.
- To specify the resolution of the down-scaled picture, set `VCEncPreProcessingCfg.scaledWidth` and `VCEncPreProcessingCfg.scaledHeight`.

- To configure the buffer to store the down-scaled picture data:
 - Set `VCEncPreProcessingCfg.busAddressScaledBuff` to the buffer physical address for the hardware to write data.
 - Set `VCEncPreProcessingCfg.virtualAddressScaledBuff` to the buffer virtual address for the software to read data.
 - Set `VCEncPreProcessingCfg.sizeScaledBuff` to the size of the buffer.
- To specify the format of the down-scaled picture, use `VCEncPreProcessingCfg.scaledOutputFormat`.

The format can be either interleaved YUV422 or semi-planar YUV420, depending on the hardware configuration. To query the hardware configuration, you can use the `EVLHwConfig_t.scaled420Support` field.

5.5 In-Loop Down-Scaling

The look-ahead encoding block integrates an in-loop down-scaler in the pass-1 encoder.

The in-loop down-scaler receives the same picture as the encoder and performs a down-scaling operation pipelined with the encoder, without writing the down-scaled picture to DDR.

The scaling ratio can be either 1:1 or 1:2, depending on the setting of the `inLoopDSRatio` parameter in the `VCEncSetPreProcessing()` function.

5.6 On-the-Fly OSD Mixer

The video encoder supports on-screen display (OSD) for blending input pictures with OSD regions. The generated blended picture is fed into the encoder as final input.

OSD regions are rectangular regions with OSD input in a certain format. The video encoder regards OSD regions as a foreground to blend OSD regions with the input picture based on the following alpha blending algorithm. OSD regions should not overlap or share CTB due to hardware restrictions.

$$result_pixel = \begin{cases} overlay_pixel & \text{Alpha} == 255 \\ (overlay_pixel * Alpha + background_pixel * (256 - Alpha)) >> 8 & \text{Alpha} \neq 255 \end{cases}$$

This feature is available only if the value of `EVLHwConfig_t.OSDSupport` is 1.

When the OSD mixer is supported, you can use the `VCEncPreProcessingCfg.overlayArea` field to define OSD regions.

5.7 Video Stabilization

The video encoder supports digital video stabilization for detecting and compensating unexpected jitter effects on the video, while expected effects such as panning are maintained. The video stabilization feature is available only if the value of `EVLHwConfig_t.vsSupport` is 1.

Video stabilization requires the current frame and the next frame as its input. During the encoding of the current frame, the video stabilization feature calculates the offset of the next frame, where only small movements are detected. The offset is used to select the encoded part of the next frame. The offsets around the stabilized picture are cropped out.

The current frame and the next frame are stored in two separate input picture buffers. For the next frame, only luma samples are used in the encoding process. Therefore, you need to specify the physical address of the next-frame luma sample data by using `VCEncIn.busLumaStab`.

The input picture of video stabilization must be at least 8 pixels larger than the actual resolution of the picture to be encoded.

5.8 Osdmap

OSD Map is a buffer specified by the user, where every 4 bits correspond to a pixel's color and alpha value (up to 12 colors and alpha values). Then the user can mix the specified area with alpha and the original image according to the specified color in the Map Buffer, and the mixed result is used as the input for encoding. This function can be used to scramble any area. For example, in car applications, color blocks can be applied to the facial part. In the past, the mosaic function could only support up to 12 regions of mosaic in VCE, and using OSD Map did not have this limitation.

The Data in OSD Map buffer are defined as following:

- Each 8x8 block corresponds to a 4-bit control information. The value of these 4-bits determines the color on this 8x8 stack.
- The MSB of one byte defines the block before the block defined in the LSB.

The specific 4-bit control information encoding is as follows:

- 4'd0: Do not stack any information
- 4'd1: Overlay the first color and alpha configuration
- 4'd2: Overlay the second color and alpha configuration
- 4'd8: Overlay the eighth color and alpha configuration
- 4'd12: Overlay the twelfth color and alpha configuration
- 4'd13~4'd15: Invalid

Note:

- Comparing with overlay and mosaic, OSDMap has the lowest priority.
- OSDMap and overlay supertile should not be both turned on at the same time.
- The sum of available colors in the OSD area and OSDMap cannot be greater than 12, because OSDMAP and OSD share registers to represent the corresponding colors in the Map.
- It is not recommended to open both OSDMAP and Mosaic functions simultaneously, because the goals of OSDMAP and Mosaic functions are similar.
- You can get information on whether this feature is supported from `EWLHwConfig_t.OSDMapSupport`. When this value is 1, it indicates support.

To use this feature, you need to configure the video encoder as follows:

- you can set the `VCEncPreProcessingCfg.osdMapEnable` to 1 to enable osdmap feature.
- Set `VCEncPreProcessingCfg.osdMapStride` to OSD Map stride in byte. OSDMap 4bits perform 8x8 or 4x4 block, so `osdMapStride` should not less than $(codingWidth + 15) >> 4$ or $(codingWidth + 7) >> 3$.
- Set `VCEncPreProcessingCfg.osdMapBlockSize` to OSD Map block size index. 8 means 8x8, 4 means 4x4.
- Set `VCEncPreProcessingCfg.osdMapAlpha[i]` to the alpha value for color *i*.
- Set `VCEncPreProcessingCfg.osdMapY[i]` to the OSD Map format color[i] Y value(0..255).
- Set `VCEncPreProcessingCfg.osdMapU[i]` to the OSD Map format color[i] U value(0..255).
- Set `VCEncPreProcessingCfg.osdMapV[i]` to the OSD Map format color[i] V value(0..255).

6 GOP Structure Configuration

6.1 Basic GOP Structures

In order to construct a GOP structure, the following inputs for each frame need to be fed through `VCEncGopPicConfig` to the video encoder:

- **Frame ID:** the ID of the frame, which is used in the configuration file to mark whether the frame is configured using `VCEncGopPicSpecialConfig` or `VCEncGopPicConfig`.
 - 0: The frame is configured using `VCEncGopPicSpecialConfig`.
 - Other values: The frame is configured using `VCEncGopPicConfig`.

In Section *Basic GOP Structures*, all frames are configured through `VCEncGopPicConfig`.

- **codingType:** the slice type.
 - I: `VCENC_INTRA_FRAME`
 - P: `VCENC_PREDICTED_FRAME`
 - B: `VCENC_BIDIR_PREDICTED_FRAME`
- **poc:** the picture order count of the frame.
- **QpOffset:** the offset to be added to the QP parameter to obtain the final QP.
- **QpFactor:** the factor used by rate distortion optimization.

A higher value means lower quality and fewer stream bits. A typical range is from 0.3 to 1.
- **temporalId:** the temporal layer ID.
- **numRefPics:** the number of reference frames kept for the current frame.

The reference frames can be used by the current frame or a subsequent frame.

- **ref_Pic**: the POC offset of each reference frame relative to the current frame.
- **used_by_cur**: whether each frame is used as a reference frame for encoding of the current frame. Available values include:
 - 0: The corresponding frame listed in ref_Pic is not used as reference when the current frame is encoded.
 - 1: The corresponding frame listed in ref_Pic is used as reference when the current frame is encoded.

If long-term reference (LTR) is used, you also need to configure `VCEncGopPicSpecialConfig`. For details, see Section *GOP Structures with Long-Term Reference*.

The following sections describe default GOP structures of size 1 and size 4, and an example GOP structure with multiple temporal layers.

6.1.1 Default GOP Structure of Size 1

Figure 6.1 shows the default GOP structure of size 1. In the GOP structure, each P-frame references only its previous frame. For the configurations of the GOP structure, see Table 6.1.



Figure 6.1 GOP Structure of Size 1

Table 6.1 VCEnc GopPicConfig Configurations for GOP Structure of Size 1

Frame ID	coding Type	poc	Qp Offset	Qp Factor	temporal Id	numRef Pics	ref_Pic	used_by_cur
1	P	1	0	0.8	0	1	-1	1

6.1.2 Default GOP Structure of Size 4

Figure 6.2 shows the default GOP structure of size 4. In the GOP structure:

- Frame 1 is a P-frame with POC 4, referencing the frame with POC 0.
 - Frame 2 is a B-frame with POC 2, referencing the frame with POC 0 and the frame with POC 4.
 - Frame 3 is a B-frame with POC 1, referencing the frame with POC 0 and the frame with POC 2.
- The frame with POC 4 is also kept as a reference frame to be used by a subsequent frame.
- Frame 4 is a B-frame with POC 3, referencing the frame with POC 2 and the frame with POC 4.

For the configurations of the GOP structure, see Table 6.2.

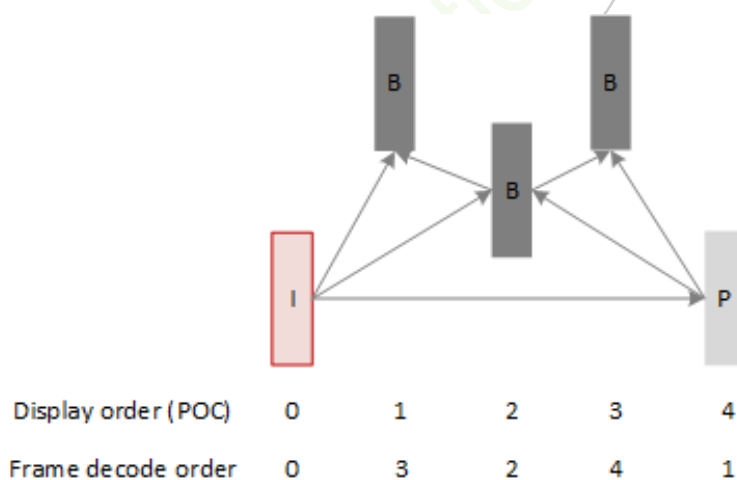


Figure 6.2 GOP Structure of Size 4

Table 6.2 VCEnc GopPicConfig Configurations for GOP Structure of Size 4

Frame ID	coding Type	poc	Qp Offset	Qp Factor	numRef Pics	ref_Pic	used_by_cur
1	P	4	0	0.5	1	-4	1
2	B	2	0	0.3536	2	-2, 2	1, 1
3	B	1	0	0.5	3	-1, 1, 3	1, 1, 0
4	B	3	0	0.5	2	-1, 1	1, 1

6.1.3 Example GOP Structure with Temporal Scalable Video Coding (SVC-T)

Figure 6.3 shows the GOP structure used in this example. For the GOP structure configurations, see Table 6.3.

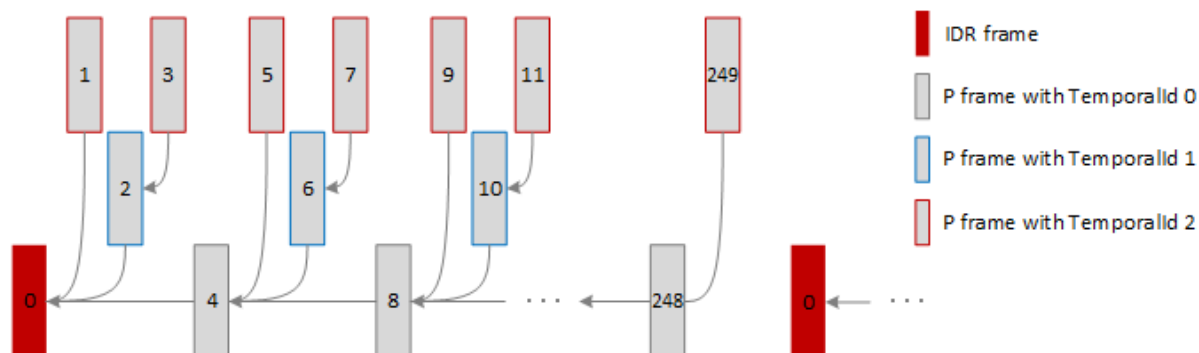


Figure 6.3 GOP Structure with SVC-T

Table 6.3 VCEnc GopPicConfig Configurations in the SVC-T Example

Frame ID	coding Type	poc	Qp Offset	Qp Factor	temporal	numRef Pics	ref_Pic	used_by_cur
Id								
1	P	1	0	0.4624	2	1	-1	1
2	P	2	0	0.4624	1	2	-1, -2	0, 1
3	P	3	0	0.4624	2	3	-1, -2, -3	1, 0, 0
4	P	4	0	0.578	0	3	-1, -3, -4	0, 0, 1

6.2 GOP Structures with Long-Term Reference

In order to construct a GOP structure with long-term reference (LTR), besides `VCEncGopPicConfig`, you also need to configure the following fields in the `VCEncGopPicSpecialConfig` structure:

NOTE: Special frame in this document refers to a frame related with LTR.

- **Frame ID:** the ID of the frame, which is used in the configuration file to mark whether the frame is configured using `VCEncGopPicSpecialConfig` or `VCEncGopPicConfig`.

- 0: The frame is configured using `VCEncGopPicSpecialConfig`.
- Other values: The frame is configured using `VCEncGopPicConfig`.

- **codingType**: the slice type.

- I: `VCENC_INTRA_FRAME`
- P: `VCENC_PREDICTED_FRAME`
- B: `VCENC_BIDIR_PREDICTED_FRAME`

The default value is -255, which indicates the video encoder uses the slice type specified in `VCEncGopPicConfig`. The specified field value is used only if it differs from the default.

- **QpOffset**: the offset to be added to the QP parameter to obtain the final QP.

The default value is -255, which indicates the video encoder uses the QP offset specified in `VCEncGopPicConfig`. The specified field value is used only if it differs from the default.

- **QpFactor**: the factor used by rate distortion optimization.

A higher value means lower quality and fewer bits. A typical range is from 0.3 to 1.

The default value is -255, which indicates the video encoder uses the QP factor specified in `VCEncGopPicConfig`. The specified field value is used only if it differs from the default.

- **temporalId**: the temporal layer ID.

The default value is -255, which indicates the video encoder uses the temporal layer ID specified in `VCEncGopPicConfig`. The specified field value is used only if it differs from the default.

- **numRefPics**: the number of reference frames kept for the current frame.

The reference frames can be used by the current frame or a subsequent frame.

- **ref_Pic**: the POC offset of each reference frame relative to the current frame.

The value L_n in tables in this section indicates the frame with its LTR index being n .

- **used_by_cur**: whether each frame is used as a reference frame for encoding of the current frame.

Available values include:

- 0: The corresponding frame listed in `ref_Pic` is not used as reference when the current frame is encoded.
- 1: The corresponding frame listed in `ref_Pic` is used as reference when the current frame is encoded.

- **i32Ltr**: the LTR index of the frame.

Available values include:

- 0: a frame that uses LTR.
- A value in [1, `VCENC_MAX_LT_REF_FRAMES`]: an LTR frame
- **i32Offset:**
 - When i32Ltr equals 0, i32Offset indicates the POC delta between the LTR frame and the first subsequent frame that uses the LTR frame as reference.
 - When i32Ltr falls in the range [1, `VCENC_MAX_LT_REF_FRAMES`], i32Offset indicates the POC delta between the first LTR frame and the first encoded frame.
- **i32Interval:**
 - When i32Ltr equals 0, i32Interval indicates the POC delta between two adjacent frames that use the same LTR frame as reference.
 - When i32Ltr falls in the range [1, `VCENC_MAX_LT_REF_FRAMES`], i32Interval indicates the POC delta between two adjacent frames with the same LTR index.

The following sections provides several example GOP structures with LTR for your reference, where the IDR frame update interval is assumed to be 250 frames.

6.2.1 Example 1: One Special Frame

Figure 6.4 shows the GOP structure used in example 1. In the GOP structure, within every 25 frames, the first frame is an I-frame and is not used for long-term reference. When coding the I-frame, its QP offset is -2, the temporal layer ID in `VCEncGopPicConfig` is used, and no reference frame is kept.

For the GOP structure configurations, see Table 6.4 and Table 6.5.

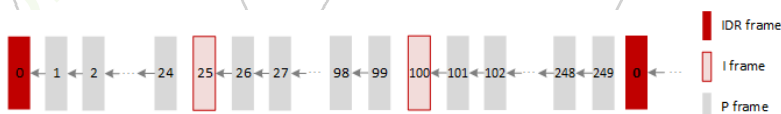


Figure 6.4 GOP Structure with One Special Frame

Table 6.4 VCEnc GopPicConfig Configurations in Example 1

Frame ID	coding Type	poc	Qp Offset	Qp Factor	temporal Id	numRef Pics	ref_Pic	used_by_cur
1	P	1	0	0.4624	0	1	-1	1

Table 6.5 VCEnc GopPicSpecialConfig Configurations for Special Frame in Example 1

Frame ID	coding Type	Qp Offset	Qp Factor	temporal Id	num Ref Pics	ref_Pic	used_by_cur	i32 Ltr	i32 Offset	i32 Interval
0	I	-2	0.4624	-255	0	0	0	0	0	25

6.2.2 Example 2: One Long-Term Reference Frame

Figure 6.5 shows the GOP structure used in example 2. In the GOP structure, a long-term reference (LTR) frame is defined with index 1. The LTR frame is updated every 100 frames and used once every 20 frames.

For the GOP structure configurations, see Table 6.6 and Table 6.7.



Figure 6.5 GOP Structure with One Long-Term Reference Frame

Table 6.6 VCEnc GopPicConfig Configurations in Example 2

Frame ID	coding Type	poc	Qp Offset	Qp Factor	temporal Id	numRef Pics	ref_Pic	used_by_cur
1	P	1	0	0.4624	0	1	-1	1

Table 6.7 VCEnc GopPicSpecialConfig Configurations in Example 2

Frame ID	coding Type	Qp Offset	Qp Factor	temporal Id	num Ref Pics	ref_Pic	used_by_cur	i32 Ltr	i32 Offset	i32 Interval
0	I	-2	0.4624	-255	1	L1	0	1	0	100
0	-255	-2	0.4624	-255	1	L1	1	0	0	20

6.2.3 Example 3: One Long-Term Reference Frame and One Short-Term Reference Frame

Figure 6.6 shows the GOP structure used in example 3. In the GOP structure, a long-term reference (LTR) frame and a short-term reference (STR) frame are defined. The P-frame defined in Table 6.8 turns into a B-frame because the interval is 1 in row 3 of Table 6.9 and the frame has two reference frames: one for LTR and one for STR. Every 20 frames, there is a special P-frame, which uses only the LTR frame.

For the GOP structure configurations, see Table 6.8 and Table 6.9.

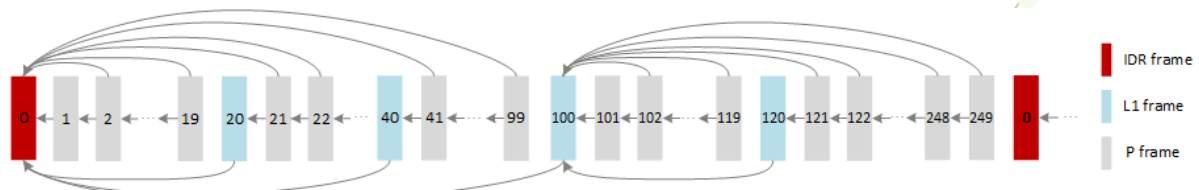


Figure 6.6 GOP Structure with One Long-Term Reference Frame and One Short-Term Reference Frame

Table 6.8 VCEnc GopPicConfig Configurations in Example 3

Frame ID	coding Type	poc	Qp Offset	Qp Factor	temporal Id	numRef Pics	ref_Pic	used_by_cur
1	P	1	0	0.4624	0	1	-1	1

Table 6.9 VCEnc GopPicSpecialConfig Configurations in Example 3

Frame ID	coding Type	Qp Offset	Qp Factor	temporal Id	num Ref Pics	ref_Pic	used_by_cur	i32 Ltr	i32 Offset	i32 Interval
0	-255	-2	0.4624	-255	1	L1	1	1	0	100
0	-255	-2	0.4624	-255	1	L1	1	0	0	20
0	-255	-2	0.4624	-255	2	-1, L1	1, 1	0	0	1

6.2.4 Example 4: Two Long-Term Reference Frames

Figure 6.7 shows the GOP structure used in example 4. In the GOP structure, two long-term reference (LTR) frames are defined with indices 1 and 2. LTR frame 1 updates every 40 frames and LTR frame 2 updates every 10 frames.

For the GOP structure configurations, see Table 6.10 and Table 6.11. In this example, references to the LTR frames are specified in `VCEncGopPicConfig`, instead of `VCEncGopPicSpecialConfig` which is used in examples 1 to 3.

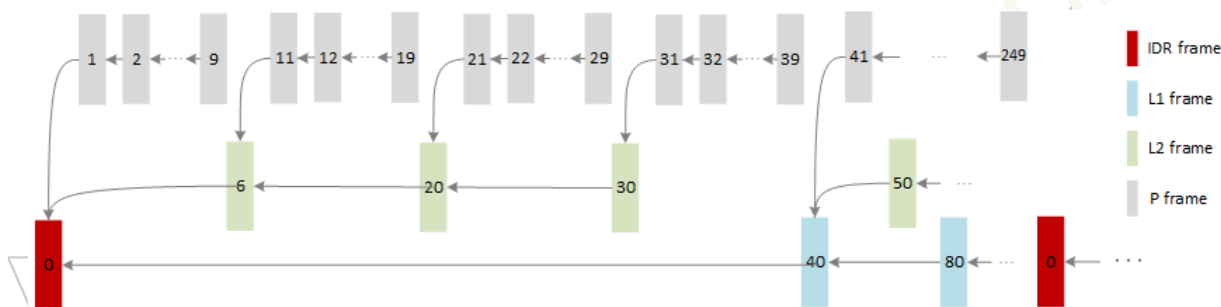


Figure 6.7 GOP Structure with Two Long-Term Reference Frames

Table 6.10 VCEnc GopPicConfig Configurations in Example 4

Frame ID	coding Type	poc	Qp Offset	Qp Factor	temporal Id	numRef Pics	ref_Pic	used_by_cur
1	P	1	0	0.4624	0	3	-1, L1, L2	1, 0, 0

Table 6.11 VCEnc GopPicSpecialConfig Configurations in Example 4

Frame ID	coding Type	Qp Offset	Qp Factor	temporal Id	num Ref Pics	ref_Pic	used_by_cur	i32 Ltr	i32 Offset	i32 Interval
0	-255	-2	0.4624	0	1	L1	1	1	0	40
0	-255	-2	0.4624	0	2	L1, L2	0, 1	2	0	10

6.2.5 Example 5: Combination of Long-Term Reference Frame and Special Frame

Figure 6.8 shows the GOP structure used in example 5. For the GOP structure configurations, see Table 6.12 and Table 6.13.

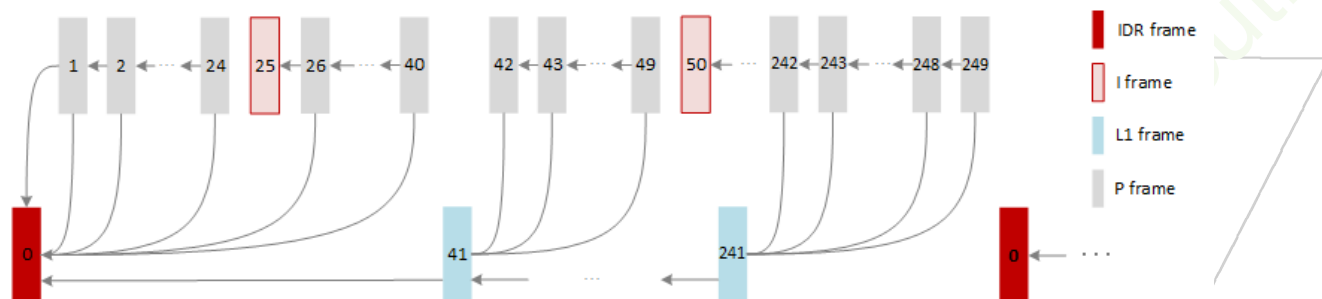


Figure 6.8 GOP Structure with Long-Term Reference Frame and Special Frame

Table 6.12 VCEnc GopPicConfig Configurations in Example 5

Frame ID	coding Type	poc	Qp Offset	Qp Factor	temporal Id	numRef Pics	ref_Pic	used_by_cur
1	P	1	0	0.4624	0	1	-1	1

Table 6.13 VCEnc GopPicSpecialConfig Configurations in Example 5

Frame ID	coding Type	Qp Offset	Qp Factor	temporal Id	num Ref Pics	ref_Pic	used_by_cur	i32 Ltr	i32 Offset	i32 Interval
0	I	-2	0.4624	-255	1	L1	0	0	0	25
0	-255	-2	0.4624	-255	1	L1	1	1	1	40
0	-255	-255	-255	-255	2	-1, L1	1, 1	0	0	1

6.3 Adaptive GOP Structure Size Decision

To enhance quality, the video encoder supports adaptively adjusting the GOP structure size based on returned encoding information.

7 Application Examples

The following code examples demonstrate basic usage of the encoder. The sample code uses the API functions introduced in this document, as well as external OS-specific functions `ReadYuv420PlanarFrame()`, `SaveStream()`, `AllocateDMABuffer()`, and `FreeDMABuffer()`.

The examples are simplified with no error handling logic defined. For a comprehensive example of how to use the encoder, see the `Test_bench.c` file.

NOTE: The provided sample code is not guaranteed to compile as such.

7.1 Encoding 300 HEVC Frames

```

/*-----
--
--      This software is confidential and proprietary and may be used
--      only as expressly authorized by a licensing agreement from
--
--
--          VeriSilicon Inc.
--
--          (C) COPYRIGHT 2021 VeriSilicon Inc
--          ALL RIGHTS RESERVED
--
--      The entire notice above must be reproduced
--      on all copies and should not be removed.
--
-------*/
/*-----
--
--      A Simple Demo of Encoding 300 Frames (HEVC)
--
-------*/

#include "hevcencapi.h"
#include "ewl.h"
#define TEST_YUV_FILE_NAME "input.yuv"
#define TEST_OUTPUT_FILE_NAME "test.hevc"
#define TEST_ALIGNMENT 64
#define TEST_WIDTH 416
#define TEST_HEIGHT 240
struct test_bench {
    char *inputFile;
    char *outputFile;
    FILE *inP;

```

```

FILE *outP;
i32 width;
i32 height;
i32 firstPic;
i32 lastPic;
i32 picture_encoded_cnt; /*count of frame finish encoding*/
u8 *lum;
u8 *cb;
u8 *cr;
u32 input_alignment;
u32 lumaSize;
u32 chromaSize;
EWLLinearMem_t pictureMem;
EWLLinearMem_t outbufMem;
u32 gopSize;
VCEncIn encIn;
const void *ewlInst;
i32 *tile_width;
i32 *tile_height;
};
/*Init tb's ewl instance to malloc/free buffer in test_bench*/
static const void *InitEwlInst(struct test_bench *tb) {
    if (NULL == tb->ewlInst) {
        /* Init EWL instance for test bench to malloc/free linear buffer*/
        EWLInitParam_t param;
        param.context = NULL;
        param.clientType = EWL_CLIENT_TYPE_MEM; //buffer operation
        param.slice_idx = 0;
        param.useVcmd = -1;
        if ((tb->ewlInst = EWLInit(&param)) == NULL) {
            return NULL;
        }
    }
    return tb->ewlInst;
}
static int AllocatedDMABuffer(struct test_bench *tb) {
    u32 lumaSize = 0, chromaSize = 0;
    u32 pictureSize = 0;
    u32 outbufSize = 0;
    int ret = EWL_OK;
    /* picture buffer size YUV 420 */
    lumaSize = STRIDE(tb->width, tb->input_alignment);
    chromaSize = STRIDE(tb->width / 2, tb->input_alignment);
    tb->lumaSize = lumaSize * tb->height;
    tb->chromaSize = chromaSize * tb->height / 2 * 2;
    pictureSize = tb->lumaSize + tb->chromaSize;
    ret = EWLMallocLinear(tb->ewlInst, pictureSize, tb->input_alignment,
                        &tb->pictureMem);
    if (ret != EWL_OK) {
        tb->pictureMem.virtualAddress = NULL;
        return 1;
    }
    /* Output buffer size */
    outbufSize = 4 * pictureSize;
    ret = EWLMallocLinear(tb->ewlInst, outbufSize, tb->input_alignment,
                        &tb->outbufMem);
    if (ret != EWL_OK) {
        tb->outbufMem.virtualAddress = NULL;
        return 1;
    }
    return ret;
}
void FreeDMABuffer(struct test_bench *tb) {
    const void *ewl_inst = tb->ewlInst;
    EWLFreeLinear(ewl_inst, &tb->pictureMem);
    EWLFreeLinear(ewl_inst, &tb->outbufMem);
    return;
}

```

```

i32 allocVariableMem(struct test_bench *tb) {
    tb->tile_width = (i32 *)calloc(1, sizeof(i32));
    if (tb->tile_width == NULL) return 0;
    tb->tile_height = (i32 *)calloc(1, sizeof(i32));
    if (tb->tile_height == NULL) return 0;
    return 1;
}

void FreeVariableMem(struct test_bench *tb) {
    free(tb->tile_width);
    free(tb->tile_height);
}

static i32 file_read(FILE *file, u8 *data, u64 seek, size_t size) {
    if ((file == NULL) || (data == NULL)) return NOK;
    fseeko(file, seek, SEEK_SET);
    if (fread(data, sizeof(u8), size, file) < size) {
        if (!feof(file)) {
            printf("Read input picture error!\n");
        }
        return NOK;
    }
    return OK;
}

static i32 ReadYuv420PlanarFrame(struct test_bench *tb, u32 src_img_size,
                                u32 src_width, u32 src_height) {
    static i32 num = 0;
    u64 seek;
    i32 w;
    i32 h;
    u8 *lum;
    u8 *cb;
    u8 *cr;
    i32 i;
    u32 luma_stride;
    u32 chroma_stride;
    u32 alignment;
    seek = ((u64)num++) * ((u64)src_img_size);
    lum = tb->lum;
    cb = tb->cb;
    cr = tb->cr;
    alignment = tb->input_alignment;
    luma_stride = STRIDE(src_width, alignment);
    chroma_stride = STRIDE(src_width / 2, alignment);
    /* Lum */
    for (i = 0; i < src_height; i++) {
        if (file_read(tb->inP, lum, seek, src_width)) return NOK;
        seek += src_width;
        lum += luma_stride;
    }
    w = ((src_width + 1) >> 1);
    h = ((src_height + 1) >> 1);
    /* Cb */
    for (i = 0; i < h; i++) {
        if (file_read(tb->inP, cb, seek, w)) return NOK;
        seek += w;
        cb += chroma_stride;
    }
    /* Cr */
    for (i = 0; i < h; i++) {
        if (file_read(tb->inP, cr, seek, w)) return NOK;
        seek += w;
        cr += chroma_stride;
    }
    return 0;
}

static int SaveStream(u8 *buf, u32 size, FILE *fp)
{
    return (fwrite(buf, 1, size, fp));
}

```

```

static int HevcInitGopConfigs(int gopSize, VCEncGopConfig *gopCfg,
                             u8 *gopCfgOffset) {
    VCEncGopPicConfig *cfg = NULL;
    gopCfg->special_size = 0;
    gopCfg->ltrcnt = 0;
    /* gopCfgOffset: keep record of gopSize's offset in pGopPicCfg */
    if (gopCfgOffset) gopCfgOffset[gopSize] = gopCfg->size;
    /* gop_config : #Frame: P 1 0 0.578 0 1 -1 1 */
    cfg = &(gopCfg->pGopPicCfg[gopCfg->size++]);
    cfg->poc = 1;
    ;
    cfg->QpOffset = 0;
    cfg->QpFactor = 0.760263; //sqrt(0.578)
    cfg->temporalId = 0;
    cfg->codingType = 1;
    cfg->nonReference = 0;
    cfg->numRefPics = 1;
    cfg->refPics->ref_pic = -1;
    cfg->refPics->used_by_cur = 1;
    return 0;
}
/*-----
--
--                               main
--
--*/

int main(i32 argc, char **argv) {
    i32 ret = OK;
    int i;
    VCEncConfig cfg;
    struct test_bench tb;
    VCEncInst hevc_encoder;
    VCEncGopPicConfig gopPicCfg[MAX_GOP_PIC_CONFIG_NUM];
    VCEncIn *pEncIn = &(tb.encIn);
    VCEncOut encOut;
    u64 streamSize = 0;
    u32 src_img_size;
    memset(&tb, 0, sizeof(struct test_bench));
    const void *ewl_inst = InitEwlInst(&tb);
    /* Step 1: Initialize an hevc encoder instance */
    memset(&cfg, 0, sizeof(VCEncConfig));
    /* Image Resolution */
    cfg.width = TEST_WIDTH;
    cfg.height = TEST_HEIGHT;
    /* 30 fps Frame Rate */
    cfg.frameRateDenom = 1;
    cfg.frameRateNum = 30;
    /* Stream Type */
    cfg.streamType = VCENC_BYTE_STREAM;
    cfg.level = VCENC_HEVC_LEVEL_6;
    cfg.tier = VCENC_HEVC_MAIN_TIER;
    cfg.profile = VCENC_HEVC_MAIN_PROFILE;
    /* Codec Format */
    cfg.codecFormat = VCENC_VIDEO_CODEC_HEVC;
    cfg.bitDepthLuma = 8;
    cfg.bitDepthChroma = 8;
    cfg.maxTLayers = 1;
    cfg.refFrameAmount = 4;
    cfg.enableSsim = 1;
    cfg.enablePsnr = 1;
    cfg.parallelCoreNum = 1;
    cfg.dumpCuInfo = 0;
    cfg.log2MaxPicOrderCntLsb = 16;
    cfg.log2MaxFrameNum = 12;
    cfg.inLoopDSRatio = 1;
    cfg.cuInfoVersion = -1;
    cfg.extSramLumHeightBwd = 16;
    cfg.extSramChrHeightBwd = 8;
}

```

```

cfg.extSramLumHeightFwd = 16;
cfg.extSramChrHeightFwd = 8;
cfg.gopSize = 1;
cfg.codedChromaIdc = VCENC_CHROMA_IDC_420;
cfg.writeReconToDDR = 1;
cfg.av1InterFiltSwitch = 1;
cfg.tune = VCENC_TUNE_PSNR;
allocVariableMem(&tb);
cfg.tile_width = tb.tile_width;
cfg.tile_height = tb.tile_height;
cfg.tiles_enabled_flag = 0;
cfg.num_tile_columns = 1;
cfg.num_tile_rows = 1;
cfg.loop_filter_across_tiles_enabled_flag = 0;
u32 log2_ctb_size = 6;
i32 width = cfg.width;
i32 height = cfg.height;
u32 ctbPerRow = ((width + (1 < log2_ctb_size) - 1) >> log2_ctb_size);
u32 ctbPerColumn = ((height + (1 < log2_ctb_size) - 1) >> log2_ctb_size);
u32 tileId;
for (tileId = 0; tileId < cfg.num_tile_columns; tileId++) {
    cfg.tile_width[tileId] =
        (tileId + 1) * ctbPerRow / cfg.num_tile_columns -
        (tileId * ctbPerRow) / cfg.num_tile_columns;
}
for (tileId = 0; tileId < cfg.num_tile_rows; tileId++) {
    cfg.tile_height[tileId] =
        (tileId + 1) * ctbPerColumn / cfg.num_tile_rows -
        (tileId * ctbPerColumn) / cfg.num_tile_rows;
}
if ((ret = VCEncInit(&cfg, &hevc_encoder, NULL)) != VCENC_OK) {
    return (int)ret;
}
tb.inputFile = TEST_YUV_FILE_NAME;
tb.outputFile = TEST_OUTPUT_FILE_NAME;
tb.firstPic = 0;
tb.lastPic = 30;
tb.width = TEST_WIDTH;
tb.height = TEST_HEIGHT;
tb.input_alignment = 64;
tb.picture_encoded_cnt = 0;
/* Step 2: Extra encoder configuration. See the next example code for how to change the
 * default encoder parameters
 */
/* GOP configuration */
memset(gopPicCfg, 0, sizeof(gopPicCfg));
tb.encIn.gopConfig.pGopPicCfg = gopPicCfg;
HevcInitGopCfgs(tb.gopSize, &(tb.encIn.gopConfig),
                tb.encIn.gopConfig.gopCfgOffset);
tb.encIn.gopConfig.outputRateNumer = 30; /* Output frame rate numerator */
tb.encIn.gopConfig.outputRateDenom = 1; /* Output frame rate denominator */
tb.encIn.gopConfig.inputRateNumer = 30; /* Input frame rate numerator */
tb.encIn.gopConfig.inputRateDenom = 1; /* Input frame rate denominator */
/* Step 3: Allocate linear memory resources (input and output buffers) */
if ((ret = AllocateDMABuffer(&tb)) != VCENC_OK) {
    goto end;
}
/* Step 4: Start the stream */
/* Output/input streams */
if (!(tb.inP = fopen(tb.inputFile, "rb"))) goto error;
if (!(tb.outP = fopen(tb.outputFile, "wb"))) goto error;
pEncIn->busOutBuf[0] = tb.outbufMem.busAddress;
pEncIn->outBufSize[0] = tb.outbufMem.size;
pEncIn->pOutBuf[0] = tb.outbufMem.virtualAddress;
pEncIn->cur_out_buffer[0] = &tb.outbufMem;
pEncIn->dec400Enable = 0;
/* Video, sequence and picture parameter sets */
if (VCEncStrmStart(hevc_encoder, pEncIn, &encOut)) {

```

```

    /* Handle the error situation here*/
    goto error;
}
/* Save the stream header */
SaveStream((u8 *)pEncIn->pOutBuf[0], encOut.streamSize, tb.outP);
streamSize += encOut.streamSize;
/* Step 5: Encode the video frames */
while (HANTRO_TRUE) {
    if (tb.picture_encoded_cnt >= tb.lastPic) break;
    /* Setup encoder input, we assume one linear buffer for the input image in */
    /* planar YCbCr 4:2:0 format */
    pEncIn->busLuma = tb.pictureMem.busAddress;
    tb.lum = (u8 *)tb.pictureMem.virtualAddress;
    pEncIn->busChromaU = pEncIn->busLuma + tb.lumaSize;
    tb.cb = tb.lum + tb.lumaSize;
    pEncIn->busChromaV = pEncIn->busChromaU + tb.chromaSize / 2;
    tb.cr = tb.cb + tb.chromaSize / 2;
    src_img_size =
        tb.width * tb.height * 12 / 8; //bit_pre_pixel of yuv420 is 12
    /* Read YUV to input buffer */
    ReadYuv420PlanarFrame(&tb, src_img_size, tb.width, tb.height);
    printf("=== Encoding %i %s ...\n", tb.picture_encoded_cnt, tb.inputFile);
    ret = VCEncStrmEncode(hevc_encoder, pEncIn, &encOut, NULL, NULL);
    /* Save encoded stream */
    switch (ret) {
        case VCENC_FRAME_ENQUEUE:
            pEncIn->picture_cnt++;
            break;
        case VCENC_FRAME_READY:
            if (encOut.codingType != VCENC_NOTCODED_FRAME) pEncIn->picture_cnt++;
            if (encOut.streamSize != 0) {
                SaveStream((u8 *)pEncIn->pOutBuf[0], encOut.streamSize, tb.outP);
                streamSize += encOut.streamSize;
                tb.picture_encoded_cnt++;
            }
            break;
        case VCENC_SYSTEM_ERROR:
            /* Fatal system error, can't continue. */
        default:
            goto error;
    }
} /* End of main encoding loop */
/* Flush encoder internal buffered frames
 * In Multicore enabled or 2-pass enabled, encoder internal cached frames
 * must be flushed
 */
ret = VCEncFlush(hevc_encoder, pEncIn, &encOut, NULL, NULL);
/* Step 6: End stream */
ret = VCEncStrmEnd(hevc_encoder, pEncIn, &encOut);
if (ret == VCENC_OK) {
    SaveStream((u8 *)pEncIn->pOutBuf[0], encOut.streamSize, tb.outP);
    streamSize += encOut.streamSize;
} else {
    /* Handle the error situation here*/
    goto error;
}
printf("Total of %d frames encoded, %lu bytes\n", tb.picture_encoded_cnt,
        (long unsigned int)streamSize);
error:
    /* Free all resources */
    if (tb.inP) {
        fclose(tb.inP);
        tb.inP = NULL;
    }
    if (tb.outP) {
        fclose(tb.outP);
        tb.outP = NULL;
    }
}

```



```

    EWLInitStruct_t param;
    param.context = NULL;
    param.clientType = EWL_CLIENT_TYPE_MEM; //buffer operation
    param.slice_idx = 0;
    param.useVcmd = -1;
    if ((tb->ewlInst = EWLInit(&param)) == NULL) {
        return NULL;
    }
}
return tb->ewlInst;
}

static int AllocatedDMABuffer(struct test_bench *tb) {
    u32 lumaSize = 0, chromaSize = 0;
    u32 pictureSize = 0;
    u32 outbufSize = 0;
    int ret = EWL_OK;
    /* picture buffer size YUV 420 */
    lumaSize = STRIDE(tb->width, tb->input_alignment);
    chromaSize = STRIDE(tb->width / 2, tb->input_alignment);
    tb->lumaSize = lumaSize * tb->height;
    tb->chromaSize = chromaSize * tb->height / 2 * 2;
    pictureSize = tb->lumaSize + tb->chromaSize;
    ret = EWLMallocLinear(tb->ewlInst, pictureSize, tb->input_alignment,
        &tb->pictureMem);

    if (ret != EWL_OK) {
        tb->pictureMem.virtualAddress = NULL;
        return 1;
    }
    /* Output buffer size */
    outbufSize = 4 * pictureSize;
    ret = EWLMallocLinear(tb->ewlInst, outbufSize, tb->input_alignment,
        &tb->outbufMem);

    if (ret != EWL_OK) {
        tb->outbufMem.virtualAddress = NULL;
        return 1;
    }
    return ret;
}

void FreeDMABuffer(struct test_bench *tb) {
    const void *ewl_inst = tb->ewlInst;
    EWLFreeLinear(ewl_inst, &tb->pictureMem);
    EWLFreeLinear(ewl_inst, &tb->outbufMem);
    return;
}

i32 allocVariableMem(struct test_bench *tb) {
    tb->tile_width = (i32 *)calloc(1, sizeof(i32));
    if (tb->tile_width == NULL) return 0;
    tb->tile_height = (i32 *)calloc(1, sizeof(i32));
    if (tb->tile_height == NULL) return 0;
    return 1;
}

void FreeVariableMem(struct test_bench *tb) {
    free(tb->tile_width);
    free(tb->tile_height);
}

static i32 file_read(FILE *file, u8 *data, u64 seek, size_t size) {
    if ((file == NULL) || (data == NULL)) return NOK;
    fseeko(file, seek, SEEK_SET);
    if (fread(data, sizeof(u8), size, file) < size) {
        if (!feof(file)) {
            printf("Read input picture error!\n");
        }
        return NOK;
    }
    return OK;
}

static i32 ReadYuv420PlanarFrame(struct test_bench *tb, u32 src_img_size,
    u32 src_width, u32 src_height) {

```

```

static i32 num = 0;
u64 seek;
i32 w;
i32 h;
u8 *lum;
u8 *cb;
u8 *cr;
i32 i;
u32 luma_stride;
u32 chroma_stride;
u32 alignment;
seek = ((u64)num++) * ((u64)src_img_size);
lum = tb->lum;
cb = tb->cb;
cr = tb->cr;
alignment = tb->input_alignment;
luma_stride = STRIDE(src_width, alignment);
chroma_stride = STRIDE(src_width / 2, alignment);
/* Lum */
for (i = 0; i < src_height; i++) {
    if (file_read(tb->inP, lum, seek, src_width)) return NOK;
    seek += src_width;
    lum += luma_stride;
}
w = ((src_width + 1) >> 1);
h = ((src_height + 1) >> 1);
/* Cb */
for (i = 0; i < h; i++) {
    if (file_read(tb->inP, cb, seek, w)) return NOK;
    seek += w;
    cb += chroma_stride;
}
/* Cr */
for (i = 0; i < h; i++) {
    if (file_read(tb->inP, cr, seek, w)) return NOK;
    seek += w;
    cr += chroma_stride;
}
return 0;
}
static int SaveStream(u8 *buf, u32 size, FILE *fp)
{
    return (fwrite(buf, 1, size, fp));
}
static int H264InitGopConfigs(int gopSize, VCEncGopConfig *gopCfg,
                             u8 *gopCfgOffset) {
    VCEncGopPicConfig *cfg = NULL;
    gopCfg->special_size = 0;
    gopCfg->ltrcnt = 0;
    /* gopCfgOffset: keep record of gopSize's offset in pGopPicCfg */
    if (gopCfgOffset) gopCfgOffset[gopSize] = gopCfg->size;
    /* gop_config : #Frame: P 1 0 0.578 0 1 -1 1 */
    cfg = &(gopCfg->pGopPicCfg[gopCfg->size++]);
    cfg->poc = 1;
    ;
    cfg->QpOffset = 0;
    cfg->QpFactor = 0.760263; //sqrt(0.578)
    cfg->temporalId = 0;
    cfg->codingType = 1;
    cfg->nonReference = 0;
    cfg->numRefPics = 1;
    cfg->refPics->ref_pic = -1;
    cfg->refPics->used_by_cur = 1;
    return 0;
}
/*-----
--
--
main
--

```

```

--
-----*/
int main(i32 argc, char **argv) {
    i32 ret = OK;
    int i;
    VCEncConfig cfg;
    struct test_bench tb;
    VCEncInst h264_encoder;
    VCEncGopPicConfig gopPicCfg[MAX_GOP_PIC_CONFIG_NUM];
    VCEncIn *pEncIn = &(tb.encIn);
    VCEncOut encOut;
    u64 streamSize = 0;
    u32 src_img_size;
    memset(&tb, 0, sizeof(struct test_bench));
    const void *ewl_inst = InitEwlInst(&tb);
    /* Step 1: Initialize an h264 encoder instance */
    memset(&cfg, 0, sizeof(VCEncConfig));
    /* Image Resolution */
    cfg.width = TEST_WIDTH;
    cfg.height = TEST_HEIGHT;
    /* 30 fps Frame Rate */
    cfg.frameRateDenom = 1;
    cfg.frameRateNum = 30;
    /* Stream Type */
    cfg.streamType = VCENC_BYTE_STREAM;
    cfg.level = VCENC_H264_LEVEL_5_1;
    cfg.tier = VCENC_HEVC_MAIN_TIER;
    cfg.profile = VCENC_H264_HIGH_PROFILE;
    /* Codec Format */
    cfg.codecFormat = VCENC_VIDEO_CODEC_H264;
    cfg.bitDepthLuma = 8;
    cfg.bitDepthChroma = 8;
    cfg.maxTlayers = 1;
    cfg.refFrameAmount = 4;
    cfg.enableSsim = 1;
    cfg.enablePsnr = 1;
    cfg.parallelCoreNum = 1;
    cfg.dumpCuInfo = 0;
    cfg.log2MaxPicOrderCntLsb = 16;
    cfg.log2MaxFrameNum = 12;
    cfg.interlacedFrame = 0;
    cfg.inLoopDSRatio = 1;
    cfg.cuInfoVersion = -1;
    cfg.extSramLumHeightBwd = 16;
    cfg.extSramChrHeightBwd = 8;
    cfg.extSramLumHeightFwd = 16;
    cfg.extSramChrHeightFwd = 8;
    cfg.gopSize = 1;
    cfg.codedChromaIdc = VCENC_CHROMA_IDC_420;
    cfg.writeReconToDDR = 1;
    cfg.avlInterFiltSwitch = 1;
    cfg.tune = VCENC_TUNE_PSNR;
    allocVariableMem(&tb);
    cfg.tile_width = tb.tile_width;
    cfg.tile_height = tb.tile_height;
    cfg.tiles_enabled_flag = 0;
    cfg.num_tile_columns = 1;
    cfg.num_tile_rows = 1;
    cfg.loop_filter_across_tiles_enabled_flag = 0;
    u32 log2_ctb_size = 4;
    i32 width = cfg.width;
    i32 height = cfg.height;
    u32 ctbPerRow = ((width + (1 << log2_ctb_size) - 1) >> log2_ctb_size);
    u32 ctbPerColumn = ((height + (1 << log2_ctb_size) - 1) >> log2_ctb_size);
    u32 tileId;
    for (tileId = 0; tileId < cfg.num_tile_columns; tileId++) {
        cfg.tile_width[tileId] =
            (tileId + 1) * ctbPerRow / cfg.num_tile_columns -

```

```

        (tileId * ctbPerRow) / cfg.num_tile_columns;
    }
    for (tileId = 0; tileId < cfg.num_tile_rows; tileId++) {
        cfg.tile_height[tileId] =
            (tileId + 1) * ctbPerColumn / cfg.num_tile_rows -
            (tileId * ctbPerColumn) / cfg.num_tile_rows;
    }
    if ((ret = VCEncInit(&cfg, &h264_encoder, NULL)) != VCENC_OK) {
        return (int)ret;
    }
    tb.inputFile = TEST_YUV_FILE_NAME;
    tb.outputFile = TEST_OUTPUT_FILE_NAME;
    tb.firstPic = 0;
    tb.lastPic = 30;
    tb.width = TEST_WIDTH;
    tb.height = TEST_HEIGHT;
    tb.input_alignment = 64;
    tb.picture_encoded_cnt = 0;
    /* Step 2: Extra encoder configuration. See the next example code for how to change the
     * default encoder parameters
     */
    /* GOP configuration */
    memset(gopPicCfg, 0, sizeof(gopPicCfg));
    tb.encIn.gopConfig.pGopPicCfg = gopPicCfg;
    H264InitGopConfigs(tb.gopSize, &(tb.encIn.gopConfig),
        tb.encIn.gopConfig.gopCfgOffset);
    tb.encIn.gopConfig.outputRateNumer = 30; /* Output frame rate numerator */
    tb.encIn.gopConfig.outputRateDenom = 1; /* Output frame rate denominator */
    tb.encIn.gopConfig.inputRateNumer = 30; /* Input frame rate numerator */
    tb.encIn.gopConfig.inputRateDenom = 1; /* Input frame rate denominator */
    /* Step 3: Allocate linear memory resources (input and output buffers) */
    if ((ret = AllocateDMABuffer(&tb)) != VCENC_OK) {
        goto end;
    }
    /* Step 4: Start the stream */
    /* Output/input streams */
    if (!(tb.inP = fopen(tb.inputFile, "rb"))) goto error;
    if (!(tb.outP = fopen(tb.outputFile, "wb"))) goto error;
    pEncIn->busOutBuf[0] = tb.outbufMem.busAddress;
    pEncIn->outBufSize[0] = tb.outbufMem.size;
    pEncIn->pOutBuf[0] = tb.outbufMem.virtualAddress;
    pEncIn->cur_out_buffer[0] = &tb.outbufMem;
    pEncIn->dec400Enable = 0;
    for (i = 0; i < VCENC_MAX_LT_REF_FRAMES; i++)
        pEncIn->long_term_ref_pic[i] = INVALIDATED_POC;
    /* Video, sequence and picture parameter sets */
    if (VCEncStrmStart(h264_encoder, pEncIn, &encOut)) {
        /* Handle the error situation here */
        goto error;
    }
    /* Save the stream header */
    SaveStream((u8 *)pEncIn->pOutBuf[0], encOut.streamSize, tb.outP);
    streamSize += encOut.streamSize;
    /* Step 5: Encode the video frames */
    while (HANTRO_TRUE) {
        if (tb.picture_encoded_cnt >= tb.lastPic) break;
        /* Setup encoder input, we assume one linear buffer for the input image in */
        /* planar YCbCr 4:2:0 format */
        pEncIn->busLuma = tb.pictureMem.busAddress;
        tb.lum = (u8 *)tb.pictureMem.virtualAddress;
        pEncIn->busChromaU = pEncIn->busLuma + tb.lumaSize;
        tb.cb = tb.lum + tb.lumaSize;
        pEncIn->busChromaV = pEncIn->busChromaU + tb.chromaSize / 2;
        tb.cr = tb.cb + tb.chromaSize / 2;
        src_img_size =
            tb.width * tb.height * 12 / 8; /*bit_pre_pixel of yuv420 is 12
        /* Read YUV to input buffer */
        ReadYuv420PlanarFrame(&tb, src_img_size, tb.width, tb.height);
    }

```

```

printf("=== Encoding %i %s ...\n", tb.picture_encoded_cnt, tb.inputFile);
ret = VCEncStrmEncode(h264_encoder, pEncIn, &encOut, NULL, NULL);
/* Save encoded stream */
switch (ret) {
    case VCENC_FRAME_ENQUEUE:
        pEncIn->picture_cnt++;
        break;
    case VCENC_FRAME_READY:
        if (encOut.codingType != VCENC_NOTCODED_FRAME) pEncIn->picture_cnt++;
        if (encOut.streamSize != 0) {
            SaveStream((u8 *)pEncIn->pOutBuf[0], encOut.streamSize, tb.outP);
            streamSize += encOut.streamSize;
            tb.picture_encoded_cnt++;
        }
        break;
    case VCENC_SYSTEM_ERROR:
        /* Fatal system error, can't continue. */
    default:
        goto error;
}
} /* End of main encoding loop */
/* Flush encoder internal buffered frames
 * In Multicore enabled or 2-pass enabled, encoder internal cached frames
 * must be flushed
 */
ret = VCEncFlush(h264_encoder, pEncIn, &encOut, NULL, NULL);
/* Step 6: End stream */
ret = VCEncStrmEnd(h264_encoder, pEncIn, &encOut);
if (ret == VCENC_OK) {
    SaveStream((u8 *)pEncIn->pOutBuf[0], encOut.streamSize, tb.outP);
    streamSize += encOut.streamSize;
} else {
    /* Handle the error situation here*/
    goto error;
}
printf("Total of %d frames encoded, %lu bytes\n", tb.picture_encoded_cnt,
       (long unsigned int)streamSize);
error:
/* Free all resources */
if (tb.inP) {
    fclose(tb.inP);
    tb.inP = NULL;
}
if (tb.outP) {
    fclose(tb.outP);
    tb.outP = NULL;
}
FreeDMABuffer(&tb);
FreeVariableMem(&tb);
end:
/* Last Step: Release the encoder instance */
ret = VCEncRelease(h264_encoder);
return ret;
}

```

7.3 Optional Encoder Settings

```

#include "hevcencapi.h"
VCEncRet    ret;
VCEncInst    encoder;
VCEncCodingCtrl codingCfg;
VCEncRateCtrl rcCfg;
/* Make sure that an encoder instance has been initialized and is available. */
/* Rate control */

```

```
/* Obtain the current settings. */
if((ret = VCEncGetRateCtrl(encoder, &rcCfg)) != VCENC_OK)
{
    /* Handle errors here. */
}
else
{
    rcCfg.bitPerSecond = 2000*1000; /* 2000 Kbps */
    rcCfg.qpHdr = 25; /* First frame QP */
    /* Retain default values for all the other parameters. */
    /* Validate the new settings. */
    if((ret = VCEncSetRateCtrl(encoder, &rcCfg)) != VCENC_OK)
    {
        /* Handle errors here. */
    }
}
/* Coding control */
/* Obtain the current settings. */
if((ret = VCEncGetCodingCtrl(encoder, &codingCfg)) != VCENC_OK)
{
    /* Handle errors here. */
}
else
{
    codingCfg.sliceSize = 2; /* 2 macroblock rows in each slice */
    codingCfg.seiMessages = 1; /* Insert SEI messages into the stream. */
    /* Retain default values for all the other parameters. */
    /* Validate the new settings. */
    if((ret = VCEncSetCodingCtrl(encoder, &codingCfg)) != VCENC_OK)
    {
        /* Handle errors here. */
    }
}
```

8 Build and Test in Linux

8.1 Building the Video Encoder

To build the video encoder, run the following command. The generated binary is located at `/bin/${ENV}/.`

```
make [target] ENV=[environment] target=[configure] [variables]
```

NOTE: If you need to override an argument, re-run the command with the new argument settings.

The available values of the arguments in the command are described as follows:

Available Targets:

video	- VC9000E encoder binary for full video formats, excluding JPEG
hevc	- VC9000E encoder binary for HEVC (H.265)
h264	- VC9000E encoder binary for H.264 (AVC)
av1	- VC9000E encoder binary for AV1
vp9	- VC9000E encoder binary for VP9
jpeg	- VC9000E encoder binary for JPEG
libenc	- VC9000E encoder libh2enc.a
libva	- VC9000E C-Model library for VA-API
env	- Prints environment variables
clean	- Cleans libh2enc.a and all executable files
cfglist	- (C-Model only) Lists all configurations supported
cfgclean	- (C-Model only) Cleans C-Model library configurations only, ~
	used to change the target
cfgsys_mk	- (C-Model only) Generates enc_cfg.h for C-Model of the current target
cfgsys_rm	- (C-Model only) Removes enc_cfg.h for C-Model of the current target
libclean	- (C-Model only) Cleans the C-Model library
help	- Prints the help information

NOTE: Make sure that 'make libclean' is run between compiling to different C-Model targets.

Available Environments:

system	- (Default) System model in x86 Linux
pci	- VSI FPGA verification board in x86 Linux

Available Target Configurations:

The configure argument is mandatory for C-Model and is for C-Model only.

V9000	- VC9000E basic configuration
DEV	- VC9000E developing configuration with latest features

Available Configuration Variables:

TRACE = [y/n]	- (For system model only) Enables or disables tracing
DEBUG = [y/n]	- Enables debug symbols

8.1.1 Build Example

To build the video encoder for FPGA debugging, you can run the following command:

```
make clean libenc video ENV=pci DEBUG=y
```

8.2 Debugging the Video Encoder

The video encoder provides the following enumerations for debugging configuration. For information about other log system enumerations and functions, see Section [Log System API](#).

- **VCEncLogOutput** for specifying the target to which logs are output.

Available values include:

- **LOG_STDOUT**: outputs all logs to stdout.
- **LOG_ONE_FILE**: outputs all logs to the file `vcenc_trace_p${pid}.log` where `${pid}` is the process ID.

- **LOG_BY_THREAD**: outputs logs of each thread to a separate log file `vcenc_trace_p${pid}_t${tid}.log` where `${pid}` is the process ID and `${tid}` is the thread ID.
- **VCEncLogLevel** for specifying the log levels to be output.

Available values include:

- **VCENC_LOG_QUIET**: does not output logs.
- **VCENC_LOG_FATAL**: outputs logs of the fatal level.
- **VCENC_LOG_ERROR**: outputs logs of the error level and above.
- **VCENC_LOG_WARN**: outputs logs of the warning level and above.
- **VCENC_LOG_INFO**: outputs logs of the information level and above.
- **VCENC_LOG_DEBUG**: outputs logs of the debugging level and above.
- **VCENC_LOG_ALL**: outputs all logs.

- **VCEncLogTraceType** for selecting the trace log types to be output.

Available options include:

- **VCENC_LOG_TRACE_API**: encoder API call log
- **VCENC_LOG_TRACE_REGS**: register configuration log
- **VCENC_LOG_TRACE_EWL**: EWL API call log
- **VCENC_LOG_TRACE_MEM**: memory usage log
- **VCENC_LOG_TRACE_RC**: rate control status log
- **VCENC_LOG_TRACE_CML**: command line log
- **VCENC_LOG_TRACE_PERF**: performance log

- (Reserved) **VCEncLogCheckType** for selecting the check log types to be output.

Available options include:

- **VCENC_LOG_CHECK_RECON**: reconstructed YUV data
- **VCENC_LOG_CHECK_QUALITY**: PSNR/SSIM for each frame
- **VCENC_LOG_CHECK_VBV**: VBV information for rate control checking
- **VCENC_LOG_CHECK_RC**: rate control information for rate control profiling
- **VCENC_LOG_CHECK_FEATURE**: feature information for coverage checking

For log control options available in the test bench, see *Hantro VC9000E Video Encoder Test Bench User Manual*.

9 Module Documentation

9.1 Numeric Data Types

This section provides the definitions of numeric data types.

Typedef

- typedef int8_t **i8**
Signed 8-bit integer.
- typedef uint8_t **u8**
Unsigned 8-bit integer.
- typedef int16_t **i16**
Signed 16-bit integer.
- typedef uint16_t **u16**
Unsigned 16-bit integer.
- typedef int32_t **i32**
Signed 32-bit integer.
- typedef uint32_t **u32**
Unsigned 32-bit integer.
- typedef int64_t **i64**
Signed 64-bit integer.
- typedef uint64_t **u64**
Unsigned 64-bit integer.
- typedef **u64 ptr_t**
Buffer address.

9.1.1 Enumeration Type

9.1.1.1 bool_e

enum `bool_e`

Specifies whether a result is correct.

Enumerator

ENCHW_NOK	The result is incorrect.
ENCHW_OK	The result is correct.

9.1.1.2 true_e

enum `true_e`

Specifies a Boolean logic.

Enumerator

ENCHW_NO	False.
ENCHW_YES	True.

9.2 Basic Encoder API

Marco Definition

- #define `MAX_STRM_BUF_NUM` 2
The supported maximum number of stream buffers.
- #define `MAX_OVERLAY_NUM` 12
The supported maximum number of OSD regions.
- #define `MAX_MOSAIC_NUM` 12
The supported maximum number of mosaic regions.
- #define `MAX_LINE_MB_NUM` 2048
The supported maximum number of MB.

- **#define MAX_OSDMAP_COLOR_NUM MAX_OVERLAY_NUM**
The supported maximum number of OSD map colors, which is the same as maximum number of OSD regions.
- **#define MAX_CORE_NUM 4**
The supported maximum number of cores.
- **#define VCENC_MAX_REF_FRAMES 8**
The supported maximum number of reference frames.
- **#define MAX_GOP_PIC_CONFIG_NUM 48**
The maximum number of RPS descriptions in SPS.
- **#define MAX_GOP_SIZE 16**
The supported maximum GOP structure size.
- **#define MAX_ADAPTIVE_GOP_SIZE 8**
The maximum GOP structure size for adaptive size decision.
- **#define VCENC_MAX_LT_REF_FRAMES VCENC_MAX_REF_FRAMES**
The supported maximum number of long-term reference (LTR) frames.
Currently, a maximum of two LTR frames are supported.
- **#define MAX_GOP_SIZE_INUSE(gopSize) (gopSize <= MAX_ADAPTIVE_GOP_SIZE ? MAX_ADAPTIVE_GOP_SIZE : MAX_GOP_SIZE)**
The maximum GOP structure size available.
- **#define CUTREE_MAX_LOOKAHEAD_FRAMES(depth, gopSize) ((depth) + MAX_GOP_SIZE_INUSE(gopSize) / 2)**
The number of look-ahead frames for CuTree use.
CuTree requires at most ($\text{MAX_GOP_SIZE} - 1$) frames for completing a GOP.
For any GOP structure size, a maximum of $(\text{depth} + \text{MAX_GOP_SIZE} / 2)$ frames are queued in CuTree.
If the current GOP is still incomplete, CuTree works on frames excluding the current GOP.
- **#define CUTREE_BUFFER_CNT(depth, gopSize) ((depth) + MAX_GOP_SIZE_INUSE(gopSize) / 2 + 1)**
The number of CuTree buffers.
The extra buffer is used for the first frame in the CuTree queue, which is already processed or output.
- **#define DEC400_HEADER_BUF_SIZE (128)**
The size of DEC400 Tile Status Header Data if existing.

Typedef

- **typedef void(* EncInputLineBufCallBackFunc) (void *pAppData)**
The prototype of the callback function that is called when a line buffer interrupt is detected during encoding of a frame.
- **typedef void(* EncStreamMultiSegCallBackFunc) (void *pAppData)**
The prototype of the callback function that is called when a stream segment interrupt is detected during encoding of a frame.
- **typedef const void * VCEncInst**

The encoder instance.

- **typedef ReEncodeStrategy**(* VCEncTryNewParamsCallBackFunc) (const VCEncStatisticOut *stat, NewEncodeParams *new_params)

The prototype of the callback function that is called after encoding statistics are collected. This callback function needs to determine whether the current frame is to be re-encoded with new parameter settings based on the statistics.

stat: the input that indicates the statistics collected for the current frame.

new_params: the output that contains the updated encoding parameter settings.

For information about the returned value, see Section [ReEncodeStrategy](#).

- **typedef void**(* VCEncSliceReadyCallBackFunc) (VCEncSliceReady *sliceReady)

The prototype of the callback function that is called when one or some slice streams are generated by the hardware and the software can then process the data in the output stream buffer without waiting for encoding of the whole frame to be completed.

For information about the input parameters, see Section [VCEncSliceReady](#).

- **typedef VceApiVersion VCEncApiVersion**

The API version information.

- **typedef VceBuildInfo VCEncBuild**

The build information.

9.2.1 Enumeration Type

9.2.1.1 VCEncRet

enum **VCEncRet**

Specifies function return values.

Enumerator

VCENC_OK	The API call is successful.
VCENC_FRAME_READY	Encoding of a frame is finished.
VCENC_FRAME_ENQUEUE	A frame is inside the encoder's internal queue but not encoded. It will be encoded and output in a subsequent encoder API call.
VCENC_FRAME_CACHE	A encoding frame task is cached and will drive the hardware in batch after accumulate some tasks.

Enumerator

VCENC_FRAME_CONTINUE	A sideline frame is encoded and application should continue to execute job in the working queue.
VCENC_ERROR	(Error) An encoder error occurs.
VCENC_NULL_ARGUMENT	(Error) A pointer argument has an invalid NULL value.
VCENC_INVALID_ARGUMENT	(Error) One of the arguments is invalid. None of the argument settings takes effect.
VCENC_MEMORY_ERROR	(Error) The encoder fails to allocate memory.
VCENC_EWL_ERROR	(Error) Initialization of the encoder system interface fails.
VCENC_EWL_MEMORY_ERROR	(Error) The EWL fails to allocate memory.
VCENC_INVALID_STATUS	(Error) The stream is started with HRD enabled and rate control parameters fails to be altered.
VCENC_OUTPUT_BUFFER_OVERFLOW	(Error) The output buffer is too small to hold the generated stream. Please allocate a larger buffer and try again.
VCENC_HW_BUS_ERROR	(Error) Memory access fails due to invalid bus address. Please reset the hardware.
VCENC_HW_DATA_ERROR	(Error) An error occurs in the hardware data.
VCENC_HW_TIMEOUT	(Error) Hardware execution has timed out. The current frame is lost. Please do some clean-up and then encode a new frame.
VCENC_HW_RESERVED	(Error) The hardware fails to be reserved for exclusive access.
VCENC_SYSTEM_ERROR	(Error) A fatal system error occurs and the encoding is terminated. Please release the encoder instance.
VCENC_INSTANCE_ERROR	(Error) The encoder instance is invalid or corrupted.
VCENC_HRD_ERROR	(Error) An HRD error occurs.
VCENC_HW_RESET	The hardware is reset by an external operation, for example, resetting the encoder core to clean up the error status when an error occurs.

Enumerator

VCENC_HW_POLL_SLICEINFO_TIMEOUT	(Error) During low-latency encoding in DDR mode, excessive cycles have been used to poll for an input row to be encoded but no valid data is found in input picture buffers. For details, see Section <i>Low-Latency Encoding Flows</i> .
VCENC_HW_UFBC_ERROR	(Error) An UFBC decoding error occurs when the input picture is fetched.
VCENC_SBI_ERROR	(Error) An SBI error occurs.

9.2.1.2 VCEncVideoCodecFormatenum *VCEncVideoCodecFormat*

Specifies video codec standards.

Enumerator

VCENC_VIDEO_CODEC_HEVC	Codec standard HEVC (H.265).
VCENC_VIDEO_CODEC_H264	Codec standard H.264 (AVC).
VCENC_VIDEO_CODEC_AV1	Codec standard AV1.
VCENC_VIDEO_CODEC_VP9	Codec standard VP9.

9.2.1.3 VCEncStreamTypeenum *VCEncStreamType*

Specifies stream types for HEVC (H.265) or H.264 (AVC) encoding.

Enumerator

VCENC_BYTE_STREAM	Byte stream, where each NAL unit starts with hexadecimal bytes '00 00 00 01'.
VCENC_NAL_UNIT_STREAM	NAL unit stream, which consists of plain NAL units without a start code.

9.2.1.4 VCEncLevel

enum `VCEncLevel`

Specifies stream levels for initialization.

Table 9.6 VCEnc Level Enumeration Values Defined for HEVC (H.265)

Value	Encoded Picture Size	Frame Rate (FPS)	Max Luma Picture Size (Samples)	Max Luma Sample Rate (Samples/Sec)	Max Bit Rate (kbps)
VCENC_HEVC_LEVEL_1	QCIF	15	36,864	552,960	128
VCENC_HEVC_LEVEL_2	CIF	30	122,880	3,686,400	1,500
VCENC_HEVC_LEVEL_2_1	Q720p	30	245,760	7,372,800	3,000
VCENC_HEVC_LEVEL_3	QHD	30	552,960	16,588,800	6,000
VCENC_HEVC_LEVEL_3_1	1280x720	30	983,040	33,177,600	10,000
VCENC_HEVC_LEVEL_4	2Kx1080	30	2,228,224	66,846,720	12,000
VCENC_HEVC_LEVEL_4_1	2Kx1080	60	2,228,224	133,693,440	20,000
VCENC_HEVC_LEVEL_5	4096x2160	30	8,912,896	267,386,880	25,000
VCENC_HEVC_LEVEL_5_1	4096x2160	60	8,912,896	534,773,760	40,000

Value	Encoded Picture Size	Frame Rate (FPS)	Max Luma Picture Size (Samples)	Max Luma Sample Rate (Samples/Sec)	Max Bit Rate (kbps)
VCENC_HEVC_LEVEL_5_2	4096x2160	120	8,912,896	1,069,547,520	60,000
VCENC_HEVC_LEVEL_6	8192x4320	30	35,651,584	1,069,547,520	60,000
VCENC_HEVC_LEVEL_6_1	8192x4320	60	35,651,584	2,139,095,040	120,000
VCENC_HEVC_LEVEL_6_2	8192x4320	120	35,651,584	4,278,190,080	240,000

Table 9.7 VCEnc Level Enumeration Values Defined for H.264 (AVC)

Value	Encoded Picture Size	Frame Rate (FPS)	Max Luma Picture Size (Samples)	Max Luma Sample Rate (Samples/Sec)	Max Bit Rate (kbps)
VCENC_H264_LEVEL_1	Sub-QCIF QCIF	15 15	99 (QCIF)	1,485	64
VCENC_H264_LEVEL_1_b	Sub-QCIF QCIF	15 15	99 (QCIF)	1,485	128
VCENC_H264_LEVEL_1_1	QCIF QVGA	30 10	396 (QCIF)	3,000	192
VCENC_H264_LEVEL_1_2	QVGA CIF	20 15	396 (QCIF)	6,000	384
VCENC_H264_LEVEL_1_3	QVGA CIF	30 30	396 (QCIF)	11,880	768
VCENC_H264_LEVEL_2	CIF	30	396 (QCIF)	11,880	2,000
VCENC_H264_LEVEL_2_1	512x384	25	792	19,800	4,000
VCENC_H264_LEVEL_2_2	VGA 720x480	15 15	1,620 (PAL)	20,250	4,000

Value	Encoded Picture Size	Frame Rate (FPS)	Max Luma Picture Size (Samples)	Max Luma Sample Rate (Samples/Sec)	Max Bit Rate (kbps)
VCENC_H264_LEVEL_3	VGA	30	1,620 (PAL)	40,500	10,000
	720x576	25			
	720x480	30			
VCENC_H264_LEVEL_3_1	1280x720	30	3,600 (HD 720p)	108,000	14,000
VCENC_H264_LEVEL_3_2	1280x720	60	5,120 (SXGA)	216,000	20,000
VCENC_H264_LEVEL_4	2048x1024	30	8,192 (1080p)	245,760	20,000
VCENC_H264_LEVEL_4_1	2048x1024	30	8,192 (1080p)	245,760	50,000
VCENC_H264_LEVEL_4_2	2048x1024	60	8,192 (1080p)	49,1520	50,000
VCENC_H264_LEVEL_5	3680x1536	25	22,080	58,9824	135,000
VCENC_H264_LEVEL_5_1	4096x2048	30	36,864	983,040	240,000
VCENC_H264_LEVEL_5_2	4096x2160	60	36,864	2,073,600	240,000
VCENC_H264_LEVEL_6	8192x4320	30	35,651,584	4,177,920	240,000
VCENC_H264_LEVEL_6_1	8192x4320	60	35,651,584	8,355,840	480,000
VCENC_H264_LEVEL_6_2	8192x4320	120	35,651,584	16,711,680	800,000

9.2.1.5 VCEncProfile

enum `VCEncProfile`

Specifies stream profiles for initialization.

For the profile support of the video encoder, see *Hantro VC9000E Series Video Encoder Hardware Features*.

Enumerator

VCENC_ADAPTIVE_PROFILE	Set profile according to codec and feature selected for encoding
VCENC_HEVC_MAIN_PROFILE	The HEVC (H.265) profile for a bit depth of 8 bits per sample with YUV420 sampling.
VCENC_HEVC_MAIN_STILL_PICTURE_PROFILE	The HEVC profile for a single still picture to be encoded with the same constraints as the Main profile.
VCENC_HEVC_MAIN_10_PROFILE	The HEVC profile for a bit depth of 8 to 10 bits per sample with YUV420 sampling.
VCENC_HEVC_MONOCHROME_PROFILE	The HEVC Monochrome profile for bit depth of 8 bits per sample with YUV400 sampling
VCENC_HEVC_MONOCHROME_10_PROFILE	The HEVC Monochrome profile for bit depth of 10 bits per sample with YUV400 sampling.
VCENC_HEVC_MAIN_10_STILL_PICTURE_PROFILE	The HEVC profile for a single still picture to be encoded with the same constraints as the Main 10 profile.
VCENC_HEVC_MAIN_422_10_PROFILE	The HEVC Monochrome profile for bit depth of 10 bits per sample with YUV422 sampling.
VCENC_H264_BASE_PROFILE	The H.264 (AVC) Baseline profile for video conferencing and mobile applications.
VCENC_H264_MAIN_PROFILE	The H.264 profile for standard-definition digital TV broadcasts that use the MPEG-4 format.
VCENC_H264_HIGH_PROFILE	The H.264 primary profile for broadcast and disk storage applications, particularly for high-definition TV applications.
VCENC_H264_HIGH_10_PROFILE	The H.264 profile for up to 10 bits per sample, which is built on top of the High profile.
VCENC_AV1_MAIN_PROFILE	The AV1 Main profile for a bit depth of 8 or 10 bits per sample with YUV420 sampling.
VCENC_AV1_HIGH_PROFILE	The AV1 High profile for a bit depth of 8 or 10 bits per sample with YUV420 sampling. NOT SUPPORT.

Enumerator

VCENC_AV1_PROFESSIONAL_PROFILE	The AV1 Professional profile for a bit depth of 8 or 10 bits per sample with YUV420 sampling. NOT SUPPORT.
VCENC_VP9_0_PROFILE	The VP9 Main profile for a bit depth of 8 bits per sample with YUV420 non-sRGB sampling.
VCENC_VP9_1_PROFILE	The VP9 Main profile for a bit depth of 8 bits per sample with YUV422, YUV440, or YUV444 sRGB sampling. NOT SUPPORT.
VCENC_VP9_2_PROFILE	The VP9 Main profile for a bit depth of 10 or 12 bits per sample with YUV420 non-sRGB sampling.
VCENC_VP9_3_PROFILE	The VP9 Main profile for a bit depth of 10 bits per sample with YUV422, YUV440, or YUV444 sRGB sampling. NOT SUPPORT.

9.2.1.6 VCEncTierenum **VCEncTier**

Specifies stream tiers for initialization.

Enumerator

VCENC_HEVC_MAIN_TIER	The Main tier.
VCENC_HEVC_HIGH_TIER	The High tier.

9.2.1.7 VCEncPictureTypeenum **VCEncPictureType**

Specifies input color formats for initialization.

For details about color formats, see *Hantro VC9000E Series Memory Buffer and Format Organization*.

Enumerator

VCENC_YUV420_PLANAR	0 - YUV420P8b_Raster_A16N.
VCENC_YUV420_SEMIPLANAR	1 - YUV420SP8b_Raster_A16N.
VCENC_YUV420_SEMIPLANAR_VU	2 - YVU420SP8b_Raster_A16N.
VCENC_YUV422_INTERLEAVED_YUYV	3 - YUV422YUYV8b_Raster_A16N.
VCENC_YUV422_INTERLEAVED_UYVY	4 - YUV422UYVY8b_Raster_A16N.
VCENC_RGB565	5 - RGB565_Raster_A16N.
VCENC_BGR565	6 - BGR565_Raster_A16N.
VCENC_RGB555	7 - XRGB1555_Raster_A16N.
VCENC_BGR555	8 - XBGR1555_Raster_A16N.
VCENC_RGB444	9 - XRGB4444_Raster_A16N.
VCENC_BGR444	10 - XBGR4444_Raster_A16N.
VCENC_RGB888	11 - XRGB8888_Raster_A16N.
VCENC_BGR888	12 - XBGR8888_Raster_A16N.
VCENC_RGB101010	13 - XRGB2101010_Raster_A16N.
VCENC_BGR101010	14 - XBGR2101010_Raster_A16N.
VCENC_YUV420_PLANAR_10BIT_I010	15 - YUV420P10bWL_Raster_A16N.
VCENC_YUV420_PLANAR_10BIT_P010	16 - YUV420SP10bWH_Raster_A16N.
	17 - YUV420P10b_Raster_A16N.
VCENC_YUV420_PLANAR_10BIT_PACKED_PLANAR	
VCENC_YUV420_10BIT_PACKED_Y0L2	18 - YUV420Y0L210b_Raster_A16N.
	19 - (Reserved)
VCENC_YUV420_PLANAR_8BIT_TILE_32_32	20 - YUV420P8b_Tile32x32_A16N for HEVC.
	20 - (Reserved)
VCENC_YUV420_PLANAR_8BIT_TILE_16_16	21 - YUV420P8b_Tile16x16_A16N for H.264 (AVC).
	21 - YUV420SP8b_YuvSp4x4_A16N.
VCENC_YUV420_SEMIPLANAR_8BIT_TILE_4_4	22 - YVU420SP8b_YuvSp4x4_A16N.
	22 - YVU420SP8b_YuvSp4x4_A16N.
VCENC_YUV420_SEMIPLANAR_VU_8BIT_TILE_4_4	23 - YUV420SP10bWH_YuvSp4x4_A16N.
	23 - YUV420SP10bWH_YuvSp4x4_A16N.
VCENC_YUV420_PLANAR_10BIT_P010_TILE_4_4	
VCENC_YUV420_SEMIPLANAR_101010	24 - YUV420SP10bDWL_Raster_A16N.

Enumerator

VCENC_YUV422_SEMIPLANAR_888	25 - YUV422SP8b_Raster_A16N
VCENC_YUV420_8BIT_TILE_64_4	26 - (Reserved) YVU420SP8b_Tile64x4_A16N.
VCENC_YUV420_UV_8BIT_TILE_64_4	27 - (Reserved) YUV420SP8b_Tile64x4_A16N.
VCENC_YUV420_10BIT_TILE_32_4	28 - (Reserved) YUV420SP10bWH_Tile32x4_A16N.
VCENC_YUV420_10BIT_TILE_48_4	29 - (Reserved) YUV420SP10bWH_Tile48x4_A16N.
VCENC_YUV420_VU_10BIT_TILE_48_4	30 - (Reserved) YVU420SP10bWH_Tile48x4_A16N.
VCENC_YUV420_8BIT_TILE_128_2	31 - (Reserved) YVU420SP8b_Tile128x2_A16N.
VCENC_YUV420_UV_8BIT_TILE_128_2	32 - (Reserved) YUV420SP8b_Tile128x2_A16N.
VCENC_YUV420_10BIT_TILE_96_2	33 - (Reserved) YUV420SP10bWH_Tile96x2_A16N.
VCENC_YUV420_VU_10BIT_TILE_96_2	34 - (Reserved) YVU420SP10bWH_Tile96x2_A16N.
VCENC_YUV420_8BIT_TILE_8_8	35 - YUV420SP8b_YuvSp8x8_A16N.
VCENC_YUV420_10BIT_TILE_8_8	36 - YUV420SP10b_YuvSp8x8_A16N.
VCENC_YVU420_PLANAR	37 - YVU420P8b_Raster_A16N.
VCENC_YUV420_UV_8BIT_TILE_64_2	38 - (Reserved) YUV420SP8b_Tile64x2_A16N.
VCENC_YUV420_UV_10BIT_TILE_128_2	39 - (Reserved) YUV420SP10bWH_Tile128x2_A16N.
VCENC_RGB888_24BIT	40 - RGB888_Raster_A16N.
VCENC_BGR888_24BIT	41 - BGR888_Raster_A16N.
VCENC_RBG888_24BIT	42 - RBG888_Raster_A16N.
VCENC_GBR888_24BIT	43 - GBR888_Raster_A16N.
VCENC_BRG888_24BIT	44 - BRG888_Raster_A16N.
VCENC_GRB888_24BIT	45 - GRB888_Raster_A16N.
VCENC_YVU422_SEMIPLANAR_888	46 - YYYY... VUVU...
VCENC_YUV444_PLANAR	YYYY... UUUU... VVVV...
VCENC_YUV444_XYUV8888	X8Y8U8V8...
VCENC_YUV444_XYUV2101010	X2Y10U10V10...

Enumerator

VCENC_RGBX8888	50 - RGBX8888_Raster_A16N
VCENC_BGRX8888	51 - BGRX8888_Raster_A16N
VCENC_RGBX1010102	52 - RGBX1010102_Raster_A16N
VCENC_BGRX1010102	53 - BGRX1010102_Raster_A16N
VCENC_YVU420_PLANAR_10BIT_P010	54 - YVU420SP10bWH_Raster_A16N
VCENC_Y8b	55 - Y8b_Raster_A16N (YCbCr 4:0:0) Monochrome 8-bit
VCENC_Y10bWL	56 - Y10bWL_Raster_A16N (YCbCr 4:0:0) Monochrome 10-bit occupies 2B [9:0]
VCENC_Y10bWH	57 - Y10bWH_Raster_A16N (YCbCr 4:0:0) Monochrome 10-bit occupies 2B [15:6]
VCENC_YUV420SP10b	58 - YUV420SP10b_Raster_A16N
VCENC_YUV422P10bWL	59 - YUV422P10bWL_Raster_A16N.
VCENC_Y8b_TILE_8_8	60 - Y8b_Tile8x8_A16N.
VCENC_Y10bWH_TILE_8_8	61 - Y10bWH_Tile8x8_A16N.
VCENC_YUV422_INTERLEAVED_YVYU	62 - YUV422YVYU8b_Raster_A16N.
VCENC_YUV422_INTERLEAVED_VYUY	63 - YUV422VYUY8b_Raster_A16N.
VCENC_YVU420_8BIT_TILE_8_8	64 - YVU420SP8b_YuvSp8x8_A64N.
VCENC_YUV400_PLANAR_10BIT_PACKED	65 - Y10b_Raster_A16N.

9.2.1.8 VCEncPictureScanType

```
enum VCEncPictureScanType
```

Specifies picture scan types for pre-processing.

This enumeration is valid only if the value of `EVLHwConfig_t.superTileXSupport` is 0.

Enumerator

VCENC_RASTER_SCAN	raster scan
VCENC_SUPERTILEX_SCAN	supertileX scan

9.2.1.9 VCEncPictureRotation

enum `VCEncPictureRotation`

Specifies picture rotation types for pre-processing.

This enumeration is valid only if the value of `EVLHwConfig_t.NonRotationSupport` is 0.

Enumerator

<code>VCENC_ROTATE_0</code>	Does not rotate.
<code>VCENC_ROTATE_90R</code>	Rotates 90 degrees clockwise.
<code>VCENC_ROTATE_90L</code>	Rotates 90 degrees counterclockwise.
<code>VCENC_ROTATE_180R</code>	Rotates 180 degrees clockwise.

9.2.1.10 VCEncPictureMirror

enum `VCEncPictureMirror`

Specifies picture mirroring modes for pre-processing.

This enumeration is valid only if the value of `EVLHwConfig_t.prpMirrorSupport` is 1.

Enumerator

<code>VCENC_MIRROR_NO</code>	Does not mirror the pictures.
<code>VCENC_MIRROR_YES</code>	Mirrors the pictures.

9.2.1.11 VCEncColorConversionType

enum `VCEncColorConversionType`

Specifies RGB-to-YUV conversion types for pre-processing.

Enumerator

VCENC_RGBTOYUV_BT601	Conversion to YUV values without any range change according to Rec. ITU-R BT.601.
VCENC_RGBTOYUV_BT709	Conversion to YUV values without any range change according to Rec. ITU-R BT.709.
VCENC_RGBTOYUV_USER_DEFINED	Conversion using custom coefficients. For the conversion equations, see Section <i>RGB-to-YUV Conversion</i> .
VCENC_RGBTOYUV_BT2020	Conversion according to Rec. ITU-R BT.2020.
VCENC_RGBFULL_TO_YUVLIMIT_BT601	Conversion from full range RGB to limited range YUV according to Rec. ITU-R BT.601.
VCENC_RGBTOYUV_BT601_FULL_RANGE	(obsolete) Same as VCENC_RGBFULL_TO_YUVLIMIT_BT601.
VCENC_RGBTOYUV_BT601_LIMITED_RANGE	(Just for test) Conversion from RGB in range of (0~219) to limited range YUV according to Rec. ITU-R BT.601.
VCENC_RGBFULL_TO_YUVLIMIT_BT709	Conversion from full range RGB to limited range YUV according to Rec. ITU-R BT.709.
VCENC_RGBTOYUV_BT709_FULL_RANGE	(obsolete) Same as VCENC_RGBFULL_TO_YUVLIMIT_BT709.
VCENC_RGBTOYUV_MAX	number of supported color conversion type

9.2.1.12 VCEncPictureCodingType

enum *VCEncPictureCodingType*

Specifies picture coding types for encoding.

Enumerator

VCENC_INTRA_FRAME	Intra-codes the picture.
VCENC_PREDICTED_FRAME	Inter-codes the picture by using a previous picture as a predictor.

Enumerator

VCENC_BIDIR_PREDICTED_FRAME	Inter-codes the picture by using previous and/or subsequent pictures as predictors.
VCENC_NOTCODED_FRAME	Does not code the picture.

9.2.1.13 VCENC_AV1_FRAME_TYPE

```
enum VCENC_AV1_FRAME_TYPE
```

Specifies frame types for AV1 encoding.

Enumerator

VCENC_AV1_KEY_FRAME	Key frame.
VCENC_AV1_INTER_FRAME	Inter frame.
VCENC_AV1_INTRA_ONLY_FRAME	Intra-only frame.
VCENC_AV1_S_FRAME	S-frame.

9.2.1.14 VCEncHDRTransferType

```
enum VCEncHDRTransferType
```

Specifies HDR10 transfer function types.

Enumerator

VCENC_HDR10_BT2020	The transfer functions defined in ITU-R-REC-BT.2020.
VCENC_HDR10_ST2084	The transfer functions defined in SMPTE ST 2084.
VCENC_HDR10_STDB67	The transfer functions defined in ARIB STD-B67.

9.2.1.15 VCEncChromaIdcType

enum `VCEncChromaIdcType`

Specifies chroma sampling modes relative to luma sampling. For details, see Section *Coding Formats and Profiles*.

This enumeration is used as the SPS syntax element `chroma_format_idc` specified in ITU-T Rec. H.265 and H.264.

Enumerator

<code>VCENC_CHROMA_IDC_400</code>	(Only for H.265/AVC) YUV400.
<code>VCENC_CHROMA_IDC_420</code>	YUV420.
<code>VCENC_CHROMA_IDC_422</code>	(Under development) YUV422.

9.2.1.16 VCENC_TuneType

enum `VCENC_TuneType`

Specifies target types based on which encoding parameters are forced for quality tuning. For details, see Section *Tuning for Various Quality Targets*.

Enumerator

<code>VCENC_TUNE_PSNR</code>	Peak signal to noise ratio (PSNR). Forced parameters include <code>aq_mode=0</code> and <code>psyFactor=0</code> .
<code>VCENC_TUNE_SSIM</code>	Structural similarity index measure (SSIM). Forced parameters include <code>aq_mode=2</code> and <code>psyFactor=0</code> .
<code>VCENC_TUNE_VISUAL</code>	Visual. Forced parameters include <code>aq_mode=2</code> and <code>psyFactor=0.75</code> .
<code>VCENC_TUNE_SHARP_VISUAL</code>	Sharpness-based visual. Forced parameters include <code>aq_mode=2</code> , <code>psyFactor=0.75</code> , <code>inLoopDSRatio=0</code> , and <code>enableRdoQuant=0</code> .

9.2.1.17 ReEncodeStrategy

enum `ReEncodeStrategy`

Defines re-encoding strategies.

Enumerator

<code>NO_RE_ENCODE</code>	Does not re-encode the frame.
<code>NEW_QP</code>	Re-encodes the frame with a new slice QP value.
<code>NEW_OUTPUT_BUFFER</code>	Re-encodes the frame with a new output stream buffer.
<code>NEW_TARGET_BIT</code>	Re-encodes the frame with a new target bit rate for block-level rate control.
<code>NEW_RE_ENCODE_FOR_TIMEOUT</code>	Re-encodes the frame for timeout.

9.2.2 Functions

9.2.2.1 VCEncGetApiVersion()

```
VCEncApiVersion VCEncGetApiVersion (
    void )
```

Returns the API version information.

This function requires no parameters.

Returns

`VCEncApiVersion`

9.2.2.2 VCEncGetBuild()

```
VCEncBuild VCEncGetBuild (
    u32 client_type )
```

Returns software or hardware build information of a client type.

This function can be called either before or after encoder instance initialization.

Parameters

in	<i>client_type</i>	The type of the client whose build information is to be queried.
----	--------------------	--

Returns

VCEncBuild

9.2.2.3 VCEncGetRoiMapVersion()

```
u32 VCEncGetRoiMapVersion (
    u32 core_id,
    void * ctx )
```

Queries the ROI map version supported by the hardware.

For details about ROI map versions, see *Hantro VC9000E Series Memory Buffer and Format Organization*.

Parameters

in	<i>core_id</i>	The ID of the subsystem to be queried.
in	<i>ctx</i>	The context of the current device.

Returns

The ROI map version

9.2.2.4 VCEncGetBitsPerPixel()

```
u32 VCEncGetBitsPerPixel (
    VCEncPictureType type )
```

Returns the number of bits per pixel of an input color format.

Parameters

in	<i>type</i>	The input color format to be queried.
----	-------------	---------------------------------------

Returns

The number of bits per pixel of the specified input color format

9.2.2.5 VCEncGetAlignedStride()

```
u32 VCEncGetAlignedStride (
    int width,
    i32 input_format,
    u32 * luma_stride,
    u32 * chroma_stride,
    u32 input_alignment,
    u32 scan_type )
```

Returns the stride in bytes based on the given input picture information.

Parameters

in	<i>width</i>	The width of the input picture.
in	<i>input_format</i>	The color format of the input picture.
out	<i>luma_stride</i>	A pointer to the storage space for receiving the luma stride.
out	<i>chroma_stride</i>	A pointer to the storage space for receiving the chroma stride.
in	<i>input_alignment</i>	The alignment of the input picture.

Returns

The stride in bytes

9.2.2.6 VCEncInit()

```
VCEncRet VCEncInit (
    const VCEncConfig * config,
    VCEncInst * instAddr,
    void * ctx )
```

Creates and initializes an encoder instance.

Before instance initialization, make sure that the EWL layer and kernel driver are ported to your platform.

The encoder instance must be freed using `VCEncRelease()` when it is no longer needed.

Parameters

in	<i>config</i>	A pointer to the configurations for initialization of the encoder instance. Most configurations cannot be altered during encoding.
out	<i>instAddr</i>	A pointer to the storage space for receiving the created encoder instance.
in	<i>ctx</i>	A pointer to the context from the application. The context may be used by EWL. For example, when the device file is open before initialization, this parameter can be used to transfer the file data. Set it to NULL in case of no context.

Returns

VCEncRet

9.2.2.7 VCEncRelease()

```
VCEncRet VCEncRelease (  
    VCEncInst inst )
```

Releases an encoder instance created using `VCEncInit()`, and frees all the resources allocated during encoder initialization.

Parameters

in	inst	The encoder instance to be released. After this function is successfully executed, the specified pointer is no longer valid.
----	------	---

Returns

VCEncRet

9.2.2.8 VCEncGetPerformance()

```
u32 VCEncGetPerformance (  
    VCEncInst inst )
```

Queries the number of hardware clock cycles used to encode the previous frame.

Parameters

in	inst	The encoder instance to be queried.
----	------	-------------------------------------

Returns

The number of clock cycles used to encode the previous frame

9.2.2.9 VCEncSetStrmCtrl()

```
VCEncRet VCEncSetStrmCtrl (  
    VCEncInst inst,  
    VCEncStrmCtrl * params )
```

Configures stream-level encoding control for an encoder instance.

This function is applicable only to encoder instances in INIT state. For details, see Section *Stream-Level Encoding Control*.

Parameters

in	<i>inst</i>	The encoder instance to be configured.
in	<i>params</i>	A pointer to the stream-level encoding control parameters.

Returns

VCEncRet

9.2.2.10 VCEncSetSeiUserData()

```
VCEncRet VCEncSetSeiUserData (  
    VCEncInst inst,  
    const u8 * pUserData,  
    u32 userDataSize )
```

Enables or disables writing user data to the encoded stream as a supplemental enhancement information (SEI) message to all subsequent encoded frames.

The payload type of the SEI message is `user_data_unregistered`. For details, see ITU-T Rec. H.265.

Parameters

in	<i>inst</i>	The encoder instance to be configured.
----	-------------	--

Parameters

in	<i>pUserData</i>	A pointer to the buffer that contains the user data. The video encoder reads data straight from the buffer during encoding of the subsequent frames. Therefore, the specified pointer to the buffer must not be freed until the user data writing is disabled.
in	<i>userDataSize</i>	The size of the user data in the buffer, in bytes. This parameter can be set to 0 or any value from 16 to 2048. If this parameter is set to 0, the user data writing is disabled.

Returns

VCEncRet

9.2.2.11 VCEncStrmGetOutput()

```
VCEncRet VCEncStrmGetOutput (
    VCEncInst inst,
    VCEncIn * pEncIn,
    VCEncOut * pEncOut,
    VCEncSliceReadyCallBackFunc sliceReadyCbFunc,
    void * pAppData )
```

Flushes out remaining frames after all the frames to be encoded have been send into encoder.

Parameters

in	<i>inst</i>	The instance that defines the encoder in use.
in	<i>pEncIn</i>	Pointer to user provided input parameters.
in	<i>sliceReadyCbFunc</i>	Pointer to the VCEncSliceReadyCallBackFunc callback function that will be called after a slice encoding has been finished by the hardware.
in	<i>pAppdata</i>	Pointer to application-specific data to be passed on to the callback function.
out	<i>pEncOut</i>	Pointer to the VCEncOut structure where the output parameters will be saved.

Returns

VCEncRet

9.2.2.12 VCEncSetTestId()

```
VCEncRet VCEncSetTestId (  
    VCEncInst inst,  
    u32 testId )
```

Imports encoder configurations from a test vector.

Parameters

in	<i>inst</i>	The encoder instance to use the configurations.
in	<i>testId</i>	The ID of the test vector.

Returns

VCEncRet

9.2.2.13 VCEncSetInputMBLines()

```
VCEncRet VCEncSetInputMBLines (  
    VCEncInst inst,  
    u32 lines )
```

Sets the valid input MB line to be encoded with low-latency encoding by an encoder instance.

Parameters

in	<i>inst</i>	The encoder instance to be configured.
in	<i>lines</i>	The order count of the input MB line to be encoded.

Returns**VCEncRet****9.2.2.14 VCEncGetEncodedMbLines()**

```
u32 VCEncGetEncodedMbLines (
    VCEncInst inst )
```

Queries the MB line encoded with low-latency encoding by an encoder instance.

Parameters

in	<i>inst</i>	The encoder instance to be queried.
----	-------------	-------------------------------------

Returns

The order count of the MB line encoded with low-latency encoding

9.2.2.15 VCEncSetError()

```
void VCEncSetError (
    VCEncInst inst )
```

Enables the error status for an encoder instance.

This function is useful in look-ahead encoding.

Parameters

in	<i>inst</i>	The encoder instance for which the error status is to be set.
----	-------------	---

Returns

None

9.2.2.16 VCEncGetCuInfo()

```
VCEncRet VCEncGetCuInfo (
    VCEncInst inst,
    VCEncCuOutData * pEncCuOutData,
    u32 ctuNum,
    u32 cuNum,
    VCEncCuInfo * pEncCuInfo )
```

(For HEVC only) Parses the encoding information of a CU from the output of a picture.

Parameters

in	<i>inst</i>	The encoder instance that outputs the picture.
in	<i>pEncCuOutData</i>	A pointer to the CU information of the picture output.
in	<i>ctuNum</i>	The index of the CTU within the picture.
in	<i>cuNum</i>	The index of the CU within the CTU.
out	<i>pEncCuInfo</i>	A pointer to the storage space for receiving the CU information.

Returns**VCEncRet****9.2.2.17 VCEncGetEncodeMaxDelay()**

```
i32 VCEncGetEncodeMaxDelay (
    VCEncInst inst )
```

Queries the updated GOP structure size (*gopSize*) after pass-1 encoding.

Parameters

in	<i>inst</i>	The encoder instance whose GOP structure size is to be queried.
----	-------------	---

Returns

The updated GOP structure size after pass-1 encoding Queries the maximum encoding delay of an encoder instance.

Parameters

in	<i>inst</i>	The encoder instance whose encoding buffer delay is to be queried.
----	-------------	--

Returns

The maximum encoding delay, in the unit of frame

9.2.2.18 VCEncGetClientType()

```
CLIENT_TYPE VCEncGetClientType (
    VCEncVideoCodecFormat codecFormat )
```

Translates an API-defined codec standard into a EWL-defined client type.

Parameters

in	<i>codecFormat</i>	The codec standard defined in API.
----	--------------------	------------------------------------

Returns

CLIENT_TYPE defined in EWL

9.3 Coding Control API

9.3.1 Functions

9.3.1.1 VCEncSetCodingCtrl()

```
VCEncRet VCEncSetCodingCtrl (  
    VCEncInst instAddr,  
    const VCEncCodingCtrl * pCodeParams )
```

Configures coding control for an encoder instance.

This function can be called to alter all the coding control parameters for encoder instances in INIT state. For encoder instances in START_STREAM or START_FRAME state, some parameters are alterable between frames without the need to restart the stream. However, to alter the other parameters, you need to restart the stream. For more information, see Section *Coding Control Settings*.

Parameters

in	<i>instAddr</i>	The encoder instance to be configured.
in	<i>pCodeParams</i>	A pointer to the coding control parameters.

Returns

VCEncRet

9.3.1.2 VCEncGetCodingCtrl()

```
VCEncRet VCEncGetCodingCtrl (
    VCEncInst inst,
    VCEncCodingCtrl * pCodeParams )
```

Queries the coding control settings of an encoder instance.

Parameters

in	<i>inst</i>	The encoder instance to be queried.
out	<i>pCodeParams</i>	A pointer to the storage space for receiving the coding control settings.

Returns

VCEncRet

9.4 Bit Rate Control API

9.4.1 Enumeration Type

9.4.1.1 rcMode_e

```
enum rcMode_e
```

Specifies rate control modes.

Enumerator

VCE_RC_CVBR	The constrained variable bit rate (CVBR) mode.
VCE_RC_CBR	The constant bit rate (CBR) mode.
VCE_RC_VBR	The variable bit rate (VBR) mode.
VCE_RC_ABR	The average bit rate (ABR) mode.
VCE_RC_CRF	The constant rate factor (CRF) mode.
VCE_RC_CQP	The constant quantization parameter (CQP) mode.

9.4.2 Functions

9.4.2.1 VCEncSetRateCtrl()

```
VCEncRet VCEncSetRateCtrl (
    VCEncInst inst,
    const VCEncRateCtrl * pRateCtrl )
```

Configures bit rate control for an encoder instance.

This function can be called to alter all the bit rate control parameters for encoder instances in INIT state. For encoder instances in START_STREAM or START_FRAME state, all the bit rate control parameters except `ctbRc`, `hrd`, `frameRateNum`, and `frameRateDenom` are alterable without the need to restart the stream if HRD is disabled. However, if HRD is enabled, the stream needs to be restarted for altering most parameters. For details, see Section *Bit Rate Control Configuration*.

Parameters

in	<i>inst</i>	The encoder instance to be configured.
in	<i>pRateCtrl</i>	A pointer to the rate control parameters.

Returns

VCEncRet

9.4.2.2 VCEncGetRateCtrl()

```
VCEncRet VCEncGetRateCtrl (
    VCEncInst inst,
    VCEncRateCtrl * pRateCtrl )
```

Queries the bit rate control settings of an encoder instance.

Parameters

in	<i>inst</i>	The encoder instance to be queried.
out	<i>pRateCtrl</i>	A pointer to the storage space for receiving the bit rate control settings.

Returns

VCEncRet

9.5 Pre-Processing API

9.5.1 Functions

9.5.1.1 VCEncSetPreProcessing()

```
VCEncRet VCEncSetPreProcessing (
    VCEncInst inst,
    const VCEncPreProcessingCfg * pPreProcCfg )
```

Configures pre-processing blocks for an encoder instance.

All pre-processing parameters are configurable before stream generation, and only some are configurable during stream generation.

Parameters

in	<i>inst</i>	The encoder instance to be configured.
in	<i>pPreProcCfg</i>	A pointer to the pre-processing parameters.

Returns

VCEncRet

9.5.1.2 VCEncGetPreProcessing()

```
VCEncRet VCEncGetPreProcessing (
    VCEncInst inst,
    VCEncPreProcessingCfg * pPreProcCfg )
```

Queries the pre-processing settings of an encoder instance.

This function can be called before and during stream generation.

Parameters

in	<i>inst</i>	The encoder instance to be queried.
out	<i>pPreProcCfg</i>	A pointer to the storage space for receiving the pre-processing settings.

Returns

VCEncRet

9.6 PPS Configuration API

9.6.1 Functions

9.6.1.1 VCEncCreateNewPPS()

```
VCEncRet VCEncCreateNewPPS (
    VCEncInst inst,
    const VCEncPPSCfg * pPPSCfg,
    i32 * newPPSId )
```

Creates a picture parameter set (PPS).

Parameters

in	<i>inst</i>	The encoder instance for which the PPS is to be created.
in	<i>pPPSCfg</i>	A pointer to the PPS parameters.
out	<i>newPPSId</i>	A pointer to the storage space for receiving the ID of the created PPS.

Returns

VCEncRet

9.6.1.2 VCEncModifyOldPPS()

```
VCEncRet VCEncModifyOldPPS (  
    VCEncInst inst,  
    const VCEncPPSCfg * pPPSCfg,  
    i32 ppsId )
```

Modifies a picture parameter set (PPS).

Parameters

in	<i>inst</i>	The encoder instance whose PPS is to be modified.
in	<i>pPPSCfg</i>	A pointer to the PPS parameters.
in	<i>ppsId</i>	The ID of the PPS to be modified.

Returns

VCEncRet

9.6.1.3 VCEncGetPPSData()

```
VCEncRet VCEncGetPPSData (  
    VCEncInst inst,
```

```
VCEncPPSCfg * pPPSCfg,  
i32 ppsId )
```

Queries the settings of a picture parameter set (PPS).

Parameters

in	<i>inst</i>	The encoder instance whose PPS is to be queried.
out	<i>pPPSCfg</i>	A pointer to the storage space for receiving the PPS parameters.
in	<i>ppsId</i>	The ID of the PPS to be queried.

Returns

VCEncRet

9.6.1.4 VCEncActiveAnotherPPS()

```
VCEncRet VCEncActiveAnotherPPS (  
    VCEncInst inst,  
    i32 ppsId )
```

Activates a picture parameter set (PPS) to replace the current PPS.

Parameters

in	<i>inst</i>	The encoder instance whose PPS is to be activated.
in	<i>ppsId</i>	The ID of the PPS to be activated.

Returns

VCEncRet

9.6.1.5 VCEncGetActivePPSId()

```
VCEncRet VCEncGetActivePPSId (
```

```
VCEncInst inst,  
i32 * ppsId )
```

Queries the ID of the active picture parameter set (PPS).

Parameters

in	<i>inst</i>	The encoder instance to be queried.
out	<i>ppsId</i>	A pointer to the storage space for receiving the ID of the active PPS.

Returns

VCEncRet

9.7 Stream Production API

9.7.1 Functions

9.7.1.1 VCEncStrmStart()

```
VCEncRet VCEncStrmStart (  
    VCEncInst inst,  
    const VCEncIn * pEncIn,  
    VCEncOut * pEncOut )
```

Starts stream to generate SPS and PPS. SPS is carried in the first NAL unit and PPS in the second NAL unit.

Parameters

in	<i>inst</i>	The encoder instance for which the stream is to be started.
in	<i>pEncIn</i>	A pointer to the input parameters.
out	<i>pEncOut</i>	A pointer to the storage space for receiving the output parameters, in which NaluSizeBuf indicates the size of NAL units.

Returns

VCEncRet

9.7.1.2 VCEncStrmEncode()

```
VCEncRet VCEncStrmEncode (
    VCEncInst inst,
    VCEncIn * pEncIn,
    VCEncOut * pEncOut,
    VCEncSliceReadyCallBackFunc sliceReadyCbFunc,
    void * pAppData )
```

Encodes a video frame in display order. The frame may not be encoded immediately.

Parameters

in	<i>inst</i>	The encoder instance to encode the frame.
in	<i>pEncIn</i>	A pointer to the input parameters.
out	<i>pEncOut</i>	A pointer to the storage space for receiving the output parameters.
in	<i>sliceReadyCbFunc</i>	The callback function to be called when the hardware finishes encoding a slice.
in	<i>pAppData</i>	A pointer to the application data to be input to the callback function.

Returns

VCEncRet

9.7.1.3 VCEncStrmEncodeExt()

```
VCEncRet VCEncStrmEncodeExt (
    VCEncInst inst,
```

```

VCEncIn * pEncIn,

const VCEncExtParaIn * pEncExtParaIn,

VCEncOut * pEncOut,

VCEncSliceReadyCallBackFunc sliceReadyCbFunc,

void * pAppData,

i32 useExtFlag )

```

Encodes a video frame in decoder order.

Parameters

in	<i>inst</i>	The encoder instance to encode the frame.
in	<i>pEncIn</i>	A pointer to the input parameters.
in	<i>pEncExtParaIn</i>	A pointer to the input parameters extended for libva.
out	<i>pEncOut</i>	A pointer to the storage space for receiving the output parameters.
in	<i>sliceReadyCbFunc</i>	The callback function to be called when the hardware finishes encoding a slice.
in	<i>pAppData</i>	A pointer to the application data to be input to the callback function.
in	<i>useExtFlag</i>	Whether to enable the extended parameters. 1: enables 0: disables

Returns

VCEncRet

9.7.1.4 VCEncStrmEnd()

```

VCEncRet VCEncStrmEnd (
    VCEncInst inst,
    const VCEncIn * pEncIn,
    VCEncOut * pEncOut )

```

Ends a stream with an end-of-sequence (EOS) code.

Parameters

in	<i>inst</i>	The encoder instance for which the stream is to be ended.
in	<i>pEncIn</i>	A pointer to the input parameters.
out	<i>pEncOut</i>	A pointer to the storage space for receiving the output parameters.

Returns**VCEncRet****9.7.1.5 VCEncFlush()**

```

VCEncRet VCEncFlush (
    VCEncInst inst,
    VCEncIn * pEncIn,
    VCEncOut * pEncOut,
    VCEncSliceReadyCallbackFunc sliceReadyCbFunc,
    void * pAppData )

```

Flushes a frame out from the internal queue of an encoder instance and encodes the frame.

Call this function after all the frames to be encoded have been sent to the encoder.

Parameters

in	<i>inst</i>	The encoder instance from whose queue a frame is to be flushed out.
in	<i>pEncIn</i>	A pointer to the input parameters.
out	<i>pEncOut</i>	A pointer to the storage space for receiving the output parameters.
in	<i>sliceReadyCbFunc</i>	The callback function to be called when the hardware finishes encoding a slice.
in	<i>pAppData</i>	A pointer to the application data to be input to the callback function.

Returns

VCEncRet

9.8 Helper API

9.8.1 Enumeration Type

9.8.1.1 anonymous enum

anonymous enum

Enumerator

HDR_TYPE_HDR10_PLUS	HDR date type is HDR10 PLUS.
HDR_TYPE_DOLBY_VISION	HDR date type is Dolby vision.
HDR_TYPE_BOTH_HDR10_DOLBY	HDR date type is MIX of HDR10 Plus and Dolby vision.

9.8.1.2 RoiQpType_e

enum RoiQpType_e

Specifies quantization parameter (QP) value types.

Enumerator

QP_TYPE_DELTA	QP delta.
QP_TYPE_ABS	Absolute QP.

9.8.1.3 RoiCuMode_e

enum `RoiCuMode_e`

Specifies CU modes.

Enumerator

<code>CU_MODE_INTER</code>	The inter mode.
<code>CU_MODE_INTRA</code>	The intra mode.
<code>CU_MODE_SKIP</code>	The skip mode.
<code>CU_MODE_IPCM</code>	The IPCM mode.

9.8.1.4 RoiIntraPartType_e

enum `RoiIntraPartType_e`

Specifies partition modes for intra-mode CU.

Enumerator

<code>ROI_PART_2N_2N_I</code>	2Nx2N.
<code>ROI_PART_N_N_I</code>	NxN.

9.8.1.5 RoiInterPartType_e

enum `RoiInterPartType_e`

Specifies partition modes for inter-mode CU.

Enumerator

<code>ROI_INTERNAL_AUTOMODE</code>	Automatic mode.
------------------------------------	-----------------

Enumerator

ROI_PART_2N_2N	2Nx2N.
ROI_PART_2N_N	2NxN.
ROI_PART_N_2N	Nx2N.
ROI_PART_2N_NU	2NxNU.
ROI_PART_2N_ND	2NxND.
ROI_PART_NL_2N	NLx2N.
ROI_PART_NR_2N	NRx2N.

9.8.2 Functions

9.8.2.1 EncGetDec400TsBufferSize()

```
void EncGetDec400TsBufferSize (
    u32 type,
    u32 width,
    u32 height,
    u32 alignment,
    u32 * luma_Size,
    u32 * chroma_Size,
    u32 * picture_Size,
    u32 scanType,
    u32 dec400Enable,
    u32 dec400TSHeaderEnable )
```

Calculat the size of tile status buffer for save the side information of DEC400 compression engine.

Parameters

in	<i>type</i>	the picture format defined in EncPixelFormat.
in	<i>width</i>	picture width in pixels
in	<i>height</i>	picture height in pixels

Parameters

in	<i>alignment</i>	the alignment in byte for picture width which is required for compress one tile. when 0 is set, assume 256 bytes are used. currently, only 256 bytes alignment is verified now.
out	<i>luma_Size</i>	the pointer will be filled with bytes size for luma tile status buffer.
out	<i>chroma_Size</i>	the pointer will be filled with bytes size for chroma tile status buffer.
out	<i>picture_Size</i>	the pointer will be filled with bytes size for status buffer of both luma and chroma.

9.8.2.2 EncAsicGetCuInfoBufferSize()

```

i32 EncAsicGetCuInfoBufferSize (
    u32 width,
    u32 height,
    u32 ctu_size,
    u32 cuInfoVersion,
    u32 cuinfoAlignment,
    u32 aqInfoAlignment,
    u32 * cuInfoSize,
    u32 * cuinfoStride,
    u32 * cuInfoTableSize,
    u32 * aqInfoSize,
    u32 * aqInfoStride )

```

Calculate the buffer size for saving one frame of cu info.

Parameters

in	<i>width</i>	picture width of the encoded stream, not aligned.
in	<i>height</i>	picture height of the encoded stream, not aligned.
in	<i>ctu_size</i>	the mb or ctu size of one coding unit. it should be 16 for h264, and 64 for hevc/av1/vp9.
in	<i>cuInfoVersion</i>	indicate format of the cuinfo output
in	<i>cuinfoAlignment</i>	alignment bytes for save cuinfo in one ctb row
in	<i>aqInfoAlignment</i>	alignment bytes for save aqinfo in one ctb row

Parameters

out	<i>cuInfoSize</i>	buffer size for saving one frame cuinfo
out	<i>cuInfoStride</i>	buffer stride for one mb/ctb row cuinfo
out	<i>cuInfoTableSize</i>	buffer size for saving cuinfo table of one frame
out	<i>aqInfoSize</i>	buffer size for saving aqinfo of one frame
out	<i>aqInfoStride</i>	buffer stride for saving aqinfo of one frame

Returns

total cuinfo buffer size for one frame, include cuinfo, cuinfo table and aq info.

9.8.2.3 EncUfbcGetSize()

```
void EncUfbcGetSize (
    u32 inputFormat,
    u32 blockType,
    u32 width,
    u32 height,
    ufbcMode mode,
    u32 * asic_format,
    u32 * headerSize,
    u64 * frameSize )
```

Calculat the size of ufbc header and payload buffer.

Parameters

in	<i>inputFormat</i>	the picture format defined in EncPixelFormat.
in	<i>blockType</i>	ufbc superblock type(0 - 32x8 1 - 16x16)
in	<i>width</i>	picture width in pixels
in	<i>height</i>	picture height in pixels
out	<i>asic_format</i>	the pointer will be filled with input format mapping API values to SW/HW register values.
out	<i>headerSize</i>	the pointer will be filled with bytes size for ufbc header buffer.
out	<i>frameSize</i>	the pointer will be filled with bytes size for ufbc payload buffer.

9.8.2.4 GetConstantVal()

```
void GetConstantVal (
    u32 format,
    char * constantVal,
    u32 consColorVal[4] )
```

Get the constant color value.

Parameters

in	<i>format</i>	the picture format defined in EncPixelFormat.
in	<i>constantVal</i>	The constant color value for pvrhc core of UFBC.
out	<i>consColorVal</i>	the pointer will be filled with input format mapping API values to SW/HW register values.

9.8.2.5 EncParseDEC400HeaderData()

```
void EncParseDEC400HeaderData (
    u32 * clearColorVal,
    u32 * fcEnable,
    u8 * hdrData )
```

Parser DEC400 TS header data to get color & enable value of fast-clear.

Parameters

in	<i>hdrData</i>	the header data of Tile-Status.
out	<i>clearColorData</i>	the color value to fast clear.
out	<i>fcEnable</i>	if fast clear is enabled or not.

9.8.2.6 VCEncPutHDR10Info()

```
int VCEncPutHDR10Info (
    HDR10Info * hdr10_info,
    u32 buffer_size,
    u8 * buffer,
    u8 hdrType )
```

Fills a buffer with HDR10 information according to the SMPTE standard as T35 packets.

Parameters

in	<i>hdr10_info</i>	The HDR information to be coded into T35 packets.
in	<i>buffer_size</i>	The size of the buffer.
out	<i>buffer</i>	A pointer to the buffer filled with T35 data.

Returns

The number of bytes filled in the buffer

9.8.2.7 VCEncFillRoiMap()

```
VCEncRet VCEncFillRoiMap (
    const void * inst,
    const RoiRect * roi,
    u8 format,
    u8 * buffer,
    u32 blk_size )
```

Fills the ROI map buffer according to the provided ROI configurations.

Parameters

in	<i>inst</i>	The encoder instance.
in	<i>roi</i>	The ROI and its attributes.
in	<i>format</i>	The format used to fill the buffer. Only formats 1 to 4 are supported. For details, see <i>Hantro VC9000E Series Memory Buffer and Format Organization</i> .
in	<i>buffer</i>	The ROI map buffer to be filled.
in	<i>blk_size</i>	The size of the unit block for the ROI map.

Returns

VCEncRet

9.9 HEIF Writer API

Typedef

- typedef const void * **VCEncHeifWriterInst**
The HEIF writer handle.

9.9.1 Enumeration Type

9.9.1.1 VCEncHeifRet

enum **VCEncHeifRet**

Defines return values of HEIF writer APIs.

Enumerator

HEIF_OK	The API call is successful.
HEIF_ERROR	An error occurs in API execution.
HEIF_END	There is no more image in the file.
HEIF_MEMFAIL	There is not enough memory.

9.9.1.2 VCEncHeifDecInfoType

enum **VCEncHeifDecInfoType**

Defines media data types.

Enumerator

IsHevcImage	Heif image.
IsHevcSequence	Heif image sequence.
IsHevcDecHeader	Decoder configuration for hevc file content.
IsAv1Image	Avif image
IsAv1Sequence	Avif image sequence
IsAv1DecHeader	Decoder configuration for av1 file content.

9.9.1.3 VCEncHeifAuxiliaryType

```
enum VCEncHeifAuxiliaryType
```

Defines auxiliary image types.

Enumerator

alpha	Alpha plane.
depth	Depth map.

9.9.1.4 VCEncHeifMirrorType

```
enum VCEncHeifMirrorType
```

Defines mirroring types.

Enumerator

horizontal_mirror	Horizontal flip.
vertical_mirror	Vertical flip.

9.9.1.5 VCENCHeifGridModeType

enum `VCENCHeifGridModeType`

Defines grid modes.

Enumerator

<code>HEIF_GRID_DISABLE</code>	Disables grid modes.
<code>HEIF_GRID_INPUT_TILE</code>	Takes each input image as a grid for composing the output image.
<code>HEIF_GRID_INPUT_PIC</code>	Splits input image into multiple grids by indicated numbers of grids (rowsxcols) for encoding and storage.
<code>HEIF_GRID_INPUT_PIC_WIDTH</code>	Splits input image into multiple grids by indicated grid image size (widthxheight) for encoding and storage.

9.9.2 Functions

9.9.2.1 VCENCHeifParseConfig()

```

VCEncHeifRet VCENCHeifParseConfig (
    char * cfgfile,
    VCEncHeifInst * heif_inst,
    VCEncVideoCodecFormat codecType )

```

Parses HEIF configurations from a file, and fills `VCEncHeifInst.heif_config` accordingly.

Parameters

in	<code>cfgfile</code>	The path to the HEIF configuration file.
in	<code>heif_inst</code>	The HEIF instance.
in	<code>codecType</code>	The video codec format, only support hevc(heif) & av1(avif) now.

Returns

VCEncHeifRet

9.9.2.2 VCEncHeifInit()

```
VCEncHeifRet VCEncHeifInit (  
    VCEncHeifInst * heif_inst,  
    i32 width,  
    i32 height,  
    u32 input_numbers )
```

Creates and initializes an HEIF instance.

After this API is successfully called, an HEIF writer is also initialized and can be used through `VCEncHeifInst.writer_inst`.

Parameters

in	<i>heif_inst</i>	The HEIF instance.
in	<i>width</i>	The width of the heif output image.
in	<i>height</i>	The height of the heif output image.
in	<i>input_numbers</i>	Input image number, for grid_mode = HEIF_GRID_INPUT_TILE, should greater than or equal to grid_cols x grid_rows.

Returns

VCEncHeifRet

9.9.2.3 VCEncHeifSetPrpForGrid()

```
VCEncHeifRet VCEncHeifSetPrpForGrid (  
    VCEncInst encoder,  
    VCEncHeifInst * heif_inst )
```

Obtains a grid image for input to an encoder.

Parameters

in	<i>encoder</i>	The encoder instance.
in	<i>heif_inst</i>	The HEIF instance.

Returns

VCEncHeifRet

9.9.2.4 VCEncHeifSetHevcDecConfig()

```
void VCEncHeifSetHevcDecConfig (  
    VCEncHeifWriterInst writer_inst,  
    VCEncStrmBufs * bufs,  
    u32 offset,  
    u32 size )
```

Sets the hevc decoder configurations for an HEIF output file.

Parameters

in	<i>writer_inst</i>	The HEIF writer handle.
in	<i>bufs</i>	The stream buffer.
in	<i>offset</i>	The stream offset.
in	<i>size</i>	The size of stream data, in bytes.

Returns

None

9.9.2.5 VCEncHeifWriteStream()

```
VCEncHeifRet VCEncHeifWriteStream (  
    VCEncHeifInst * heif_inst,
```

```
VCEncStrmBufs * bufs,  
u32 offset,  
u32 streamSize )
```

Writes encoded stream to a file.

Parameters

in	<i>heif_inst</i>	The HEIF instance.
in	<i>bufs</i>	The stream buffer.
in	<i>offset</i>	The stream offset.
in	<i>streamSize</i>	The size of the stream data, in bytes.

Returns

VCEncHeifRet

9.9.2.6 VCEncHeifCloseWriter()

```
void VCEncHeifCloseWriter (  
    VCEncHeifInst * heif_inst )
```

Releases the writer of an HEIF instance.

Parameters

in	<i>heif_inst</i>	The HEIF instance.
----	------------------	--------------------

Returns

None

9.10 Log System API

Marco Definition

- #define **APITRACEERR**(fmt, ...)

The function that obtains error logs from the encoder API module.

- **#define APITRACEWRN(fmt, ...)**

The function that obtains warning logs from the encoder API module.

- **#define APITRACE(fmt, ...) VCEncTraceMsg(NULL, VCENC_LOG_INFO, VCENC_LOG_TRACE_API, fmt, ##__VA_ARGS__)**

The function that obtains information logs from the encoder API module.

This function is used for tracing API calls.

- **#define APITRACEPARAM_X(fmt, ...) VCEncTraceMsg(NULL, VCENC_LOG_INFO, VCENC_LOG_TRACE_API, fmt, ##__VA_ARGS__)**

The function that obtains information logs from the encoder API module.

This function is used for tracing API pointer-type parameters.

- **#define APITRACEPARAM(fmt, ...) VCEncTraceMsg(NULL, VCENC_LOG_INFO, VCENC_LOG_TRACE_API, fmt, ##__VA_ARGS__)**

The function that obtains information logs from the encoder API module.

This function is used for tracing API integer-type parameters.

- **#define APITRACE_ERR(inst, fmt, ...)**

The function that obtains error logs from the encoder API module, with the instance specified.

- **#define APITRACE_INFO(inst, fmt, ...) VCEncTraceMsg(inst, VCENC_LOG_INFO, VCENC_LOG_TRACE_API, fmt, ##__VA_ARGS__)**

The function that obtains information logs from the encoder API module, with the instance specified.

- **#define APITRACE_PARAM(inst, fmt, ...) VCEncTraceMsg(inst, VCENC_LOG_INFO, VCENC_LOG_TRACE_API, fmt, ##__VA_ARGS__)**

The function that obtains information logs from the encoder API module, with the instance specified.

This function is used for tracing API integer-type parameters.

- **#define PERFTRACE(inst, fmt, ...) VCEncTraceMsg(inst, VCENC_LOG_INFO, VCENC_LOG_TRACE_PERF, fmt, ##__VA_ARGS__)**

The function that obtains information logs for tracing performance.

- **#define PTRACE_E(fmt, ...) VCEncTraceMsg(NULL, VCENC_LOG_ERROR, VCENC_LOG_TRACE_EWL, fmt, ##__VA_ARGS__)**

The function that obtains error logs from the EWL module.

This function is used by EWL for hardware implementation.

- **#define PTRACE_I(fmt, ...) VCEncTraceMsg(NULL, VCENC_LOG_INFO, VCENC_LOG_TRACE_EWL, fmt, ##__VA_ARGS__)**

The function that obtains information logs from the EWL module.

This function is used by EWL for hardware implementation.

- **#define PTRACE(...) VCEncTraceMsg(NULL, VCENC_LOG_INFO, VCENC_LOG_TRACE_EWL, ##__VA_ARGS__)**

The function that obtains information logs from the EWL module.

- **#define EWLTRACE_I(inst, fmt, ...) VCEncTraceMsg(inst, VCENC_LOG_INFO, VCENC_LOG_TRACE_EWL, fmt, ##__VA_ARGS__)**

The function that obtains information logs from the EWL module.

This function is used by EWL for C-Model implementation.

- **#define EWLTRACE_E**(inst, fmt, ...) **VCEncTraceMsg**(inst, VCENC_LOG_ERROR, VCENC_LOG_TRACE_EWL, fmt, ##__VA_ARGS__)

The function that obtains error logs from the EWL module.

This function is used by EWL for C-Model implementation.

- **#define MEM_LOG_I**(fmt, ...) **VCEncTraceMsg**(NULL, VCENC_LOG_INFO, VCENC_LOG_TRACE_MEM, fmt, ##__VA_ARGS__)

The function that obtains information logs for tracing memory usage.

- **#define CMLTRACE**(...)

The function that obtains information logs for tracing command lines used to set up the test bench.

- **#define RC_LOG_I**(fmt, ...) **VCEncTraceMsg**(NULL, VCENC_LOG_INFO, VCENC_LOG_TRACE_RC, fmt, ##__VA_ARGS__)

The function that obtains information logs for tracing the rate control process.

- **#define RC_LOG_E**(fmt, ...)

The function that obtains error logs for tracing the rate control process.

- **#define RC_LOG_D**(fmt, ...) **VCEncTraceMsg**(NULL, VCENC_LOG_DEBUG, VCENC_LOG_TRACE_RC, fmt, ##__VA_ARGS__)

The function that obtains debugging logs for tracing the rate control process.

9.10.1 Marco Definition

9.10.1.1 APITRACEERR

```
#define APITRACEERR(  
    fmt,  
    ... )
```

Value:

```
VCEncTraceMsg(NULL, VCENC_LOG_ERROR, VCENC_LOG_TRACE_API, "[%s:%d]" fmt, \  
    __FUNCTION__, __LINE__, ##__VA_ARGS__)
```

9.10.1.2 APITRACEWRN

```
#define APITRACEWRN(  
    fmt,  
    ... )
```

Value:

```
VCEncTraceMsg(NULL, VCENC_LOG_WARN, VCENC_LOG_TRACE_API, \  
    "[%s:%d]Warning: " fmt, __FUNCTION__, __LINE__, ##__VA_ARGS__)
```

9.10.1.3 APITRACE_ERR

```
#define APITRACE_ERR(  
    inst,  
    fmt,  
    ... )
```

Value:

```
VCEncTraceMsg(inst, VCENC_LOG_ERROR, VCENC_LOG_TRACE_API, "[%s:%d]" fmt, \  
    __FUNCTION__, __LINE__, ##__VA_ARGS__)
```

9.10.1.4 CMLTRACE

```
#define CMLTRACE(  
    ... )
```

Value:

```
do {  
    VCEncTraceMsg(NULL, VCENC_LOG_INFO, VCENC_LOG_TRACE_CML, ##__VA_ARGS__); \  
} while (0)
```

9.10.1.5 RC_LOG_E

```
#define RC_LOG_E(  
    fmt,  
    ... )
```

Value:

```
VCEncTraceMsg(NULL, VCENC_LOG_ERROR, VCENC_LOG_TRACE_RC, "[%s:%d]" fmt, \  
    __FUNCTION__, __LINE__, ##__VA_ARGS__)
```

9.10.2 Enumeration Type

9.10.2.1 VCEncLogEnvControl

```
enum VCEncLogEnvControl
```

Defines log control information types obtained from the environment.

Enumerator

VCENC_LOG_OUTPUT	Information for the environment to control output files.
VCENC_LOG_LEVEL	Information for the environment to control the output level.
VCENC_LOG_TRACE	Information for the environment to control trace information for prompt and debugging.
VCENC_LOG_CHECK	Information for the environment to control test-only information check.

9.10.2.2 VCEncLogOutput

```
enum VCEncLogOutput
```

Defines log output modes.

Enumerator

LOG_STDOUT	Outputs all logs to stdout.
LOG_ONE_FILE	Outputs all logs to the same file. The log file is <code>vcenc_trace_p\${pid}.log</code> where <code>\${pid}</code> is the process ID.
LOG_BY_THREAD	Outputs logs of each thread to a separate log file. The log files are named <code>vcenc_trace_p\${pid}t\${tid}.log</code> where <code>\${pid}</code> is the process ID and <code>\${tid}</code> is the thread ID.
LOG_STDERR	Outputs all logs to stderr.

9.10.2.3 VCEncLogFileType

enum `VCEncLogFileType`

Defines log types.

Enumerator

<code>STREAM_TRACE_FILE</code>	Trace log, used to debug or analyze the status of the encoder behavior.
<code>STREAM_CHECK_FILE</code>	(Reserved) Check log, which is used in tests to check whether a feature is enabled and properly configured.

9.10.2.4 VCEncLogTraceType

enum `VCEncLogTraceType`

Defines trace log types.

Enumerator

<code>VCENC_LOG_TRACE_API</code>	Encoder API call log.
<code>VCENC_LOG_TRACE_REGS</code>	Register configuration log.
<code>VCENC_LOG_TRACE_EWL</code>	EWL API call log.
<code>VCENC_LOG_TRACE_MEM</code>	Memory usage log.
<code>VCENC_LOG_TRACE_RC</code>	Rate control status log.
<code>VCENC_LOG_TRACE_CML</code>	Command line log.
<code>VCENC_LOG_TRACE_PERF</code>	Performance log.

9.10.2.5 VCEncLogCheckType

enum `VCEncLogCheckType`

(Reserved) Defines check log types.

Enumerator

<code>VCENC_LOG_CHECK_RECON</code>	Reconstructed YUV data.
<code>VCENC_LOG_CHECK_QUALITY</code>	PSNR/SSIM for each frame.
<code>VCENC_LOG_CHECK_VBV</code>	VBV information for rate control checking.
<code>VCENC_LOG_CHECK_RC</code>	Rate control information for rate control profiling.
<code>VCENC_LOG_CHECK_FEATURE</code>	Feature information for coverage checking.

9.10.2.6 VCEncLogLevel

enum `VCEncLogLevel`

Defines output log levels.

Enumerator

<code>VCENC_LOG_QUIET</code>	No logs output.
<code>VCENC_LOG_FATAL</code>	The fatal level.
<code>VCENC_LOG_ERROR</code>	The error level.
<code>VCENC_LOG_WARN</code>	The warning level.
<code>VCENC_LOG_INFO</code>	The information level.
<code>VCENC_LOG_DEBUG</code>	The debugging level.
<code>VCENC_LOG_ALL</code>	All logs output.

9.10.3 Functions

9.10.3.1 VCEncLogInit()

```
int VCEncLogInit (
    unsigned int out_dir,
    unsigned int out_level,
    unsigned int trace_map,
    unsigned int check_map )
```

Initializes the log system.

Parameter settings are prioritized in ascending order as follows: default values < environment settings < command line settings.

Parameters

in	<i>out_dir</i>	The log output mode.
in	<i>out_level</i>	The lowest log level to be output. Logs of the specified level and higher are output.
in	<i>trace_map</i>	A bitmap where each bit controls whether to output each type of trace logs. 0: do not output. 1: output. For the mappings between bit indexes and trace log types, see Section <i>VCEncLogTraceType</i> .
in	<i>check_map</i>	(Reserved) A bitmap where each bit controls whether to output each type of check logs. 0: do not output. 1: output. For the mappings between bit indexes and check log types, see Section <i>VCEncLogCheckType</i> .

9.10.3.2 VCEncTraceMsg()

```
void VCEncTraceMsg (
    const void * inst,
    VCEncLogLevel level,
```

```
unsigned int log_trace_mask,  
const char * fmt,  
... )
```

Prints trace logs of a given type and levels.

Parameters

in	<i>inst</i>	The instance for which trace logs are to be printed.
in	<i>level</i>	The lowest log level to be output. Logs of the specified level and higher are output.
in	<i>log_trace_mask</i>	A bitmap where each bit controls whether to print each type of trace logs. 0: do not print. 1: print. For the mappings between bit indexes and trace log types, see Section VCEncLogTraceType .
in	<i>fmt</i>	A string in the printf format.

9.10.3.3 VCEncCheckMsg()

```
void VCEncCheckMsg (  
    const void * inst,  
    VCEncLogLevel level,  
    unsigned int log_check_mask,  
    const char * fmt,  
    ... )
```

(Reserved) Prints check logs of a given type and levels.

9.10.3.4 VCEncLogDestory()

```
int VCEncLogDestory (  
    void )
```

Releases resources allocated to the log system.

9.10.3.5 EncTraceRegs()

```
void EncTraceRegs (
    const void * ewl,
    unsigned int readWriteFlag,
    unsigned int mbNum,
    unsigned int * regs )
```

(Reserved) Outputs register information.

9.10.3.6 VCEncLogGetEnvSetting()

```
void VCEncLogGetEnvSetting (
    VCEncLogSetting * env_log )
```

Queries the parameters for controlling the log system.

Parameters

in	<i>env_log</i>	The log settings read from the environment or specified during initialization.
----	----------------	--

10 Data Structure Documentation

10.1 BezierDict

Contains the information in Bezier curve packets defined by HDR10+.

Data Fields

- **u32 tone_mapping_flag [3]**
tone_mapping_flag[w] indicates whether the metadata of the tone mapping function in the w-th processing window is present.
- **u32 knee_point_x [3]**
knee_point_x[w] indicates the X coordinate of the separation point between the linear part and the curved part of the tone mapping function in the w-th processing window.
- **u32 knee_point_y [3]**
knee_point_y[w] indicates the Y coordinate of the separation point between the linear part and the curved part of the tone mapping function in the w-th processing window.
- **u32 num_bezier_curve_anchors [3]**
num_bezier_curve_anchors[w] indicates the number of the intermediate anchor parameters of the tone mapping function in the w-th processing window.
- **u32 bezier_curve_anchors [3 * 15]**
*bezier_curve_anchors[w * 15 + i]* indicates the i-th intermediate anchor parameter of the tone mapping function in the w-th processing window in the scene.
- **u32 color_saturation_mapping_flag [3]**
color_saturation_mapping_flag[w] indicates the color saturation mapping flag for the w-th processing window.
0: compliant with SMPTE ST 2094-40.
1: reserved for future use.
- **u32 color_saturation_weight [3]**
color_saturation_weight[w] indicates the color saturation gain in the w-th processing window in the scene.

10.2 DistDict

Contains the information in **DistDict** packets defined by HDR10+.

Data Fields

- **u32 maxscl** [3 *3]

The maximum linearized RGB values in each processing window in the scene.

*maxscl[w*3+i] indicates the maximum value for the i-th color component of R, G, and B in the w-th processing window.*

- **u32 average_maxrgb** [3]

average_maxrgb[w] indicates the average of maximum linearized RGB values in the w-th processing window in the scene.

- **u32 num_distribution_maxrgb_percentiles** [3]

num_distribution_maxrgb_percentiles[w] indicates the number of maximum linearized RGB values at given percentiles in the w-th processing window in the scene.

- **u32 distribution_maxrgb_percentages** [3 *15]

*distribution_maxrgb_percentages[w*15+i] indicates the integer percentage of maximum linearized RGB values at the i-th percentile in the w-th processing window in the scene.*

- **u32 distribution_maxrgb_percentiles** [3 *15]

*distribution_maxrgb_percentiles[w*15+i] indicates the number of maximum linearized RGB values at the i-th percentile in the w-th processing window in the scene.*

- **u32 fraction_bright_pixels** [3]

fraction_bright_pixels[w] indicates the fraction of selected pixels in the picture that contains the brightest pixel in the w-th processing window in the scene.

10.3 dolby_vision_attr

HDR10Info member: Dolby vision data packet

See **HDR10Info**.

Data Fields

- **u32 itu_t_t35_country_code**

The country code defined in ITU-T Rec. T.35.

- **u32 itu_t_t35_terminal_provider_code**

The terminal provider code defined in ITU-T Rec. T.35.

- **u32 itu_t_t35_terminal_provider_oriented_code**
The terminal provider oriented code defined in ITU-T Rec. T.35.
- **u32 application_identifier**
The application identifier.
- **u32 application_version**
The application version.
- **u32 data_type_code**
The data type code defined by Dolby Vision.
- **u32 metadata_refresh_flag**
The metadata refresh flag defined by Dolby Vision.
This field is used to identify whether the data to be parsed is compatible with HDR10+ or Dolby Vision.
0: HDR10+.
1: Dolby Vision.
- **MetaData metadata**
The metadata packets of Dolby Vision.

10.4 ExtDmDataBlock

Contains the information of an extended display mapping metadata block in **ExtDmDataBlock** packets defined by Dolby Vision.

Data Fields

- **u32 ext_block_length**
A length used to derive the size of the extended display mapping metadata block payload in bytes.
The value range is from 0 to 1023, inclusive.
- **u32 ext_block_level**
The level of payload contained in the extended display mapping metadata block.
The value range is from 0 to 255, inclusive.
- **u32 min_PQ**
The minimum luminance value of the current picture in 12-bit PQ encoding.
The value range is from 0 to 4095, inclusive.
- **u32 max_PQ**
The maximum luminance value of the current picture in 12-bit PQ encoding.
The value range is from 0 to 4095, inclusive.
- **u32 avg_PQ**
The midpoint luminance value of the current picture in 12-bit PQ encoding.
The value range is from 0 to 4095, inclusive.

- **u32 target_max_PQ**
*The maximum luminance value of a target display in 12-bit PQ encoding.
The value range is from 0 to 4095, inclusive.*
- **u32 trim_slope**
*The slope metadata.
The value range is from 0 to 4095, inclusive.
If trim_slope is not present, the value is 2048.*
- **u32 trim_offset**
*The offset metadata.
The value range is from 0 to 4095, inclusive.
If trim_offset is not present, the value is 2048.*
- **u32 trim_power**
*The power metadata.
The value range is from 0 to 4095, inclusive.
If trim_power is not present, the value is 2048.*
- **u32 trim_chroma_weight**
*The chroma weight metadata.
The value range is from 0 to 4095, inclusive.
If trim_chroma_weight is not present, the value is 2048.*
- **u32 trim_saturation_gain**
*The saturation gain metadata.
The value range is from 0 to 4095, inclusive.
If trim_saturation_gain is not present, the value is 2048.*
- **u32 ms_weight**
Reserved.
- **u32 active_area_left_offset**
*The X offset between the left border of the active area and the leftmost selected pixel of the current picture.
The value range is from 0 to 8191, inclusive. For details, see ProcessingWindow definitions in SMPTE ST 2094-10.*
- **u32 active_area_right_offset**
*The X offset between the right border of the active area and the rightmost selected pixel of the current picture.
The value range is from 0 to 8191, inclusive. For details, see ProcessingWindow definitions in SMPTE ST 2094-10.*
- **u32 active_area_top_offset**
*The Y offset between the top border of the active area and the topmost selected pixel of the current picture.
The value range is from 0 to 8191, inclusive. For details, see ProcessingWindow definitions in SMPTE ST 2094-10.*
- **u32 active_area_bottom_offset**
*The Y offset between the bottom border of the active area and the bottommost selected pixel of the current picture.
The value range is from 0 to 8191, inclusive. For details, see ProcessingWindow definitions in SMPTE ST 2094-10.*

10.5 ExternalSEI

Defines the external SEI information.

Data Fields

- **u8 nalType**

The type of the external SEI NAL unit.

For HEVC (H.265), the field value can be 39 or 40 where 39 indicates PREFIX_SEI_NUT and 40 indicates SUFFIX_SEI_NUT.

For H.264 (AVC), the field can be left unspecified and the value is fixed to 6.

For details about the field values, see ITU-T Rec. H.264.

- **u8 payloadType**

The payload type of the external SEI.

For example, the field value is 1 for picture timing.

For details, see Annex D.2.1 in ITU-T Rec. H.265 for HEVC (H.265) or Annex D.1.1 in ITU-T Rec. H.264 for H.264 (AVC).

- **u32 payloadDataSize**

The payload data length of the external SEI.

- **u8 * pPayloadData**

The payload data of the external SEI.

- **u32 frame_id**

The frame ID of the external SEI.

10.6 h264FramInfo

Contains the information of a frame that the video encoder outputs for H.264 (AVC).

Data Fields

- **u32 picBytes**

The number of bytes encoded.

- **u32 pSkipMbNum**

The number of skip MBs.

- **u32 lpcmMbNum**

The number of IPCM MBs.

- **u32 inter16x16MbNum**

The number of inter 16x16 MBs.

- **u32 inter16x8MbNum**

The number of inter 16x8 MBs.

- **u32 inter8x16MbNum**

The number of inter 8x16 MBs.

- **u32 inter8x8MbNum**
The number of inter 8x8 MBs.
- **u32 intra16MbNum**
The number of intra 16x16 MBs.
- **u32 intra8MbNum**
The number of intra 8x8 MBs.
- **u32 intra4MbNum**
The number of intra 4x4 MBs.
- **u32 residualBits**
The number of residual bits.
- **u32 headBits**
The number of CU mode bits.
- **u32 madiVal**
The average MAD for all blocks of the input picture, which can be used to measure the space complexity of the input picture.
- **u32 madpVal**
The average MAD for all blocks of predicted residual, which can be used to measure the time complexity of the input picture.
- **u32 LCU_cnt**
The number of LCUs (that is, CTBs) in the frame.
- **u32 psnrVal [3]**
The fixed-point PSNR in 16.16 precision.
psnrVal[0]: the fixed-point PSNR for the Y component.
psnrVal[1]: the fixed-point PSNR for the U component.
psnrVal[2]: the fixed-point PSNR for the V component.
- **u64 sseSum [3]**
The frame SSE.
sseSum[0]: the frame SSE for the Y component.
sseSum[1]: the frame SSE for the U component.
sseSum[2]: the frame SSE for the V component.
- **u64 sse_info [8][3]**
The SSE for a maximum of eight regions.
sse_info[n][0]: the SSE for the Y component in region n.
sse_info[n][1]: the SSE for the U component in region n.
sse_info[n][2]: the SSE for the V component in region n.
- **u32 qp_hist [52]**
The QP histogram.
- **u32 picPoc**
The POC of the frame.
- **u32 picSliceNum**
The number of slices in the frame.

- **u32 picNumIntra**
The number of intra blocks in each 8x8 unit.
- **u8 startQp**
The initial QP of the frame, which refers to the `sliceQp` in the slice header.
- **u8 isPSkip**
*Whether the CUs in the frame are skip CUs.
0: the CUs in the frame are not skip CUs.
1: the CUs in the frame are skip CUs.*
- **u8 picType**
*The type of the frame.
For the available values, see Section [VCEncPictureCodingType](#).*
- **u8 meanQp**
The mean QP of the frame.
- **u32 gopPicIdx**
*The index of the picture in the current GOP.
The pictures are indexed starting from 0 at each IDR frame.*
- **u32 picNum**
The number of the picture.
- **u64 pixLumaSum**
The Sum luma of the picture.
- **u16 pixLumaMax**
The Max luma of the picture.
- **u16 pixLumaMin**
The min luma of the picture.

10.7 hdr10_plus_attr

HDR10Info member: HDR10 plus data packet

See **HDR10Info**.

Data Fields

- **u32 itu_t_t35_country_code**
The country code defined in ITU-T Rec. T.35.
- **u32 itu_t_t35_terminal_provider_code**
The terminal provider code defined in ITU-T Rec. T.35.
- **u32 itu_t_t35_terminal_provider_oriented_code**

The terminal provider oriented code defined in ITU-T Rec. T.35.

- **u32 application_identifier**

The application identifier.

- **u32 application_version**

The application version.

- **u32 num_windows**

The number of windows.

A maximum of three windows are supported.

- **ProcWindows processing_windows [3]**

The information in processing window packets of HDR10+, with `processing_windows[w]` for the *w*-th processing window.

- **u32 targeted_system_display_maximum_luminance**

The maximum luminance for display, in the unit of 0.0001 candelas per square meter.

- **u32 targeted_system_display_actual_peak_luminance_flag**

The actual peak luminance flag of the targeted system display.

0: compliant with SMPTE ST 2094-40.

1: reserved for future use.

- **PeakLum targeted_system_display_actual_peak_luminance**

The information in actual peak luminance packets of HDR10+.

- **DistDict distribution**

The information in distribution packets of HDR10+.

- **MasteringDict mastering**

The information in mastering packets of HDR10+.

- **BezierDict bezier_curve**

The information in bezier curve packets of HDR10+.

10.8 Hdr10DisplaySei

Contains the color volume information of the mastering display for HDR10.

Data Fields

- **u8 hdr10_display_enable**

Whether to insert mastering display color volume information.

0: do not insert.

1: insert.

- **u16 hdr10_dx0**

The normalized X chromaticity coordinate of component 0.

The value range is from 0 to 50000, inclusive.

This field is equivalent to `display_primaries_x[0]` in the mastering display color volume information.

- **u16_hdr10_dy0**

The normalized Y chromaticity coordinate of component 0.

The value range is from 0 to 50000, inclusive.

This field is equivalent to `display_primaries_y[0]` in the mastering display color volume information.

- **u16_hdr10_dx1**

The normalized X chromaticity coordinate of component 1.

The value range is from 0 to 50000, inclusive.

This field is equivalent to `display_primaries_x[1]` in the mastering display color volume information.

- **u16_hdr10_dy1**

The normalized Y chromaticity coordinate of component 1.

The value range is from 0 to 50000, inclusive.

This field is equivalent to `display_primaries_y[1]` in the mastering display color volume information.

- **u16_hdr10_dx2**

The normalized X chromaticity coordinate of component 2.

The value range is from 0 to 50000, inclusive.

This field is equivalent to `display_primaries_x[2]` in the mastering display color volume information.

- **u16_hdr10_dy2**

The normalized Y chromaticity coordinate of component 2.

The value range is from 0 to 50000, inclusive.

This field is equivalent to `display_primaries_y[2]` in the mastering display color volume information.

- **u16_hdr10_wx**

The normalized X chromaticity coordinate of the white point.

The value range is from 0 to 50000, inclusive.

This field is equivalent to `white_point_x` in the mastering display color volume information.

- **u16_hdr10_wy**

The normalized Y chromaticity coordinate of the white point.

The value range is from 0 to 50000, inclusive.

This field is equivalent to `white_point_y` in the mastering display color volume information.

- **u32_hdr10_maxluma**

The nominal maximum display luminance.

This field is equivalent to `max_display_mastering_luminance` in the mastering display color volume information.

- **u32_hdr10_minluma**

The nominal minimum display luminance.

This field is equivalent to `min_display_mastering_luminance` in the mastering display color volume information.

10.9 HDR10Info

Same data struct for save HDR10+ and Dolby's info.

Data Fields

- **u32 frame_num**
current frame number
- **u32 hdr_type**
HDR data type[1,2:1], 1: HDR10 plus; 2: dolby vision; 3: both in same frame.
- **hdr10_plus_attr** **hdr10_attr**
- **dolby_vision_attr** **dolby_attr**

10.10 Hdr10LightLevelSei

Contains the content light level information for HDR10.

Data Fields

- **u8 hdr10_lightlevel_enable**
*Whether to insert content light level information.
0: do not insert.
1: insert.*
- **u16 hdr10_maxlight**
The max_content_light_level field in the content light level information.
- **u16 hdr10_avglight**
The max_pic_average_light_level field in the content light level information.

10.11 HeifOffset

Defines a pair of coordinate offsets.

Data Fields

- `uint32_t horizontal`
The horizontal offset.
- `uint32_t vertical`
The vertical offset.

10.12 hevcFramInfo

Contains the information of a frame that the video encoder outputs for HEVC (H.265).

Data Fields

- `u32 picBytes`
The number of bytes encoded.
- `u32 inter64x64CuNum`
The number of inter 64x64 CUs.
- `u32 inter32x32CuNum`
The number of inter 32x32 CUs.
- `u32 inter16x16CuNum`
The number of inter 16x16 CUs.
- `u32 inter8x8CuNum`
The number of inter 8x8 CUs.
- `u32 intra32x32CuNum`
The number of intra 32x32 CUs.
- `u32 intra16x16CuNum`
The number of intra 16x16 CUs.
- `u32 intra8x8CuNum`
The number of intra 8x8 CUs.
- `u32 intra4x4CuNum`
The number of intra 4x4 CUs.
- `u32 residualBits`
The number of TU residual bits.
- `u32 headBits`
The number of CU mode bits and MV bits.
- `u32 madiVal`

The average MAD for all blocks of the input picture, which can be used to measure the space complexity of the input picture.

- **u32 madpVal**

The average MAD for all blocks of predicted residual, which can be used to measure the time complexity of the input picture.

- **u32 LCU_cnt**

The number of LCUs (that is, CTBs) in the frame.

- **u64 sseSum [3]**

The frame SSE.

sseSum[0]: the frame SSE for the Y component.

sseSum[1]: the frame SSE for the U component.

sseSum[2]: the frame SSE for the V component.

- **u64 sse_info [8][3]**

The SSE for a maximum of eight regions.

sse_info[n][0]: the SSE for the Y component in region *n*.

sse_info[n][1]: the SSE for the U component in region *n*.

sse_info[n][2]: the SSE for the V component in region *n*.

- **u32 psnrVal [3]**

The fixed-point PSNR in 16.16 format.

psnrVal[0]: the fixed-point PSNR for the Y component.

psnrVal[1]: the fixed-point PSNR for the U component.

psnrVal[2]: the fixed-point PSNR for the V component.

- **u32 qp_hist [52]**

The QP histogram.

- **u32 picPoc**

The POC of the frame.

- **u32 picSliceNum**

The number of slices in the frame.

- **u32 picNumIntra**

The number of intra blocks in each 8x8 unit.

- **u32 picNumMerge**

The number of inter merge blocks in each 8x8 unit.

- **u32 pSkipNum**

The number of skip blocks in each 8x8 unit.

- **u32 ipcmNum**

The number of ipcm blocks in each 8x8 unit.

- **u8 startQp**

The initial QP of the frame, which refers to the *sliceQp* in the slice header.

- **u8 isPSkip**

Whether the CUs in the frame are skip CUs.

0: the CUs in the frame are not skip CUs.

1: the CUs in the frame are skip CUs.

- **u8 picType**

The type of the frame.

For the available values, see Section [VCEncPictureCodingType](#).

- **u8 meanQp**

The mean QP of the frame.

- **u32 gopPicIdx**

The index of the picture in the current GOP.

The pictures are indexed starting from 0 at each IDR frame.

- **u32 picNum**

The number of the picture.

- **u64 pixLumaSum**

The Sum luma of the picture.

- **u16 pixLumaMax**

The Max luma of the picture.

- **u16 pixLumaMin**

The min luma of the picture.

10.13 ITU_T_T35

Contains ITU-T T.35 information configurations.

Data Fields

- **u8 t35_enable**

Whether to insert dynamic HDR information as metadata.

0: do not insert.

1: insert.

- **u8 t35_country_code**

The country code defined in ITU-T Rec. T.35.

- **u8 t35_country_code_extension_byte**

The country code extension defined in ITU-T Rec. T.35.

- **const u8 * p_t35_payload_bytes**

A pointer to the buffer that stores T.35 payload data.

- **u32 t35_payload_size**

The size of valid T.35 payload data in bytes.

10.14 MasteringDict

Contains the information in Mastering packets defined by HDR10+.

Data Fields

- **u32 mastering_display_actual_peak_luminance_flag**
*The actual peak luminance flag of the mastering display.
0: compliant with SMPTE ST 2094-40.
1: reserved for future use.*
- **u32 num_rows_mastering_display_actual_peak_luminance**
The number of rows in the `MasteringDict.mastering_display_actual_peak_luminance` array.
- **u32 num_cols_mastering_display_actual_peak_luminance**
The number of columns in the `MasteringDict.mastering_display_actual_peak_luminance` array.
- **u32 mastering_display_actual_peak_luminance [31 * 31]**
The normalized actual peak luminance of the mastering display.

10.15 MetaData

Contains the information in **MetaData** packets defined by Dolby Vision.

Data Fields

- **u32 num_ext_blocks**
The number of extended display mapping metadata blocks.
- **ExtDmDataBlock ext_dm_data_block [100]**
The information of each extended display mapping metadata block.

10.16 NewEncodeParams

Contains the updated parameters for re-encoding.

Data Fields

- **ReEncodeStrategy strategy**
The re-encoding strategy.
- **EWLLinearMem_t output_buffer_mem [MAX_STRM_BUF_NUM]**
The information about new output buffers.
This field is valid only if the value of `NewEncodeParams.strategy` is `NEW_OUTPUT_BUFFER`.
- **u32 qp**
The new slice QP value.
This field is valid only if the value of `NewEncodeParams.strategy` is `NEW_QP`.
- **u32 try_counter**
The number of times the callback function has tried to re-encode the current picture.
- **u32 targetPicSize**
The new target picture size used by the block-level rate control algorithm.
This field is valid only if the value of `NewEncodeParams.strategy` is `NEW_TARGET_BIT`.
- **u32 minPicSize**
The minimum target picture size used by the block-level rate control algorithm.
This field is valid only if the value of `NewEncodeParams.strategy` is `NEW_TARGET_BIT`.
- **u32 maxPicSize**
The maximum target picture size used by the block-level rate control algorithm.
This field is valid only if the value of `NewEncodeParams.strategy` is `NEW_TARGET_BIT`.

10.17 PeakLum

Contains the information in **PeakLum** packets defined by HDR10+.

Data Fields

- **u32 num_rows_targeted_system_display_actual_peak_luminance**
The number of rows in the `PeakLum.targeted_system_display_actual_peak_luminance` array.
- **u32 num_cols_targeted_system_display_actual_peak_luminance**
The number of columns in the `PeakLum.targeted_system_display_actual_peak_luminance` array.
- **u32 targeted_system_display_actual_peak_luminance [31 * 31]**
The normalized actual peak luminance of targeted system display.

10.18 ProcWindows

Contains the information of a processing window in **ProcWindows** packets defined by HDR10+.

Data Fields

- **u32 window_upper_left_corner_x**
The X coordinate of the top-left pixel of the processing window.
- **u32 window_upper_left_corner_y**
The Y coordinate of the top-left pixel of the processing window.
- **u32 window_lower_left_corner_x**
The X coordinate of the bottom-left pixel of the processing window.
- **u32 window_lower_left_corner_y**
The Y coordinate of the bottom-left pixel of the processing window.
- **u32 center_of_elipse_x**
The X coordinate of the center position of concentric internal and external ellipses of the elliptical pixel selector in the processing window.
- **u32 center_of_elipse_y**
The Y coordinate of the center position of concentric internal and external ellipses of the elliptical pixel selector in the processing window.
- **u32 rotation_angle**
The clockwise rotation angle in degrees, relative to the positive direction of the X-axis of the concentric internal and external ellipses of the elliptical pixel selector in the processing window.
- **u32 semimajor_axis_internal_elipse**
The semi-major axis value, in pixels, of the internal ellipse of the elliptical pixel selector in the processing window.
- **u32 semimajor_axis_external_elipse**
The semi-major axis value, in pixels, of the external ellipse of the elliptical pixel selector in the processing window.
- **u32 semiminor_axis_external_elipse**
The semi-minor axis value, in pixels, of the external ellipse of the elliptical pixel selector in the processing window.
- **u32 overlap_process_option**
The method to combine rendered pixels in the processing window with at least one elliptical pixel selector.
0: weighted averaging.
1: layering.

10.19 RoiRect

Defines a ROI and contains its attributes.

Data Fields

- **u16 x0**
The X coordinate of the top-left corner of the ROI.
- **u16 y0**
The Y coordinate of the top-left corner of the ROI.
- **u16 dx**
The width of the ROI.
- **u16 dy**
The height of the ROI.
- **u8 qp**
The QP delta or the absolute QP value, according to the setting of `RoiRect.qp_type`.
- **u8 qp_type**
The value type of `RoiRect.qp`.
For the available value types, see Section [RoiQpType_e](#).
- **u8 qp_valid**
Whether the setting of `RoiRect.qp` is valid.
0: invalid.
1: valid.
- **u8 cu_size**
The CU size used to encode in the specified ROI.
- **u8 cu_size_valid**
Whether the setting of `RoiRect.cu_size` is valid.
0: invalid.
1: valid.
- **u8 mode**
The CU mode used to encode the specified ROI.
For the available CU modes, see Section [RoiCuMode_e](#).
- **i8 part**
The partition mode used to encode the specified ROI.
The available partition modes vary according to the CU mode. For details, see Section [RoiIntraPartType_e](#) and Section [RoiInterPartType_e](#).
- **u8 mode_valid**
Whether the settings of `RoiRect.mode` and `RoiRect.part` are valid.
0: invalid.
1: valid.
- **u8 intra_part_valid**
Obsoleted.
-

```

union {
    struct {
        i32 dir
            The direction.
        u8 ref_idx [2]
            The index of the reference used in each of the two reference lists.
        u8 mvx [2]
            The horizontal motion vector (MV) for the two references.
        u8 mvy [2]
            The vertical MV for the two references.
    } inter [2]
        The inter PU information for two PUs.
    struct {
        i32 luma_mode [4]
            The luma modes, only luma_mode[0] valid for ROI_PART_2N_2N_I and all entries valid for ROI_
        i32 chroma_mode
            The chroma mode. For the available values, see ITU-T Rec. H.265 and H.264.
    } intra
        The intra PU information.
    } info
        (Under development) The prediction unit (PU) information.

```

10.20 ufbc_afbc_param

Contains parameters when the core of UFBC is AFBC.

Data Fields

- **u32 yuvTrans**
Whether to enable RGB-to-YUV conversion in UFBC.
0: disable.
1: enable.
- **u32 blockType**
The superblock type of UFBC.
0: 32x8.
1: 16x16.

- **u32 blockSplit**

Whether to enable block split mode in UFBC.

0: Block split mode off.

1: Block split mode on.

10.21 ufbc_param

Contains the UFBC parameters.

Data Fields

- **ufbcMode mode**

The core mode of UFBC.

UFBC_MODE_NONE: UFBC disable.

UFBC_MODE_AFBC_0: Use the core of AFBC version 0.(only support 32x8 superblock).

UFBC_MODE_AFBC_1: Use the core of AFBC version 1.(Support 32x8 and 16x16 superblock).

UFBC_MODE_DEC400: Use the core of Dec400.

UFBC_MODE_PVRIC: Use the core of PVRIC.

-

union {

ufbc_afbc_param afbc

ufbc_pvric_param pvric

} **param**

10.22 ufbc_pvric_param

Contains parameters when the core of UFBC is PVRIC.

Data Fields

- **u32 blockType**

Surface tile mode.

0: 8x8.

1: 16x4.

2: 32x2.

- **u32 consColorVal [4]**

constant color value.

10.23 VceApiVersion

Defines the information of the encoder API version.

Data Fields

- **u32 major**
A major version with achitecture update or product promotion.
- **u32 minor**
A minor version with milestone features added.
- **u32 micro**
A micro version with feature improvement or interface changes.

10.24 VceBuildInfo

Defines the build information.

Data Fields

- **u32 swBuild**
The software build ID, which equals the changelist number in Perforce.
- **u32 hwBuild**
The hardware build ID, which comes from swreg0.

10.25 VCEncAGopInfo

For 1pass agop.

Data Fields

- int **sliceType**
- **i32 gopSize**
- **u32 motionScore** [2][2]
- **u32 poc**
- double **intra_ratio**
- double **skip_ratio**
- **u32 PBFrame4NRdCost**

10.26 VCEncAv1Para

Data Fields

- unsigned int **frame_type**
frame_type in uncompressed header
- unsigned char **slice_type**
converted from frame_type to adapt for general codec
- unsigned int **idr_pic_flag**
if current frame is an KEY frame
- unsigned int **reference_pic_flag**
if current frame is used as reference
- unsigned char **num_ref_idx_l0**
number of forward reference pictures.
- unsigned char **num_ref_idx_l1**
number of backward reference pictures.
- unsigned int **refresh_frame_flags**
refresh_frame_flags in uncompressed header
- unsigned int **ref_order_hint** [AV1_REF_FRAME_NUM]
ref_order_hint[] in uncompressed header
- unsigned int **ref_frame_idx** [AV1_REFS_PER_FRAME]
ref_frame_idx[] in uncompressed header
- unsigned short **order_hint**
The order_hint in uncompressed header.

10.26.1 Detailed Description

Input parameters extension for av1

10.27 VCEncCodingCtrl

Defines the coding control parameters.

Data Fields

- **u32 sliceSize**

The number of CTB or MB rows in each slice.

If the field value is 0, slice encoding is disabled and the entire frame is encoded as a slice.

If the field value is not 0, a total of $\lceil (\text{height} + \text{ctu_size} - 1) / \text{ctu_size} \rceil$ slices are created, with the height of each slice equal to $(\text{ctu_size} * \text{sliceSize})$ except the last slice.

- **u32 seiMessages**

Whether to insert picture timing and buffering period SEI messages into the stream.

0: do not insert.

1: insert.

- **u32 vuiVideoFullRange**

The input pixel sample range for VUI encoding.

0: Y samples in range from 16 to 235.

1: Y samples in range from 0 to 255.

- **u32 disableDeblockingFilter**

Whether to disable de-blocking filters. For HEVC (H.265), this field is used to generate `deblocking_filter_disabled_flag`.

0: enables de-blocking filters.

1: disables de-blocking filters.

2: disables de-blocking filters for slice edges, and enables them for other regions.

- **u32 sramPowerdownDisable**

Whether to disable the automatic SRAM power-down mode.

0: enable. SRAM is powered on only when it is in use.

1: disable. SRAM is always powered on.

- **u32 sramPowerdownMode**

- **u32 sramPowerdownTimerDiv32**

- **i32 tc_Offset**

The de-blocking parameter `tc_offset`.

- **i32 beta_Offset**

The de-blocking parameter `beta_offset`.

- **u32 enableDeblockOverride**

(For HEVC only) Whether to enable de-blocking override between frames.

0: disable.

1: enable. De-blocking parameters can be changed for each frame.

This field is used to generate `pps.deblocking_filter_override_enabled_flag` for HEVC (H.265).

- **u32 deblockOverride**

(For HEVC only) Whether to enable de-blocking override between slices. This field is used to generate `deblocking_filter_override_flag` in the slice header.

0: disable. `deblocking_filter_disabled_flag`, `tc_offset`, and `beta_offset` in PPS are used.

1: enable. `deblocking_filter_disabled_flag`, `tc_offset`, and `beta_offset` in the slice header are used.

This field is valid only if the value of `VCEncCodingCtrl.enableDeblockOverride` is 1.

- **u32 enableTS**
log2 Max Transform Skip Size - 2
- **u32 log2MaxTSBlockSizeMinus2**
- **u32 enableSao**
Whether to enable sample adaptive offset (SAO) for HEVC (H.265) or CDEF filter for AV1.
0: disable.
1: enable.
- **u32 enableScalingList**
(For HEVC only) Whether to enable the default scaling list.
0: disable. The average scaling list is used.
1: enable. The default scaling list is used.
- **u32 sampleAspectRatioWidth**
The sample aspect ratio in horizontal direction, in an arbitrary unit.
The value range is from 0 to 65535, inclusive. If the field value is 0, it indicates that the sample aspect ratio in horizontal direction is unspecified.
- **u32 sampleAspectRatioHeight**
The sample aspect ratio in vertical direction, in the same arbitrary unit as `VCEncCodingCtrl.sampleAspectRatioWidth`.
The value range is from 0 to 65535, inclusive. If the field value is 0, it indicates that the sample aspect ratio in vertical direction is unspecified.
- **u32 enableCabac**
(Only for AVC) The entropy coding mode.
0: content-adaptive variable-length coding (CAVLC).
1: content-based adaptive binary arithmetic coding (CABAC).
- **u32 cabacInitFlag**
The CABAC table initialization flag `cabac_init_flag` for HEVC (H.265) in the slice header.
Valid values include 0 and 1. Currently, the hardware supports only value 0.
- **u32 cirStart**
The order count of the first MB or CTB for cyclic intra refresh (CIR).
The value range is from 0 to `ctbPerFrame - 1`, inclusive. `ctbPerFrame` is the total number of MB or CTB in each frame.
- **u32 cirInterval**
The interval for CIR, in MB or CTB.
The value range is from 0 to `ctbPerFrame`, inclusive. `ctbPerFrame` is the total number of MB or CTB in each frame.
- **i32 pcm_enabled_flag**
Whether to enable IPCM encoding.
0: disable.
1: enable.
IPCM encoding can be lossless. It is specified in SPS.
- **i32 pcm_loop_filter_disabled_flag**
(For HEVC only) Whether to disable de-blocking and SAO filters for IPCM.
0: enables de-blocking and SAO filters around IPCM ROIs.
1: disables de-blocking and SAO filters for IPCM. This prevents the lossless content in the ROIs from being changed by the filters.

- **VCEncPictureArea intraArea**
The ROI for forcing the intra mode.
- **VCEncPictureArea ipcm1Area**
ROI 1 for forcing the IPCM mode.
- **VCEncPictureArea ipcm2Area**
ROI 2 for forcing the IPCM mode.
- **VCEncPictureArea ipcm3Area**
ROI 3 for forcing the IPCM mode.
- **VCEncPictureArea ipcm4Area**
ROI 4 for forcing the IPCM mode.
- **VCEncPictureArea ipcm5Area**
ROI 5 for forcing the IPCM mode.
- **VCEncPictureArea ipcm6Area**
ROI 6 for forcing the IPCM mode.
- **VCEncPictureArea ipcm7Area**
ROI 7 for forcing the IPCM mode.
- **VCEncPictureArea ipcm8Area**
ROI 8 for forcing the IPCM mode.
- **VCEncPictureArea roi1Area**
ROI 1 for forcing a QP delta or an absolute QP value.
- **VCEncPictureArea roi2Area**
ROI 2 for forcing a QP delta or an absolute QP value.
- **i32 aifQpDelta**
AIF QP delta is QP delta between AIF frame and its reference frame.
- **VCEncPictureArea roi3Area**
ROI 3 for forcing a QP delta or an absolute QP value.
This field is valid only if the value of `EWLHwConfig_t.ROI8Support` is 1.
- **VCEncPictureArea roi4Area**
ROI 4 for forcing a QP delta or an absolute QP value.
This field is valid only if the value of `EWLHwConfig_t.ROI8Support` is 1.
- **VCEncPictureArea roi5Area**
ROI 5 for forcing a QP delta or an absolute QP value.
This field is valid only if the value of `EWLHwConfig_t.ROI8Support` is 1.
- **VCEncPictureArea roi6Area**
ROI 6 for forcing a QP delta or an absolute QP value.
This field is valid only if the value of `EWLHwConfig_t.ROI8Support` is 1.
- **VCEncPictureArea roi7Area**
ROI 7 for forcing a QP delta or an absolute QP value.
This field is valid only if the value of `EWLHwConfig_t.ROI8Support` is 1.

- **VCEncPictureArea roi8Area**
ROI 8 for forcing a QP delta or an absolute QP value.
This field is valid only if the value of `EWLHwConfig_t.ROI8Support` is 1.
- **i32 roi1DeltaQp**
The QP delta for ROI 1.
The value range is from -30 to 0 , inclusive.
- **i32 roi2DeltaQp**
The QP delta for ROI 2.
The value range is from -30 to 0 , inclusive.
- **i32 roi3DeltaQp**
The QP delta for ROI 3.
The value range is from -30 to 0 , inclusive.
- **i32 roi4DeltaQp**
The QP delta for ROI 4.
The value range is from -30 to 0 , inclusive.
- **i32 roi5DeltaQp**
The QP delta for ROI 5.
The value range is from -30 to 0 , inclusive.
- **i32 roi6DeltaQp**
The QP delta for ROI 6.
The value range is from -30 to 0 , inclusive.
- **i32 roi7DeltaQp**
The QP delta for ROI 7.
The value range is from -30 to 0 , inclusive.
- **i32 roi8DeltaQp**
The QP delta for ROI 8.
The value range is from -30 to 0 , inclusive.
- **i32 roi1Qp**
The absolute QP value for ROI 1.
The value range is from -1 to 51 , inclusive. A value of -1 indicates that the absolute QP value is not used.
- **i32 roi2Qp**
The absolute QP value for ROI 2.
The value range is from -1 to 51 , inclusive. A value of -1 indicates that the absolute QP value is not used.
- **i32 roi3Qp**
The absolute QP value for ROI 3.
The value range is from -1 to 51 , inclusive. A value of -1 indicates that the absolute QP value is not used.
- **i32 roi4Qp**
The absolute QP value for ROI 4.
The value range is from -1 to 51 , inclusive. A value of -1 indicates that the absolute QP value is not used.
- **i32 roi5Qp**

The absolute QP value for ROI 5.

The value range is from -1 to 51, inclusive. A value of -1 indicates that the absolute QP value is not used.

- **i32 roi6Qp**

The absolute QP value for ROI 6.

The value range is from -1 to 51, inclusive. A value of -1 indicates that the absolute QP value is not used.

- **i32 roi7Qp**

The absolute QP value for ROI 7.

The value range is from -1 to 51, inclusive. A value of -1 indicates that the absolute QP value is not used.

- **i32 roi8Qp**

The absolute QP value for ROI 8.

The value range is from -1 to 51, inclusive. A value of -1 indicates that the absolute QP value is not used.

- **u32 fieldOrder**

The field order for HEVC (H.265) interlaced coding.

0: bottom field first.

1: top field first.

- **i32 chroma_qp_offset**

The chroma QP offset.

The value range is from -12 to 12, inclusive.

- **u32 roiMapDeltaQpEnable**

Whether to enable the ROI map for forcing QP delta or absolute QP values.

0: disable.

1: enable.

- **u32 roiMapDeltaQpBlockUnit**

The size of the unit block for the QP map.

0: 64x64.

1: 32x32.

2: 16x16.

3: 8x8.

- **u32 ipcmMapEnable**

Whether to enable the ROI map for forcing IPCM.

0: disable.

1: enable.

- **u32 RoimapCuCtrl_index_enable**

Whether to enable the index for the CU control map.

0: disable.

1: enable.

- **u32 RoimapCuCtrl_enable**

(Reserved) Whether to enable the CU control map.

0: disable.

1: enable.

- **u32 RoimapCuCtrl_ver**

The version of the CU information format used in the CU control map.

- **u32 RoiQpDelta_ver**
The version of the QP information format used in the QP map.
- **u32 skipMapEnable**
Whether to enable the skip-mode ROI map.
0: disable.
1: enable.
- **u32 rdoqMapEnable**
Whether to enable the RDOQ map.
0: disable.
1: enable.
- **VCEncPictureArea rect0**
Region 0 for SSE information statistics.
This field is valid only if the value of `VCEncConfig.enableFrameInfo` is 1.
- **VCEncPictureArea rect1**
Region 1 for SSE information statistics.
This field is valid only if the value of `VCEncConfig.enableFrameInfo` is 1.
- **VCEncPictureArea rect2**
Region 2 for SSE information statistics.
This field is valid only if the value of `VCEncConfig.enableFrameInfo` is 1.
- **VCEncPictureArea rect3**
Region 3 for SSE information statistics.
This field is valid only if the value of `VCEncConfig.enableFrameInfo` is 1.
- **VCEncPictureArea rect4**
Region 4 for SSE information statistics.
This field is valid only if the value of `VCEncConfig.enableFrameInfo` is 1.
- **VCEncPictureArea rect5**
Region 5 for SSE information statistics.
This field is valid only if the value of `VCEncConfig.enableFrameInfo` is 1.
- **VCEncPictureArea rect6**
Region 6 for SSE information statistics.
This field is valid only if the value of `VCEncConfig.enableFrameInfo` is 1.
- **VCEncPictureArea rect7**
Region 7 for SSE information statistics.
This field is valid only if the value of `VCEncConfig.enableFrameInfo` is 1.
- **u32 noiseReductionEnable**
Obsoleted.
- **u32 noiseLow**
Obsoleted.
- **u32 firstFrameSigma**
Obsoleted.
- **u32 noiseSigmaEst**

0 = disable noise sigma estimation; 1 = enable noise sigma estimation

- **u32 noiseSigmaY**
valid value range :[1...30] , default: 10
- **u32 noiseSigmaU**
valid value range :[1...30] , default: 10
- **u32 noiseSigmaV**
valid value range :[1...30] , default: 10
- **u32 noiseReductionStrength_IntraY**
valid value range :[0...32] , default: 7
- **u32 noiseReductionStrength_IntraU**
valid value range :[0...32] , default: 7
- **u32 noiseReductionStrength_IntraV**
valid value range :[0...32] , default: 7
- **u32 noiseReductionStrength_InterY**
valid value range :[0...32] , default: 7
- **u32 noiseReductionStrength_InterU**
valid value range :[0...32] , default: 7
- **u32 noiseReductionStrength_InterV**
valid value range :[0...32] , default: 7
- **u32 noiseReduction_ChromaMaxMV**
valid value range :[0...15] , default: 4
- **u32 gdrDuration**
The duration it takes to perform gradual decoder refresh (GDR), in the unit of frame.
The value range is from 0 to 0xFFFFFFFF, inclusive. If the field value is 0, GDR is disabled.
- **u32 inputLineBufEn**
Whether to enable low-latency encoding.
0: disable.
1: enable.
- **u32 inputLineBufLoopBackEn**
Whether to enable the loopback mode for the input buffer.
0: disable.
1: enable.
- **u32 inputLineBufDepth**
The depth of the input buffer, in the unit of MB line.
- **u32 inputLineBufHwModeEn**
Whether to enable hardware handshake.
0: disable.
1: enable.
- **u32 amountPerLoopBack**

The number of handshake synchronizations every loopback.

- **EncInputLineBufCallBackFunc inputLineBufCbFunc**

The callback function that is called when a line buffer interrupt is detected.

- **void * inputLineBufCbData**

The context to be used by `VCEncCodingCtrl.inputLineBufCbFunc`.

- **u32 sbi_id_0**

The ID of FLEXA SBI 0.

- **u32 sbi_id_1**

The ID of FLEXA SBI 1.

- **u32 sbi_id_2**

The ID of FLEXA SBI 2.

- **u32 segmentUnitHeight**

The height of each FLEXA SBI segment unit, in pixels.

- **u32 lowlatGatingDisable**

lowlatency handshake auto gating disable

0: enable.

1: disable.

- **u32 lowlatGatingType**

lowlatency handshake auto gating type

- **u32 lowlatGatingCyc**

bus idle counter max value

- **u32 enable_slice_irq**

disable TU32.

- **u32 streamMultiSegmentMode**

(Reserved) The multi-segment mode of output stream buffers.

0: single-segment.

1: multi-segment with no synchronization.

2: multi-segment with software handshake.

- **u32 streamMultiSegmentAmount**

(Reserved) The number of segments in a stream output buffer.

The field value is greater than or equal to 2.

- **EncStreamMultiSegCallBackFunc streamMultiSegCbFunc**

(Reserved) The callback function that is called when a stream segment interrupt is detected.

- **void * streamMultiSegCbData**

(Reserved) The context to be used by `VCEncCodingCtrl.streamMultiSegCbFunc`.

- **u32 batchCount**

Batch Mode Control Parameters. only work when using VCMD. In batch mode, software will cache the encoding tasks in current instance. The maximum cache size is defined as `parallelCoreNum`. When accumulate task to this value, software will drive the hardware to run these tasks in batch way.

- **i32 smartModeEnable**

Obsoleted.

- **i32 smartH264Qp**

Obsoleted.

- **i32 smartHevcLumQp**

Obsoleted.

- **i32 smartHevcChrQp**

Obsoleted.

- **i32 smartH264LumDcTh**

Obsoleted.

- **i32 smartH264CbDcTh**

Obsoleted.

- **i32 smartH264CrDcTh**

Obsoleted.

- **i32 smartHevcLumDcTh [3]**

Obsoleted.

- **i32 smartHevcChrDcTh [3]**

Obsoleted.

- **i32 smartHevcLumAcNumTh [3]**

Obsoleted.

- **i32 smartHevcChrAcNumTh [3]**

Obsoleted.

- **i32 smartMeanTh [4]**

Obsoleted.

- **i32 smartPixNumCntTh**

Obsoleted.

- **u32 ctbRcSkinQPDelta**

Negative qp delta of skin area.

- **u32 ctbRcSkinMinQPDelta**

*Skin area not set too small QPDelta(complexity + edge + skin), which use ctbRcSkinMinQPDelta - direction to limit it.
ctbRcSkinMinQPDelta - direction always less than 0.*

- **u32 ctbRcSkinCbMin**

Min cb value to be regarded as skin Area.

- **u32 ctbRcSkinCbMax**

Max cb value to be regarded as skin Area.

- **u32 ctbRcSkinCrMin**

Min cr value to be regarded as skin Area.

- **u32 ctbRcSkinCrMax**

- Max cr value to be regarded as skin Area.*
- **u32 ctbRcSkinLumMin**

Min lum value to be regarded as skin Area.
- **u32 ctbRcSkinLumMax**

Max lum value to be regarded as skin Area.
- **u32 ctbRcSkinGradTh**

Skin Grad threshold for skin detection.
- **u32 ctbRcTrailStrengthMax**

Trailing area detection max strength.
- **i32 ctbRcTrailDeltaQp**

Trailing area base qp delta.
- **u32 ctbRcEdgeTh**

Strength to calculate edge threshold. The texture will be more easily regarded as edge with bigger value
- **u32 ctbRcEdgeMaxQpDelta**

Edge area not set too big QPDelta(complexity + edge), which use ctbRcEdgeMaxQPDelta - direction to limit it.
- **u32 ctbRcDirection**

Subscript of ctbRcThresholdI/ctbRcThresholdP/ctbRcThresholdB, It used with ctbRcThreshold to decrease/increase block QP. When the block complexity is between ctbRcThreshold[0] and ctbRcThreshold[Direction] will decrease QP, and the complexity is bigger than ctbRcThreshold[Direction] will increase QP.
- **u32 ctbRcThresholdI [16]**

A threshold array, it used with ctbRcDirection to decrease/increase block QP of Intra frame.
- **u32 ctbRcThresholdP [16]**

A threshold array, it used with ctbRcDirection to decrease/increase block QP of P frame
- **u32 ctbRcThresholdB [16]**

A threshold array, it used with ctbRcDirection to decrease/increase block QP of B frame
- **i32 visualBitRateTolerance**

Subjective prefer cause bitrate increasment tolerance.
- **u32 IntraBiasChromaStrength**

Chroma error check strength for subjective quality prior mode decision.
- **u32 IntraBiasStrength**

Intra mode bias strength for subjective quality prior mode decision.
- **u32 IntraBiasMvThreshold**

Min MV threshold for Intra mode bias.
- **u32 bTrailAvoidIntraBias**

Whether intraBias is avoided in trail area.

- **ITU_T_T35 itu_t_t35**

The T.35 code and payload data information for HDR10.

- **u32 write_once_HDR10**

Whether to write HDR10 information only before the first IDR frame.

0: write before each IDR frame.

1: write only before the first IDR frame.

- **Hdr10DisplaySei Hdr10Display**

The colume information of the mastering display for HDR10.

- **Hdr10LightLevelSei Hdr10LightLevel**

The HDR10 light-level information.

- **VuiColorDescription vuiColorDescription**

The color description in the VUI.

For details, see Section [VUI Video Signal Type](#).

- **u32 vuiVideoSignalTypePresentFlag**

Whether to present the video signal type in the VUI.

0 (default): do not present.

1: present.

For more information about VUI, see Section [VUI Video Signal Type](#).

- **u32 vuiVideoFormat**

The video format in the VUI. For details, see Section [VUI Video Signal Type](#).

0: component.

1: PAL.

2: NTSC.

3: SECAM.

4: MAC.

5 (default): UNDEF.

- **u32 RpsInSliceHeader**

(For HEVC only) Whether to specify the RPS in the slice header instead of using the pre-defined RPS from the SPS.

0: use the pre-defined RPS from the SPS.

1: specify the RPS in the slice header.

- **u32 enableRdoQuant**

Whether to enable rate-distortion optimized quantization (RDOQ). For details, see Section [Rate-Distortion Optimized Quantization \(RDOQ\)](#).

0: disable.

1: enable.

- **u32 enableDynamicRdo**

Whether to enable dynamic RDO level selection. For details, see Section [Tuning for Various Quality Targets](#).

0: disable.

1: enable.

- **u32 dynamicRdoCu16Bias**

The bias used for 16x16 CU cost calculation in dynamic RDO level selection.

- **u32 dynamicRdoCu16Factor**

The factor used for 16x16 CU cost calculation in dynamic RDO level selection.

- **u32 dynamicRdoCu32Bias**

The bias used for 32x32 CU cost calculation in dynamic RDO level selection.

- **u32 dynamicRdoCu32Factor**

The factor used for 32x32 CU cost calculation in dynamic RDO level selection.

- **u32 meVertSearchRange**

The ME vertical search range, in pixels.

The value must be less than or equal to the maximum vertical search range allowed, which can be queried through `EWLHwConfig_t.meVertSearchRangeHEVC` or `EWLHwConfig_t.meVertSearchRangeH264`.

- **u32 layerInRefIdcEnable**

(Only for H.264) Whether to enable layer information in the `nal_ref_idc` syntax element.

0: disable. In this case, the value of `nal_ref_idc` can be 0 or 1.

1: enable. In this case, the value of `nal_ref_idc` can be 0 to 3.

The feature is supported only for specific custom versions.

- **u32 prefixNalSvcFlag**

(Only for H.264) Whether to add prefix NAL units between slices for SVC-T.

0: do not add.

1: add.

- **u32 svctEnable**

(Only for H.264) Whether to enable SVC-T.

0: disable.

1: enable.

- **u32 aq_mode**

The adaptive quantization mode. For details, see Section [Adaptive Quantization](#).

0 (default): none.

1: uniform AQ.

2: auto variance.

3: auto variance with bias to dark scenes.

- **float aq_strength**

The strength of adaptive quantization. A great value reduces blocking and blurring in flat and textured areas. For details, see Section [Adaptive Quantization](#).

The value range is from 0.0 to 3.0, inclusive. The default value is 1.0.

- **float psyFactor**

The strength of psycho-visual encoding.

The value range is 0 to 4, inclusive. If the field value is 0, psycho-visual encoding is disabled. Otherwise, the greater the field value, the better the subjective quality.

- **u32 intraReconEnable**

Whether to enable polling line input.

0: disable.

1: enable.

- **u32 inputSliceInfoPollEn**

- **u32 sw_skip_flag**

- **u32 gopMaxBSize**

10.27.1 Field Documentation

10.27.1.1 deblockOverride

u32 deblockOverride

Enable Transform Skip

10.27.1.2 enable_slice_irq

u32 enable_slice_irq

Whether to enable slice interrupts.

0: disable.

1: enable.

The callback function `VCEncSliceReadyCallBackFunc()` can be called only if slice interrupts and slice encoding are enabled.

10.27.1.3 visualBitRateTolerance

i32 visualBitRateTolerance

\visual quality prior mode decision

10.27.1.4 intraReconEnable

u32 intraReconEnable

Whether to enable intraRecon.

0: disable.

1: enable.

10.28 VCEncColorConversion

Contains configurations of RGB-to-YUV conversion. For details, see Section [RGB-to-YUV Conversion](#).

Data Fields

- **VCEncColorConversionType type**

The RGB-to-YUV conversion type.

- **u16 coeffA**

The custom coefficient A.

- **u16 coeffB**

The custom coefficient B.

- **u16 coeffC**

The custom coefficient C.

- **u16 coeffE**

The custom coefficient E.

- **u16 coeffF**

The custom coefficient F.

- **u16 coeffG**

The custom coefficient G.

- **u16 coeffH**

The custom coefficient H.

- **u16 LumaOffset**

The custom luma offset.

10.29 VCEncConfig

Defines the configurations for encoder instance initialization.

Data Fields

- **VCEncMode enc_mode**
encoding mode
- **VCEncStreamType streamType**
(Only for HEVC/H.264) The stream type.
- **VCEncProfile profile**
The stream profile.
- **VCEncLevel level**
The stream level.
If the field value is 0, the level is selected automatically. For details, see Section [Automatic Level Detection](#).
- **VCEncTier tier**
The stream tier.
- **u32 width**
The width of the encoded picture after rotation, in pixels.
The field value is a multiple of 2.
The width is restricted by level limitations. For details, see [VCEncLevel](#).
- **u32 height**
The height of the encoded picture after rotation, in pixels.
The field value is a multiple of 2.
The width is restricted by level limitations. For details, see [VCEncLevel](#).
- **u32 frameRateNum**
The numerator for calculating the target frame rate. For details, see Section [Frame Rate Descriptor](#).
The value range is from 1 to 1048575, inclusive.
- **u32 frameRateDenom**
The denominator for calculating the target frame rate. For details, see Section [Frame Rate Descriptor](#).
The value range is from 1 to [VCEncConfig.frameRateNum](#), inclusive.
- **u32 refFrameAmount**
The number of reference frame buffers.
The value range is from 0 to 8, inclusive. For example:
 - For encoding of I-frames only, the field value is 0.
 - If gopSize=1 and interlacedFrame=0, the field value is 1.
 - If gopSize=1 and interlacedFrame=1, the field value is 2.
 - If gopSize=2 or 3, the field value is 2.
 - If gopSize=4, 5, 6, or 7, the field value is 3.
 - If gopSize=8, the field value is 4.
 - If gopSize=8 with SVC-T, 7 B-frames, 1 P-frame, and 4 layers, the field value is 8.
- **u32 strongIntraSmoothing**
Whether to enable normal or strong smoothing.
0: normal smoothing.
1: strong smoothing.
- **u32 compressor**

Whether to enable reference frame compression (RFC). For details, see Section [Reference Frame Compression](#).

0: disables RFC.

1: enables RFC only for luma data. Currently, luma-only RFC is not supported.

2: enables RFC only for chroma data. Currently, chroma-only RFC is not supported.

3: enables RFC for both luma and chroma data.

- **u32 interlacedFrame**

Whether the input frames are progressive or interlaced.

Interlaced frame input is supported only for HEVC (H.265). For details, see Section [Interlaced Coding for HEVC](#).

0: progressive frame input.

1: interlace frame input.

- **u32 bitDepthLuma**

The bit depth of luma samples in the encoded stream.

8: 8-bit luma samples.

10: 10-bit luma samples.

- **u32 bitDepthChroma**

The bit depth of chroma samples in the encoded stream.

8: 8-bit chroma samples.

10: 10-bit chroma samples.

- **u32 enableOutputCuInfo**

Whether to output CU/MB statistics. For details, see Section [CU/MB Statistics Output](#).

0: do not output.

1: output.

- **u32 enableFrameInfo**

Whether to output frame statistics. For details, see Section [Frame Statistics Output](#).

0: disable.

1: enable.

- **u32 enableLumaInfo**

Whether to output luma statistics. For details, see Section [ss_outLumaInfo](#).

0: disable.

1: enable.

- **u32 enableOutputCtbBits**

Whether to output CTB bit statistics. For details, see Section [Bit Statistics Output](#).

0: do not output.

1: output.

- **u32 enableSsim**

Whether to enable SSIM calculation.

0: disable.

1: enable.

- **u32 enablePsnr**

Whether to enable PSNR calculation.

0: disable.

1: enable.

- **u32 maxTlLayers**

The maximum number of temporal layers.

- **u32 rdoLevel**

The RDO level, which controls how much effort the video encoder uses for mode selection. For details, see Section [RDO Level Selection](#).

0: runs 1x candidates.

1: runs 2x candidates.

2: runs 3x candidates.

- **VCEncVideoCodecFormat codecFormat**

The video codec standard.

- **u32 verbose**

Obsoleted. Use `VCEncLogSetting.out_level` instead.

- **u32 exp_of_input_alignment**

The alignment requirement for both luma and chroma input frame buffers.

The start address of each line in the buffer must be aligned to $\text{power}(2, \text{exp_of_input_alignment})$.

`exp_of_input_alignment` must be greater than or equal to the `AXI_burst_align_rd_prp` subfield of `VCEncConfig.AXIAAlignment`.

- **u32 exp_of_ref_alignment**

The alignment requirement for luma reference frame buffers.

The start address of each line in the buffer must be aligned to $\text{power}(2, \text{exp_of_ref_alignment})$.

`exp_of_ref_alignment` must be greater than or equal to the `AXI_burst_align_rd_lu_ref_prefetch` and `AXI_burst_align_wr_luma_ref` subfields of `VCEncConfig.AXIAAlignment`.

- **u32 exp_of_ref_ch_alignment**

The alignment requirement of chroma reference frame buffers.

The start address of each line in the buffer must be aligned to $\text{power}(2, \text{exp_of_ref_ch_alignment})$.

`exp_of_ref_ch_alignment` must be greater than or equal to the `AXI_burst_align_rd_ch_ref_prefetch` and `AXI_burst_align_wr_chroma_ref` subfields of `VCEncConfig.AXIAAlignment`.

- **u32 exp_of_aqinfo_alignment**

The alignment requirement of adaptive quantization output buffers.

The start address of each line in the buffer must be aligned to $\text{power}(2, \text{exp_of_aqinfo_alignment})$.

- **u32 exp_of_tile_stream_alignment**

The alignment requirement of output stream buffers.

The buffer size must be aligned to $\text{power}(2, \text{exp_of_tile_stream_alignment})$.

`exp_of_tile_stream_alignment` must be greater than or equal to the `AXI_burst_align_wr_stream` subfield of `VCEncConfig.AXIAAlignment`.

- **u32 externalReconAlloc**

The allocation mode of reconstructed frames.

0: Reconstructed frames are allocated by the encoder.

1: Reconstructed frames are allocated by the application.

- **u32 P010RefEnable**

Whether to store reference frames in tiled or raster P010 format in buffers.

0 (default): tile.

1: raster.

- **u32 ctbRcMode**

The block-level QP adjustment mode for precise and subjective rate control.

0: no block-level QP adjustment.

1: block-level QP adjustment for subjective rate control only.

2: block-level QP adjustment for precise rate control only.

3: block-level QP adjustment for both precise and subjective rate control.

- **u32 picOrderCntType**

The method to encode POC in the slice header.

This field is equivalent to `pic_order_cnt_type` defined in the ITU-T Rec. H.264.

- **u32 log2MaxPicOrderCntLsb**

The number of bits used to encode POC in the slice header.

This field is equal to $(4 + \log_2_max_pic_order_cnt_lsb_minus4)$, which is defined in the ITU-T Rec. H.264.

- **u32 log2MaxFrameNum**

The number of bits used to encode `frame_num` in the H.264 (AVC) slice header.

This field is equal to $(4 + \log_2_max_frame_num_minus4)$, which is defined in the ITU-T Rec. H.264.

- **u32 dumpRegister**

Whether to dump register values for debugging.

0: do not dump.

1: dump.

- **u32 dumpCuInfo**

Whether to dump CU/MB statistics for debugging after encoding each frame.

0: do not dump.

1: dump.

- **u32 dumpCtbBits**

Whether to dump CTB bit statistics after encoding one frame.

0: do not dump.

1: dump.

- **u32 rasterscan**

Whether to dump reconstructed YUV frames in tiled or raster format for debugging when the encoder runs on FPGA and hardware.

0 (default): tiled format.

1: raster format.

- **u32 parallelCoreNum**

The number of cores that run in parallel at the frame level.

- **u32 pass**

A multi-pass coding configuration, for internal testing only.

- **bool bPass1AdaptiveGop**

Whether to enable GOP structure size adaption based on pass-1 information.

true: enable.

false: disable.

- **u8 lookaheadDepth**

The number of look-ahead frames used to encode each frame.

0: disables look-ahead encoding.

4 to 40: enables look-ahead encoding with the specified number of look-ahead frames.

- **u32 extDSRatio**

The external down-scaling ratio for the pass-1 encoder.

0: down-scaling disabled.

1: 1/2 down-scaling ratio.

- **u32 inLoopDSRatio**

The in-loop down-scaling ratio for the pass-1 encoder.

0: down-scaling disabled.

1: 1/2 down-scaling ratio.

- **i32 cuInfoVersion**

The CU information version.

If the field value is -1, the version is decided by the hardware support.

Other valid values include 0, 1, and 2. For details, see Hantro VC9000E Series Memory Buffer and Format Organization.

- **u32 gopSize**

The GOP structure size.

To enable GOP structure size adaption, set this field to 0.

- **u32 gopMaxBSize**

The GOP structure maximum number of B frames for AGOP.

To enable GOP structure maximum number of B frames adaption for AGOP, set this field to 7.

- **u32 aifEnable**

enable or disable anti intra flicker (aif)

0 - disable aif

1 - enable aif

- **u32 numRefP**

reference frame number of P frame, numRefP=1|2

- **u32 slice_idx**

The index of the server slice node.

- **u32 extSramLumHeightBwd**

The capacity of external SRAM for luma backward reference, in the unit of reference line.

This field is valid only if external SRAM is enabled to save motion estimation (ME) bandwidth.

- **u32 extSramChrHeightBwd**

The capacity of external SRAM for chroma backward reference, in the unit of reference line.

This field is valid only if external SRAM is enabled to save ME bandwidth.

- **u32 extSramLumHeightFwd**

The capacity of external SRAM for luma forward reference, in the unit of reference line.

This field is valid only if external SRAM is enabled to save ME bandwidth.

- **u32 extSramChrHeightFwd**

The capacity of external SRAM for chroma forward reference, in the unit of reference line.

This field is valid only if external SRAM is enabled to save ME bandwidth.

- **u64 AXIAlignment**

The AXI alignment. The start address of each line in the buffer must be aligned to two to the Nth power where N is the value of the corresponding field.

Bits 35:32: AXI_burst_align_wr_cuinfo

Bits 31:28: AXI_burst_align_wr_common

Bits 27:24: AXI_burst_align_wr_stream

Bits 23:20: AXI_burst_align_wr_chroma_ref

Bits 19:16: AXI_burst_align_wr_luma_ref

Bits 15:12: AXI_burst_align_rd_common

Bits 11:8: AXI_burst_align_rd_prp

Bits 7:4: AXI_burst_align_rd_ch_ref_prefetch

Bits 3:0: AXI_burst_align_rd_lu_ref_prefetch

- **u32 irqTypeMask**

The interrupt type bitmap of the encoder.

Bit 8: irq_type_sw_reset_mask

Bit 7: irq_type_fuse_error_mask

Bit 6: irq_type_buffer_full_mask

Bit 5: irq_type_bus_error_mask

Bit 4: irq_type_timeout_mask

Bit 3: irq_type_strm_segment_mask

Bit 2: irq_type_line_buffer_mask

Bit 1: irq_type_slice_rdy_mask

Bit 0: irq_type_frame_rdy_mask

Each bit indicates whether a normal or abnormal interrupt is raised.

0: normal interrupt. The interrupt makes VCMD exit the stall status and continue with subsequent commands in the VCMD buffer.

1: abnormal interrupt. The interrupt keeps VCMD in stall status and directly transfers the interrupt to CPU.

- **u32 irqTypeCutreeMask**

The interrupt type bitmap of CuTree.

Bit 5: irq_type_bus_error_mask

Bit 4: irq_type_timeout_mask

Bit 0: irq_type_frame_rdy_mask

Each bit indicates whether a normal or abnormal interrupt is raised.

0: normal interrupt. The interrupt makes VCMD exit the stall status and continue with subsequent commands in the VCMD buffer.

1: abnormal interrupt. The interrupt keeps VCMD in stall status and directly transfers the interrupt to CPU.

- **VCEncChromaIdcType codedChromaIdc**

The chroma sampling modes relative to luma sampling. For details, see Section [Coding Formats and Profiles](#).

This field is used as the SPS syntax element `chroma_format_idc` specified in ITU-T Rec. H.265 and H.264.

- **VCENC_TuneType tune**

The type of the target based on which some encoding parameters are forced for quality tuning. For details, see Section [Tuning for Various Quality Targets](#).

The default value is `VCENC_TUNE_PSNR`.

- **u32 writeReconToDDR**

Whether the hardware writes reconstructed frames to DDR for pure I-frame encoding.

0: does not write.

1: writes.

- **u32 TxTypeSearchEnable**

(For AV1 only) Whether to enable TX type search.

0: disable.

1 (default): enable.

- **u32 av1InterFiltSwitch**

(For AV1 only) Whether to enable interpolation filter switch.

0: disable.

1 (default): enable.

- **u32 fileListExist**

A flag for stream-level encoding control.

Set this flag to 1 only if `VCEncSetStrmCtrl()` is used to reset the instance status for a new stream.

- **u32 burstMaxLength**

The maximum AXI burst length, in the unit of AXI bus width.

- **u32 enableTMVP**

Whether to enable temporal motion vector prediction (TMVP).

0: disable.

1: enable.

- **u32 refRingBufEnable**

Whether to enable reference ring buffer.

0: disable.

1: enable.

- **u32 ctbRcTrailStrength**

Trailing area detection strength.

- **i32 tiles_enabled_flag**

(For HEVC only) Whether to enable tile-based encoding.

0: disable.

1: enable.

- **i32 num_tile_columns**

The number of tile columns to divide the picture.

The only available value is 2.

- **i32 num_tile_rows**

The number of tile rows to divide the picture.

- **i32 loop_filter_across_tiles_enabled_flag**

Whether to enable the loop filter across the tile boundary.

0: disable.

1: enable.

- **i32 * tile_width**

An array with a total of `VCEncConfig.num_tile_columns` entries.

Each entry indicates the width of a tile column.

- **i32 * tile_height**
An array with a total of `VCEncConfig.num_tile_rows` entries.
Each entry indicates the width of a tile column.
- **i32 tileMvConstraint**
Reserved.
- **u32 secure_mode**
Whether to enable security encoding.
0: disable.
1: enable.
- **u32 bIOBufferBinding**
Whether to bind input and output buffers.
0: do not bind.
1: bind.
- **VCEncTryNewParamsCallBackFunc cb_try_new_params**
The callback function called to obtain new parameters for re-encoding.
- **ufbc_param ufbcParam**
Contains the UFBC parameters.
- **u32 priority**
the priority of current instance
- **u32 core_mask**
the core bit mask of current instance
0x0: not specify, anycore.
0x1: sepcfiy core0.
0x2: sepcfiy core1.
- **u32 enableLeadingPictures**
enable encoding of leading pictures
- **u32 testId**
the testId for internal test
- **char * enc_dev**
the device name of enc driver.
default is "vsi_vcx"
- **char * mem_dev**
the device name of memalloc driver
default is "memalloc"
- **u32 useVcmd**
Whether to use vcmd.
This field is valid only for cmodel.
- **u32 lowlatGatingDisable**
lowlatency handshake auto gating disable
0: enable.
1: disable.

- **u32 lowlatGatingType**
lowlatency handshake auto gating type
- **u32 lowlatGatingCyc**
bus idle counter max value
- **u32 disableTU32**
disable TU32.

10.30 VCEncConsumedAddr

Contains the bus addresses consumed by the encoder for encoding of a picture.

Data Fields

- **ptr_t inputbufBusAddr**
The bus address of the input buffer.
- **ptr_t outbufBusAddr**
The bus address of the output buffer.
- **ptr_t dec400TableBusAddr**
The bus address of the DEC400 compression table buffer.
- **ptr_t roiMapDeltaQpBusAddr**
The bus address of the QP map buffer.
- **ptr_t roiMapCuCtrlInfoBusAddr**
The bus address of the CU control map buffer.
- **ptr_t overlayInputBusAddr [MAX_OVERLAY_NUM]**
The bus addresses of the OSD inputs.
- **ptr_t osdMapInputBusAddr**
(Under development) The bus address of the OSD map buffer.

10.31 VCEncCuInfo

Contains the statistics of a CU from the video encoder.

You can enable or disable CU statistics output through `VCEncConfig.enableOutputCuInfo`.

When CU statistics output is enabled, you can obtain the CU information by calling `VCEncGetCuInfo()`.

The video encoder supports CU information output in different formats. Only fields valid for the version selected through `VCEncConfig.cuInfoVersion` are parsed and filled into this structure. For details about CU information formats, see *Hantro VC9000E Series Memory Buffer and Format Organization*.

Data Fields

- **u8 cuLocationX**
The X coordinate of the top-left corner of the CU relative to its CTU.
- **u8 cuLocationY**
The Y coordinate of the top-left corner of the CU relative to its CTU.
- **u8 cuSize**
The CU size.
8: 8x8 pixels.
16: 16x16 pixels.
32: 32x32 pixels.
64: 64x64 pixels.
- **u8 cuMode**
The CU mode.
0: inter mode.
1: intra mode.
- **u32 cost**
The SSE cost of the selected CU mode.
This field is invalid for the IPCM mode.
- **u32 costOfOtherMode**
The SSE cost of the inter or intra CU mode other than the selected CU mode.
This field is invalid for the IPCM mode.
- **u32 costIntraSatd**
The SATD cost of the intra mode.
- **u32 costInterSatd**
The SATD cost of the inter mode.
- **u8 interPredIdc**
The prediction direction.
0: unidirectional by list 0.
1: unidirectional by list 1.
2: bi-directional.
This field is valid only for an inter-mode CU.
- **VCEncMv mv [2]**
The motion information.
mv [0] is for list 0 if list 0 is valid.
mv [1] is for list 1 if list 1 is valid.
This field is valid only for an inter-mode CU.
- **u8 intraPartMode**
The partition mode.
0: 2Nx2N.
1: NxN.
This field is valid only for an intra-mode CU.
- **u8 intraPredMode [16]**

The prediction mode.

Each entry can be set to a value in range 0 to 34, inclusive.

- 0: planar.
- 1: DC.
- 2 to 34: angular in ITU-T Rec. H.265.

For HEVC (H.265) and AV1 encoding:

- If the value of `intraPartMode` is 1, `intraPredMode[0]` is valid.
- If the value of `intraPartMode` is 0, `intraPredMode[0]` to `intraPredMode[3]` are valid.

For AV1, this field is equivalent to `y_mode` in the specification.

For H.264 (AVC) encoding, `intraPredMode[0]` to `intraPredMode[15]` are valid.

- **u8 qp**

The QP used to encode the CU.

- **u32 mean**

The mean value of the CU in the input picture.

- **u32 variance**

The variance value of the CU in the input picture.

10.32 VCEncCuOutData

Contains the CU or MB statistics of a picture, which is output by the hardware.

Data Fields

- **u32 * ctuOffset**

A pointer to the memory that stores the total CU number by the end of each CTU.

- **u8 * cuData**

The CU/MB statistics of a picture that is output by the hardware.

10.33 VCEncCuStatis

The CU statistics for adaptive GOP structure size decision.

Data Fields

- **u32 intraCu8Num**
The number of 8x8 blocks in intra mode.
- **u32 skipCu8Num**
The number of 8x8 blocks in skip mode.
- **u32 PBFrame4NRdCost**
The rate distortion (RD) cost used in reconstructed frame down-sampling ME search for inter-frames.

10.34 VCEncExtParaIn

Defines the libva-private input to the encoder.

Data Fields

- struct node * **next**
*A pointer to the next **VCEncExtParaIn** instance.*
- **VCEncReconPara recon**
The parameters for the reconstructed frame.
- **VCEncReconPara reflight0** [2]
The parameters for reference list 0.
- **VCEncReconPara reflight1** [2]
The parameters for reference list 1.
- union {
 VCEncH264Para h264Para
 The parameters for H.264.
 VCEncHevcPara hevcPara
 The parameters for HEVC.
 VCEncAv1Para av1Para
 The parameters for av1.
} **params**

The parameters for HEVC (H.265) or H.264 (AVC).

10.35 VCEncGopConfig

Defines the GOP structure.

Data Fields

- **VCEncGopPicConfig * pGopPicCfg**
A pointer to the array that contains the reference descriptions (*VCEncGopPicConfig*) of all frames in the GOP.
- **u8 size**
The number of *VCEncGopPicConfig* entries in *VCEncGopConfig.pGopPicCfg*.
The value range is from 0 to *MAX_GOP_PIC_CONFIG_NUM*, inclusive.
- **u8 id**
The index of the *VCEncGopPicConfig* entry used by the current frame in *VCEncGopConfig.pGopPicCfg*.
The value range is from 0 to (*VCEncGopConfig.size* - 1), inclusive.
- **u8 id_next**
The index of the *VCEncGopPicConfig* entry used by the next frame in *VCEncGopConfig.pGopPicCfg*.
The value range is from 0 to (*VCEncGopConfig.size* - 1), inclusive.
- **u8 special_size**
The number of *VCEncGopPicSpecialConfig* entries in *VCEncGopConfig.pGopPicSpecialCfg*.
The value range is from 0 to *MAX_GOP_PIC_CONFIG_NUM*, inclusive.
- **i32 delta_poc_to_next**
The difference between the POC of the next frame and the POC of the current frame.
- **VCEncGopPicSpecialConfig * pGopPicSpecialCfg**
A pointer to the array that contains the reference descriptions (*VCEncGopPicSpecialConfig*) of all special frames in the GOP.
- **u8 ltrcnt**
The number of long-term reference (LTR) frames.
The value range is from 0 to *VCENC_MAX_LT_REF_FRAMES*, inclusive.
- **u32 u32LTR_idx [VCENC_MAX_LT_REF_FRAMES]**
The index of each LTR frame.
- **i32 idr_interval**
The interval between two IDR frames, which defines the length of the GOP.
- **i32 intraPeriod**
The interval between two non-IDR intra frames within the GOP.
- **i32 firstPic**
The first frame in the input file to be encoded.
- **i32 lastPic**
The last frame in the input file to be encoded.

- **i32 outputRateNumer**
The numerator of the output frame rate.
- **i32 outputRateDenom**
The denominator of the output frame rate.
- **i32 inputRateNumer**
The numerator of the input frame rate.
- **i32 inputRateDenom**
The denominator of the input frame rate.
- **i32 gopLowdelay**
Whether the GOP configuration is low-delay configuration.
- **i32 interlacedFrame**
Whether input formats are interlaced.
The interlaced frame input format is valid only for HEVC (H.265). For details, see Section *Interlaced Coding for HEVC*.
- **u8 gopCfgOffset [MAX_GOP_SIZE+1]**
The start index of frames in *VCEncGopConfig.pGopPicCfg* for the GOP structure of each size.
- **u8 gopCfgOffsetLP**
The start index of frames in *VCEncGopConfig.pGopPicCfg* for the GOP structure of leading pictures.
- **VCEncGopPicConfig * pGopPicCfgPass1**
A pointer to the array that contains the reference descriptions (*VCEncGopPicConfig*) of all frames for use of pass-1 encoder.
- **VCEncGopPicConfig * pGopPicCfgPass2**
A pointer to the array that contains the reference descriptions (*VCEncGopPicConfig*) of all frames for use of pass-2 encoder.

10.36 VCEncGopPicConfig

Defines reference description for a frame in a GOP.

Data Fields

- **i32 poc**
The picture order count (POC) of the frame in the GOP.
The value range is from 1 to *gopSize*, inclusive.
- **i32 QpOffset**
The quantization parameter (QP) offset of the frame.
- **double QpFactor**
The QP factor of the frame.

- **i32 temporalId**
The temporal layer ID where the frame locates.
- **VCEncPictureCodingType codingType**
The coding type of the frame.
- **u32 nonReference**
Whether the frame is used for future reference.
0: used for future reference.
1: not used for future reference.
- **u32 numRefPics**
The number of reference frames kept for the current frame.
The value range is from 0 to VCENC_MAX_REF_FRAMES, inclusive.
- **VCEncGopPicRps refPics [VCENC_MAX_REF_FRAMES]**
The reference frame information used to encode the current frame.

10.37 VCEncGopPicRps

Contains reference frame information.

Data Fields

- **i32 ref_pic**
The POC delta of the short-term reference frame relative to the current picture, or the index of LTR.
- **u32 used_by_cur**
Whether the current picture uses the reference frame.
0: does not use.
1: uses.

10.38 VCEncGopPicSpecialConfig

Defines reference description for a special frame in a GOP.

Data Fields

- **u32 poc**
The picture order count (POC) of the frame in the GOP.
The value range is from 1 to `gopSize`, inclusive.
- **i32 QpOffset**
The quantization parameter (QP) offset of the frame.
- **double QpFactor**
The QP factor of the frame.
- **i32 temporalId**
The temporal layer ID where the frame locates.
- **VCEncPictureCodingType codingType**
The coding type of the frame.
- **u32 nonReference**
Whether the frame is used for future reference.
0: used for future reference.
1: not used for future reference.
- **u32 numRefPics**
The number of reference frames kept for the current frame.
The value range is from 0 to `VCENC_MAX_REF_FRAMES`, inclusive.
- **VCEncGopPicRps refPics [VCENC_MAX_REF_FRAMES]**
The reference frame information used to encode the current frame.
- **i32 i32Ltr**
The index of the long-term reference (LTR) frame.
0: The current frame is a frame uses LTR.
A value in the range from 1 to `VCENC_MAX_LT_REF_FRAMES`: The current frame is an LTR frame.
- **i32 i32Offset**
The POC of the first special frame relative to the IDR frame.
The value must be less than `VCEncGopPicSpecialConfig.i32Interval`.
- **i32 i32Interval**
The interval between two pictures to which the special frame configurations are applied.
- **i32 i32short_change**
Whether only short-term reference frames are involved in the structure configurations.
0: Long-term reference frames are involved.
1: Only short-term reference frames are involved.
The field value is derived from `VCEncGopPicSpecialConfig.i32Ltr` and `VCEncGopPicSpecialConfig.refPics`, with no need for manual configuration.

10.39 VCEncH264Para

Contains H.264 (AVC) parameters for libva.

Data Fields

- unsigned int **idr_pic_flag**
Whether the picture is an IDR picture.
0: The picture is not an IDR picture.
1: The picture is an IDR picture.
- unsigned int **reference_pic_flag**
Whether the picture is a reference picture.
0: The picture is not a reference picture.
1: The picture is a reference picture.
- unsigned char **slice_type**
The slice type.
Valid value ranges include 0 to 2 and 5 to 7.
 - Values 0 and 5 indicate P slice.
 - Values 1 and 6 indicate B slice.
 - Values 2 and 7 indicate I slice.*Switching slices are not supported.*
For more details about the values, see ITU-T Rec. H.264.
- unsigned short **idr_pic_id**
The ID of an IDR picture.
This field is equivalent to the `idr_pic_id` syntax element in ITU-T Rec. H.264.
- unsigned char **num_ref_idx_l0_active_minus1**
The maximum reference index on reference list 0.
The valid value range is from 0 to 31, inclusive.
- unsigned char **num_ref_idx_l1_active_minus1**
The maximum reference index on reference list 1.
The valid value range is from 0 to 31, inclusive.
- unsigned short **pic_order_cnt_lsb**
The POC modulo `MaxPicOrderCntLsb`,
where `MaxPicOrderCntLsb` equals $\text{power}(2, \text{VCEncConfig.log2MaxPicOrderCntLsb})$.
This field is valid only if the value of `VCEncConfig.picOrderCntType` is 0.

10.40 VCEncHeifConfig

Contains the HEIF configurations parsed from an input configuration file.

Data Fields

- **VCEncHeifDecInfoType DecInfoType**
The media data type.

- **char output_file [256]**
The path to the HEIF output file.
- **int img_width**
The width of the encoded image, or the grid image when grid mode was enabled.
- **int img_height**
The height of the encoded image, or the grid image when grid mode was enabled.
- **u32 total_numbers**
The total number of images saved to the HEIF output file.
- **u32 primary_number**
The sequence number of the primary image item.
- **u32 hidden_number**
The sequence number of the hidden image item.
- **bool thumb_enable**
Whether to save thumbnail images to the HEIF output file.
false: do not save.
true: save.
- **bool crop_enable**
Whether to crop images before saving them to the HEIF output file.
false: do not crop.
true: crop.
- **VCEncHeifCropCfg crop_config**
The cropping configurations.
- **bool mirror_enable**
Whether to mirror images before saving them to the HEIF output file.
false: do not mirror.
true: mirror.
- **VCEncHeifMirrorType mirror_type**
The mirroring type.
- **bool rotate_enable**
Whether to rotate images before saving them to the HEIF output file.
false: do not rotate.
true: rotate.
- **int angle**
The angle of rotation, in degrees.
Valid values include 90, 180, and 270.
- **VCENCHeifGridModeType grid_mode**
The grid mode.
- **VCEncHeifGridCfg grid_config**
The image grid configurations.
- **VCEncHeifOSDConfig OSD_config**

The OSD configurations.

- **bool overlay_enable**

Whether to overlay images before saving them to the HEIF output file.

false: do not overlay.

true: overlay.

- **VCEncHeifOverlayCfg overlay_config**

The overlaying configurations.

- **bool add_group**

Whether to add a group to the HEIF output file.

false: do not add.

true: add.

- **bool auxiliary_enable**

Whether to use an auxiliary image for complementing the master image.

false: do not use.

true: use.

- **VCEncHeifAuxiliaryType auxType**

The type of the auxiliary image.

- **bool editList_enable**

Whether to add an edit list to each sequence.

false: do not add.

true: add.

- **char slave_file [256]**

The path to the file that stores the HEVC (H.265) stream of the auxiliary image.

10.41 VCEncHeifCropCfg

Contains image cropping configurations.

Data Fields

- **uint32_t cropped_width**

The width of the cropped image.

- **uint32_t cropped_height**

The height of the cropped image.

- **uint32_t horizontal_offset**

The X offset of the clean aperture center minus (width-1)/2.

- **uint32_t vertical_offset**

The Y offset of the clean aperture center minus (height-1)/2.

10.42 VCEncHeifEditListCfg

Configuration of edit list for sequence.

Data Fields

- uint32_t **repeatTimes**

10.43 VCEncHeifGridCfg

Contains configurations of the output image derived from image grids.

Data Fields

- uint32_t **rows**
*The number of rows in the unit of grid.
The field value is greater than or equal to 1.*
- uint32_t **cols**
*The number of columns in the unit of grid.
The field value is greater than or equal to 1.*
- uint32_t **image_width**
The width of the output image which is composed of grids, in pixels.
- uint32_t **image_height**
The height of the output image which is composed of grids, in pixels.

10.44 VCEncHeifInst

Defines an HEIF instance.

Data Fields

- **VCEncHeifConfig * heif_config**
The HEIF configurations parsed from the input configuration file.
- **VCEncHeifWriterInst writer_inst**
The HEIF writer handle.
- **u32 input_numbers**
The number of images input to the encoder.
- **u32 saved_numbers**
The number of image items saved to the HEIF output file.

10.45 VCEncHeifOSDConfig

Saves the OSD configurations of the undivied image, for display OSD at grid mode 2/3.

Data Fields

- **u32 overlayEnables**
Any of the OSD regions is enabled.
- **VCEncOverlayArea overlayArea [MAX_OVERLAY_NUM]**
The OSD regions.
- **u32 osdMapEnable**
- **char * osdMapInput**
- **u32 osdMapStride**
- **u32 osdMapBlockSize**
- **u32 osdMapAlpha [MAX_OSDMAP_COLOR_NUM]**
- **u32 osdMapY [MAX_OSDMAP_COLOR_NUM]**
- **u32 osdMapU [MAX_OSDMAP_COLOR_NUM]**
- **u32 osdMapV [MAX_OSDMAP_COLOR_NUM]**

10.46 VCEncHeifOverlayCfg

Contains image overlay configurations.

Data Fields

- uint32_t **overlay_images**
The total number of input images to overlay.
- uint32_t **output_width**
The width of the output overlaid image.
- uint32_t **output_height**
The height of the output overlaid image.
- HeifOffset * **offsets**
*The offset of each input image.
The number of specified offsets must equal the value of `VCEncHeifOverlayCfg.overlay_images`.*

10.47 VCEncHeifSampleCfg

Sample information of sequence.

Data Fields

- uint64_t **duration**
Duration of sample in ImageSequence timeBase units.

10.48 VCEncHeifStreamData

Contains information of stream data for HEIF.

Data Fields

- uint8_t * **stream_data**
The address of the stream data buffer.
- uint64_t **stream_len**
The length of the stream data, in bytes.

10.49 VCEncHvcPara

Contains HEVC (H.265) parameters for libva.

Data Fields

- unsigned int **idr_pic_flag**
Whether the picture is an IDR picture.
0: The picture is not an IDR picture.
1: The picture is an IDR picture.
- unsigned int **reference_pic_flag**
Whether the picture is a reference picture.
0: The picture is not a reference picture.
1: The picture is a reference picture.
- unsigned char **slice_type**
The slice type.
0: P slice.
1: B slice.
2: I slice.
Switching slices are not supported.
For more details about the values, see ITU-T Rec. H.265.
- unsigned short **idr_pic_id**
The ID of an IDR picture.
This field is equivalent to the `idr_pic_id` syntax element in ITU-T Rec. H.264.
- unsigned char **num_ref_idx_l0_active_minus1**
The maximum reference index on reference list 0.
The valid value range is from 0 to 31, inclusive.
- unsigned char **num_ref_idx_l1_active_minus1**
The maximum reference index on reference list 1.
The valid value range is from 0 to 31, inclusive.
- unsigned short **pic_order_cnt_lsb**
The POC modulo `MaxPicOrderCntLsb`,
where `MaxPicOrderCntLsb` equals `power(2, VCEncConfig.log2MaxPicOrderCntLsb)`.
This field is valid only if the value of `VCEncConfig.picOrderCntType` is 0.
- unsigned char **rps_neg_pic_num**
The number of reference pictures that have negative POC delta compared with the current frame.
- unsigned char **rps_pos_pic_num**
The number of reference pictures that have positive POC delta compared with the current frame.
- signed int **rps_delta_poc** [VCENC_MAX_REF_FRAMES]
The POC delta of reference frames, in ascending order.
- unsigned char **rps_used_by_cur** [VCENC_MAX_REF_FRAMES]
The flags that indicate whether each reference frame is used during the encoding of the current frame.

10.50 VCEncIn

Defines the input to the video encoder.

Data Fields

- **ptr_t busLuma**

The bus address of an input picture buffer of the video encoder.

This field specifies the bus address of the Y-plane buffer if the picture data is stored in a planar or semi-planar YUV format, or the bus address of the buffer that store the complete picture data if the picture data is stored in an interleaved YUV or RGB format.

- **ptr_t busChromaU**

The bus address of an input picture buffer of the video encoder.

This field specifies the bus address of the U-plane buffer if the picture data is stored in a planar YUV format, or the bus address of the UV-plane buffer if the picture data is stored in a semi-planar YUV format.

This field is invalid if the input picture data is stored in an interleaved YUV or RGB format.

- **ptr_t busChromaV**

The bus address of an input picture buffer of the video encoder.

This field specifies the bus address of the V-plane buffer if the picture data is stored in a planar YUV format.

This field is invalid if the input picture data is stored in a semi-planar YUV format or an interleaved YUV or RGB format.

- **ptr_t busLumaOrig**

A bus address of the externally down-scaled picture input to the pass-1 encoder.

This field specifies the bus address of the Y-plane buffer if the picture data is stored in a planar or semi-planar YUV format, or the bus address of the buffer that store the complete picture data if the picture data is stored in an interleaved YUV or RGB format.

- **ptr_t busChromaUOrig**

A bus address of the externally down-scaled picture input to the pass-1 encoder.

This field specifies the bus address of the U-plane buffer if the picture data is stored in a planar YUV format, or the bus address of the UV-plane buffer if the picture data is stored in a semi-planar YUV format.

This field is invalid if the input picture data is stored in an interleaved YUV or RGB format.

- **ptr_t busChromaVOrig**

A bus address of the externally down-scaled picture input to the pass-1 encoder.

This field specifies the bus address of the V-plane buffer if the picture data is stored in a planar YUV format.

This field is invalid if the input picture data is stored in a semi-planar YUV format or an interleaved YUV or RGB format.

- **u32 timeIncrement**

The duration of the previous picture, in the unit of 1/FPS.

The value is 0 for the first picture, and typically equal to `frameRateDenom` for the subsequent pictures.

- **u32 vui_timing_info_enable**

Whether to write VUI timing information in SPS.

0: do not write.

1: write.

- **u32 * pOutBuf [MAX_STRM_BUF_NUM]**

The virtual address of each output stream buffer.

- **ptr_t busOutBuf [MAX_STRM_BUF_NUM]**

The bus address of each output stream buffer.

- **u32 outBufSize [MAX_STRM_BUF_NUM]**
The size of each output stream buffer, in bytes.
- **void * cur_out_buffer [MAX_STRM_BUF_NUM]**
The addresses of the buffers used in case of output buffer overflow.
The application needs to check these buffers, and replace the application layer maintained output buffers with these buffers if valid output data is detected in these buffers.
- **VCEncPictureCodingType codingType**
The proposed picture coding type.
Valid values include **VCENC_INTRA_FRAME**, **VCENC_PREDICTED_FRAME**, and **VCENC_BIDIR_PREDICTED_FRAME**.
- **ptr_t busLumaStab**
The bus address of the Y-plane buffer of the next picture for stabilization. For details, see Section **Video Stabilization**.
- **i32 poc**
The picture order count (POC).
- **VCEncGopConfig gopConfig**
The configurations of the current GOP structure.
- **i32 gopSize**
The size of the current GOP structure.
- **i32 gopPicIdx**
The encoded order count of the current picture within the GOP structure.
The value range is from 0 to (gopSize - 1), inclusive.
- **i8 * pRoiMapDelta**
The virtual address of the ROI map buffer.
- **u32 roiMapDeltaSize**
The ROI map size.
- **ptr_t roiMapDeltaQpAddr**
The bus address of the ROI map buffer.
- **ptr_t RoimapCuCtrlAddr**
The bus address of the CU control map buffer.
- **ptr_t RoimapCuCtrlIndexAddr**
Obsoleted.
- **ptr_t SliceInfo_Base**
The base bus address of the slice information for low-latency encoding.
- **ptr_t extSRAMLumBwdBase**
The base bus address of external SRAM for luma backward reference.
This field is valid only if external SRAM is enabled to save motion estimation (ME) bandwidth.
- **ptr_t extSRAMLumFwdBase**
The base bus address of external SRAM for luma forward reference.
This field is valid only if external SRAM is enabled to save ME bandwidth.

- **ptr_t extSRAMChrBwdBase**
*The base bus address of external SRAM for chroma backward reference.
This field is valid only if external SRAM is enabled to save ME bandwidth.*
- **ptr_t extSRAMChrFwdBase**
*The base bus address of external SRAM for chroma forward reference.
This field is valid only if external SRAM is enabled to save ME bandwidth.*
- **ptr_t dec400LumTableBase**
*The base bus address of the compression table.
This field specifies the bus address of the Y-plane buffer if the input picture data is stored in a planar or semi-planar YUV format, or the bus address of the buffer that stores the complete picture data if the input picture data is stored in an interleaved YUV or RGB format.
This field is valid only if DEC400 is used to decompress the input picture.*
- **ptr_t dec400CbTableBase**
*The base bus address of the compression table.
This field specifies the bus address of the U-plane buffer if the input picture data is stored in a planar YUV format, or the bus address of the UV-plane buffer if the input picture data is stored in a semi-planar YUV format.
This field is invalid if the input picture data is stored in an interleaved YUV or RGB format.
This field is valid only if DEC400 is used to decompress the input picture.*
- **ptr_t dec400CrTableBase**
*The base bus address of the compression table.
This field specifies the bus address of the V-plane buffer if the input picture data is stored in a planar YUV format.
This field is invalid if the input picture data is stored in a semi-planar YUV format or an interleaved YUV or RGB format.
This field is valid only if DEC400 is used to decompress the input picture.*
- **i32 picture_gopIdx**
The encoded picture GOP count.
- **i32 picture_cnt**
The encoded picture count of the current picture.
- **i32 picture_cnt_next**
The encoded picture count of the next picture.
- **i32 last_idr_picture_cnt**
The encoded picture count of the last IDR frame.
- **u32 lineBufWrCnt**
The number of MB lines that have been fed to the input MB line buffer.
- **u32 initSegNum**
The number of input data segments stored for SBI.
- **u32 hashType**
*The method to calculate the hash value of the output stream.
0: hash value calculation disabled.
1: standard CRC32.
2: 32-bit checksum.*
- **u32 sceneChange**

Whether the current frame is a scene change frame. For details, see Section [External Scene Change Indication](#).

- **i16 gmv [2][2]**

The global motion vectors (MVs).

- **u32 blsIDR**

Whether to encode the current picture as an IDR frame, after which a new GOP is followed.

0: do not encode as an IDR frame.

1: encode as an IDR frame.

- **u32 blsIntraOnly**

Whether to encode the current picture as a non-IDR intra frame.

0: do not encode as a non-IDR intra frame.

1: encode as a non-IDR intra frame.

- **u32 blsPeriodUsingLTR**

Whether to periodically use long-term reference (LTR).

0: do not periodically use.

1: periodically use.

- **u32 blsPeriodUpdateLTR**

Whether to periodically update LTR frames.

0: do not periodically update.

1: periodically update.

- **VCEncGopPicConfig gopCurrPicConfig**

The reference descriptions of the current picture.

- **VCEncGopPicConfig gopNextPicConfig**

The reference descriptions of the next picture.

- **i32 long_term_ref_pic [VCENC_MAX_LT_REF_FRAMES]**

The POC of each LTR frame.

A value greater than or equal to 0 indicates the POC of an LTR frame.

A value of -1 indicates that the frame is not an LTR frame.

- **u32 bLTR_used_by_cur [VCENC_MAX_LT_REF_FRAMES]**

Whether the current picture uses each LTR frame.

0: does not use.

1: uses.

- **u32 bLTR_need_update [VCENC_MAX_LT_REF_FRAMES]**

Whether to update the POC of each LTR frame after encoding the current frame.

0: do not update.

1: update.

- **i8 i8SpecialRpsIdx**

The special RPS index used by the current picture.

The field value is greater than or equal to -1.

If the field value is -1, it indicates that the current picture does not use special RPS index.

- **i8 i8SpecialRpsIdx_next**

(For H.264 only) The special RPS index used by the next picture.

The field value is greater than or equal to -1.

If the field value is -1, it indicates that the next picture does not use special RPS index.

- **u8 u8IdxEncodedAsLTR**

The LTR index of the current frame if it is encoded as an LTR frame.

The value range is from 0 to `VCENC_MAX_LT_REF_FRAMES`, inclusive.

If the field value is 0, it indicates that the current frame is not encoded as an LTR frame.

- **u32 resendVPS**

Whether to re-send the video parameter set (VPS). For details, see Section [VPS/SPS/PPS Re-Sending](#).

0: do not re-send.

1: re-send.

- **u32 resendSPS**

Whether to re-send the sequence parameter set (SPS). For details, see Section [VPS/SPS/PPS Re-Sending](#).

0: do not re-send.

1: re-send.

- **u32 resendPPS**

Whether to re-send the picture parameter set (PPS). For details, see Section [VPS/SPS/PPS Re-Sending](#).

0: do not re-send.

1: re-send.

- **u32 sendAUD**

Whether to send the access unit delimiter (AUD).

0: do not send.

1: send.

- **u32 bSkipFrame**

Whether to code all MBs or CTBs in the current frame in skip mode. For details, see Section [Skipped-Frame Insertion](#).

0: do not code in skip mode.

1: code in skip mode.

- **u32 flexRefsEnable**

Whether to enable external reference frame selection. For details, see Section [Reference Customization](#).

0: disable.

1: enable.

- **u32 dec400Enable**

[0,7] Whether to enable DEC400.

0: global-bypass or indicate dec400 doesn't exit.

1: stream-bypass.

2: enable.

- **u32 axiFEEEnable**

Whether to enable AXI FE.

0: disable AXI FE.

1: enable AXI FE in normal mode.

2: enable AXI FE in bypass mode.

3: enable AXI FE in security mode.

- **u32 apbFTEEnable**

Whether to enable APB filters.

0: disable.

1: enable.

- **ptr_t overlayInputYAddr [MAX_OVERLAY_NUM]**
*The base bus address of the input buffer for each OSD channel.
This field specifies the bus address of the Y-plane buffer if the picture data is stored in a planar or semi-planar YUV format, or the bus address of the buffer that store the complete picture data if the picture data is stored in an interleaved YUV or RGB format.*
- **ptr_t overlayInputUAddr [MAX_OVERLAY_NUM]**
*The base bus address of the input buffer for each OSD channel.
This field specifies the bus address of the U-plane buffer if the picture data is stored in a planar YUV format, or the bus address of the UV-plane buffer if the picture data is stored in a semi-planar YUV format.
This field is invalid if the input picture data is stored in an interleaved YUV or RGB format.*
- **ptr_t overlayInputVAddr [MAX_OVERLAY_NUM]**
*The base bus address of the input buffer for each OSD channel.
This field specifies the bus address of the V-plane buffer if the picture data is stored in a planar YUV format.
This field is invalid if the input picture data is stored in a semi-planar YUV format or an interleaved YUV or RGB format.*
- **u32 overlayEnable [MAX_OVERLAY_NUM]**
*Whether to enable each OSD channel.
0: disable.
1: enable.*
- **u32 osdDec400Enable [MAX_OVERLAY_NUM]**
*Whether the OSD input picture is DEC400 compressed.
0: not compressed.
1: bypassed.
2: compressed.*
- **ptr_t osdDec400TableBase [MAX_OVERLAY_NUM][3]**
The base bus address of the compression table for OSD input pictures.
- **u32 osdMapEnable**
*(Under development) Whether to enable the OSD map.
0: disable.
1: enable.*
- **ptr_t osdMapInputAddr**
(Under development) The base bus address of the OSD map buffer.
- **u32 externalSEICount**
*The number of **ExternalSEI** entries in **VCEncIn.pExternalSEI**. For details, see Section **External SEI Information**.*
- **ExternalSEI * pExternalSEI**
*The external SEI information. For details, see Section **External SEI Information**.*
- **VCEncInTileExtra * tileExtra**
*The parameters for the second tile to the last tile in the input picture in multi-tile scenarios.
If **VCEncIn.busLuma** is not specified for the second tile, this field is automatically filled for the second tile to the last tile according to the tile distribution.*
- **void * internal**

internal context for the input, API user should not use it.

- void * **extension**
- u32 **dec400TSHdrEnable**

Whether to process dec400 tile status header buffer.

- u32 **fcEnable** [3]

Enable the fast clear function of dec400.

- u32 **clearColorLow** [3]

The clear color in 16 bpp or 32 bpp for YUV channel.

10.50.1 Field Documentation

10.50.1.1 dec400Enable

u32 dec400Enable

[8,15] DEC400 tileSize.

- 0: default.
- 1: 128byte.
- 2: 512byte.
- 3: 256byte.

[16,23] DEC400 tileMode.

- 0: default.
- 1: TILE_8x8_x.
- 2: TILE_8x8_y.
- 3: TILE_16x4.
- 4: TILE_8x4.
- 5: TILE_4x8.
- 6: RASTER_16x4.
- 7: TILE_64x4.
- 8: TILE_32x4.
- 9: RASTER_256x1.
- 10: RASTER_128x1.
- 11: RASTER_64x1.
- 12: TILE_16x8.

- 13: RASTER_32x4.
- 14: RASTER_32x1.
- 15: RASTER_16x1.
- 16: TILE_8x4_s.
- 17: TILE_16x4_s.
- 18: TILE_32x4_s.
- 19: TILE_32x8.

[25,27] DEC400 RGB format.

- 0: default.
- 1: RGB8.
- 2: RGB10.
- 3: RGB4.
- 4: RGB1555.
- 5: RGB565.

[28,28] DEC400 RGB format X or A.

- 0: ARGB.
- 1: XRGB.

[29,31] DEC400 data alignment.

- 0: default.
- 1: 32byte.
- 2: 64byte.
- 3: 1byte.
- 4: 16byte.

10.50.1.2 extension

void* extension

extension parameters

10.51 VCEncInTileExtra

Defines input buffers and output stream buffers for the second tile to the last tile in multi-tile scenarios.

Data Fields

- **ptr_t busLuma**

The bus address of an input picture buffer of the video encoder.

This field specifies the bus address of the Y-plane buffer if the picture data is stored in a planar or semi-planar YUV format, or the bus address of the buffer that stores the complete picture data if the picture data is stored in an interleaved YUV or RGB format.

- **ptr_t busChromaU**

The bus address of an input picture buffer of the video encoder.

This field specifies the bus address of the U-plane buffer if the picture data is stored in a planar YUV format, or the bus address of the UV-plane buffer if the picture data is stored in a semi-planar YUV format.

This field is invalid if the input picture data is stored in an interleaved YUV or RGB format.

- **ptr_t busChromaV**

The bus address of an input picture buffer of the video encoder.

This field specifies the bus address of the V-plane buffer if the picture data is stored in a planar YUV format.

This field is invalid if the input picture data is stored in a semi-planar YUV format or an interleaved YUV or RGB format.

- **u32 * pOutBuf [MAX_STRM_BUF_NUM]**

The virtual address of each output stream buffer.

- **ptr_t busOutBuf [MAX_STRM_BUF_NUM]**

The bus address of each output stream buffer.

- **u32 outBufSize [MAX_STRM_BUF_NUM]**

The size of each output stream buffer, in bytes.

- **void * cur_out_buffer [MAX_STRM_BUF_NUM]**

A buffer identifier to re-encode when the output buffer overflows.

The application needs to check this buffer and replace the output buffer maintained by the application layer. This function is not used internally by the video encoder.

10.52 VCEncLogSetting

Contains the log settings read from the environment or specified during initialization.

Data Fields

- **VCEncLogOutput out_dir**

The log output mode.

- **VCEncLogLevel out_level**

The lowest log level to be output.

Logs of the specified level and higher are output.

- unsigned int **k_trace_map**

A bitmap where each bit indicates whether to output each type of trace logs.

0: do not output.

1: output.

*For the mappings between bit indexes and trace log types, see Section **VCEncLogTraceType**.*

*The values are parsed from the `trace_map` parameter in **VCEncLogInit()**.*

- unsigned int **k_check_map**

(Reserved) A bitmap where each bit indicates whether to output each type of check logs.

0: do not output.

1: output.

*For the mappings between bit indexes and check log types, see Section **VCEncLogCheckType**.*

*The values are parsed from the `check_map` parameter in **VCEncLogInit()**.*

10.53 VCEncMode

Encoding mode controls the flow of encoding.

Data Fields

- **u32 batch_flag: 1**

enable or disable batch mod for multi-frame aggregation

0 = disable batch mode

1 = enable batch mode

- **u32 bSendSPSonFPSAdjust: 1**

send new SPS on FPS change or not

0 = not send SPS

1 = send SPS

- **u32 bInsertIDRonFPSAdjust: 1**

insert IDR on FPS change or not

0 = not insert IDR

1 = insert IDR

10.54 VCEncMv

Contains the motion information of a coding unit (CU) in inter mode.

Data Fields

- **u8 refIdx**

The index of the reference list in use.

0: reference list 0.

1: reference list 1.

- **i16 mvX**

The horizontal motion in the unit of 1/4 pixel.

- **i16 mvY**

The vertical motion in the unit of 1/4 pixel.

10.55 VCEncOut

Defines the output from the video encoder.

Data Fields

- **VCEncPictureCodingType codingType**

The realized picture coding type.

Valid values include `VCENC_INTRA_FRAME`, `VCENC_PREDICTED_FRAME`, `VCENC_BIDIR_PREDICTED_FRAME`, and `VCENC_NOTCODED_FRAME`.

- **u32 streamSize**

The size of the output stream in bytes.

- **u32 * pNaluSizeBuf**

The output buffer that stores the size of each NAL unit in bytes.

`pNaluSizeBuf[n]` indicates the size of NAL unit `n`.

A zero value is written after the last NAL unit.

- **u32 numNalus**

The number of NAL units.

- **VCEncOutFrameInfo * frameInfo**

The frame information of a picture.

- **u16 * lumaInfo**

The virtual address of luma information .

- **u32 av1Vp9FrameNotShow**

(For AV1/VP9 only) A 32-bit flag for IVF timestamp generation.

Bit `n` indicates whether frame `n` (an OBU frame for AV1 or raw frame for VP9) in the output buffer is a not-show frame.

0: show frame.

1: not-show frame.

- **u32 maxSliceStreamSize**
The maximum size of output slices in bytes.
- **ptr_t busScaledLuma**
The bus address of the buffer of the scaled encoder picture.
- **u8 * scaledPicture**
The virtual address of the scaled encoder picture.
- **VCEncCuOutData cuOutData**
A pointer to the CU/MB statistics that is output by the hardware.
- **u16 * ctbBitsData**
A pointer to the CTB encoded bit statistics that are output by the hardware.
- **u8 boolCurlsLongTermRef**
Whether the current frame is used for long-term reference (LTR) by a subsequent frame.
true: used for LTR by a subsequent frame.
false: not used for LTR by a subsequent frame.
- **double ssim [3]**
The SSIM values calculated by the hardware for Y, U, and V.
The field is valid only if the value of *EWLHwConfig_t.ssimSupport* is 1.
- **double psnr [3]**
The PSNR values calculated by the hardware for Y, U, and V.
The field is valid only if the value of *EWLHwConfig_t.psnrSupport* is 1.
- **double psnr_y**
An approximate PSNR value for Y, which excludes the eight bottom lines of each CTB.
The hardware calculates SSE, and the software calculates log10f.
- **double psnr_y_predeb**
The PSNR value for Y before de-blocking.
The hardware calculates SSE, and the software calculates log10f.
- **double psnr_cb_predeb**
The PSNR value for U before de-blocking.
The hardware calculates SSE, and the software calculates log10f.
- **double psnr_cr_predeb**
The PSNR value for V before de-blocking.
The hardware calculates SSE, and the software calculates log10f.
- **u32 sliceHeaderSize**
The size of the output slice header in multi-tile scenarios.
- **VCEncCuStatis cuStatis**
The CU statistics for adaptive GOP structure size decision.
- **u32 nextSceneChanged**
The scene change detection result for the next frame, output by the video stabilization module.
For details, see Section *Scene Change Detection*.
- **i32 picture_cnt**

The encoded picture count.

- **u32 PreDataSize**

The number of bytes in the stream before hardware encoding, excluding the initial stream header data.

- **u32 PreNaluNum**

The number of NAL units in the stream before hardware encoding, excluding the initial stream header data.

- **u32 PostDataSize**

The number of bytes in the stream after hardware encoding.

- **i32 poc**

The picture order count (POC).

- **VCEncConsumedAddr consumedAddr**

The bus addresses of the consumed buffers that should be released after this frame is encoded.

- **VCEncOutTileExtra * tileExtra**

The parameters for the second tile column to the last tile column in multi-tile scenarios.

10.56 VCEncOutFrameInfo

Contains the frame information of a picture that is output by the hardware.

Data Fields

- struct **hevcFramInfo** **hevcInfo**

The frame information for HEVC (H.265) encoding.

- struct **h264FramInfo** **h264Info**

The frame information for H.264 (AVC) encoding.

10.57 VCEncOutTileExtra

Contains the output parameters for the second tile column to the last tile column in multi-tile scenarios.

Data Fields

- **u32 streamSize**
The size of the output stream in bytes.
- **u32 * pNaluSizeBuf**
*The output buffer that stores the size of each NAL unit in bytes.
`pNaluSizeBuf[n]` indicates the size of NAL unit `n`.
A zero value is written after the last NAL unit.*
- **u32 numNalus**
The number of NAL units.
- **VCEncCuOutData cuOutData**
A pointer to the CU/MB statistics that is output by the hardware.

10.58 VCEncOverlayArea

Defines an OSD region. For details, see Section *On-the-Fly OSD Mixer*.

Data Fields

- **u32 enable**
*Whether to enable the OSD region.
0: disable.
1: enable.*
- **u32 format**
*The input format of the OSD region.
0: ARGB8888.
1: NV12.
2: bitmap.*
- **u32 alpha**
*The global alpha value for the OSD region.
This field is invalid for ARGB8888.*
- **u32 xoffset**
The horizontal offset, in pixels, of the top-left corner of the OSD region relative to the encoder picture.
- **u32 cropXoffset**
The horizontal offset, in pixels, of the top-left corner of the crop area in the OSD input relative to the OSD input picture.
- **u32 yoffset**
The vertical offset, in pixels, of the top-left corner of the OSD region relative to the encoder picture.
- **u32 cropYoffset**

The vertical offset, in pixels, of the top-left corner of the crop area in the OSD input relative to the OSD input picture.

- **u32 width**

The width of the OSD input.

- **u32 cropWidth**

The width of the crop area in the OSD input, which is the width of the final OSD region.

- **u32 scaleWidth**

(Reserved) The width of the upscaled OSD input.

This field is valid only for specific custom versions.

- **u32 height**

The height of the OSD input.

- **u32 cropHeight**

The width of the crop area in the OSD input, which is the height of the final OSD region.

- **u32 scaleHeight**

(Reserved) The height of the upscaled OSD input.

This field is valid only for specific custom versions.

- **u32 Ystride**

The Y stride of the OSD input, in bytes.

The OSD region data is also stored with this specified stride.

- **u32 UVstride**

The UV stride of the OSD input, in bytes.

The OSD region data is also stored with this specified stride.

- **u32 bitmapY**

The global Y value for the bitmap format.

- **u32 bitmapU**

The global U value for the bitmap format.

- **u32 bitmapV**

The global V value for the bitmap format.

- **u32 superTile**

Whether the OSD input data is organized in supertile mode.

0: non-supertile mode.

1: X-major supertile mode.

2: Y-major supertile mode.

This field is valid only for the ARGB8888 format.

10.59 VCEncPictureArea

Defines a rectangular area in a picture.

Data Fields

- **u32 enable**

Whether to enable the area.

0: disable.

1: enable.

- **u32 top**

The top MB or CTB row inside the area.

The value range is from 0 to (heightMbs - 1), inclusive, where heightMbs indicates the picture height in the unit of MB or CTB.

- **u32 left**

The leftmost MB or CTB column inside the area.

The value range is from 0 to (widthMbs - 1), inclusive, where widthMbs indicates the picture width in the unit of MB or CTB.

- **u32 bottom**

The bottom MB or CTB row inside the area.

The value range is from `VCEncPictureArea.top` to (heightMbs - 1), inclusive, where heightMbs indicates the picture height in the unit of MB or CTB.

- **u32 right**

The rightmost MB or CTB column inside the area.

The value range is from `VCEncPictureArea.left` to (widthMbs - 1), inclusive, where widthMbs indicates the picture width in the unit of MB or CTB.

10.60 VCEncPPSCfg

Contains user-provided picture parameter set (PPS) parameters.

Data Fields

- **i32 chroma_qp_offset**

The chroma QP offset.

- **i32 tc_Offset**

The de-blocking parameter `tc_offset`.

- **i32 beta_Offset**

The de-blocking parameter `beta_offset`.

10.61 VCEncPreProcessingCfg

Contains pre-processing configurations.

Data Fields

- **u32 origWidth**
The width of the input picture, in pixels.
- **u32 origHeight**
The height of the input picture, in pixels.
- **u32 xOffset**
*The horizontal offset, in pixels, of the top-left corner of the encoded picture relative to the input picture.
The value must be an even integer.*
- **u32 yOffset**
*The vertical offset, in pixels, of the top-left corner of the encoded picture relative to the input picture.
The value must be an even integer.*
- **u32 alignmentWidth**
The aligned width of the input picture, in pixels.
- **u32 alignmentHeight**
The aligned height of the input picture, in pixels.
- **VCEncPictureType inputType**
The color format of the input picture.
- **VCEncPictureRotation rotation**
The rotation type.
- **VCEncPictureMirror mirror**
The mirroring mode.
- **u32 videoStabilization**
*Whether to enable video stabilization.
0: disable.
1: enable.*
- **VCEncColorConversion colorConversion**
The RGB-to-YUV conversion configurations.
- **u32 scaledWidth**
*(Optional) The width of the down-scaled picture.
The value is a multiple of 4, in the range from 16 to `VCEncConfig.width`, inclusive.
If either the value of this field or that of `VCEncPreProcessingCfg.scaledHeight` is 0, down-scaling is disabled.*
- **u32 scaledHeight**
*(Optional) The height of the down-scaled picture.
The value is a multiple of 2, in the range from 96 to `VCEncConfig.height`, inclusive.
If either the value of this field or that of `VCEncPreProcessingCfg.scaledWidth` is 0, down-scaling is disabled.*
- **u32 scaledOutput**
*Whether to output the down-scaled picture to a buffer.
0: do not output.
1: output.*

- **u32 scaledOutputFormat**

The color format of the output down-scaled picture.

0 (default): YUV422 interleaved.

1: YUV420 semi-planar.

- **u32 * virtualAddressScaledBuff**

The virtual address of the down-scaled picture buffer.

- **ptr_t busAddressScaledBuff**

The physical address of the down-scaled picture buffer.

- **u32 sizeScaledBuff**

The size of the down-scaled picture buffer.

The size is greater than or equal to $(VCEncPreProcessingCfg.scaledWidth * VCEncPreProcessingCfg.scaledOutput * 2)$ bytes.

- **u32 inLoopDSRatio**

The down-scaling ratio of the inloop down-scaler in the pass-1 encoder for look-ahead encoding.

0: down-scaling disabled.

1: 1/2 down-scaling ratio.

This field is used only for parameter query through the `VCEncGetPreProcessing()` function.

- **i32 constChromaEn**

Whether to encode the picture with constant U and V values.

0: do not encode with a constant U or V value.

1: encode with constant U and V values.

- **u32 constCb**

The constant U value.

The field value is 128 for 8-bit encoding and 512 for 10-bit encoding.

This field is valid only if the value of `VCEncPreProcessingCfg.constChromaEn` is 1.

- **u32 constCr**

The constant V value.

The field value is 128 for 8-bit encoding and 512 for 10-bit encoding.

This field is valid only if the value of `VCEncPreProcessingCfg.constChromaEn` is 1.

- **u32 input_alignment**

The input alignment of the input picture, in bytes.

- **VCEncOverlayArea overlayArea [MAX_OVERLAY_NUM]**

The OSD regions.

- **u32 mosEnable [MAX_MOSAIC_NUM]**

Whether to enable each mosaic region.

0: disable.

1: enable.

- **u32 mosSizeIndex**

Mosaic size index.

- **u32 mosXoffset [MAX_MOSAIC_NUM]**

The horizontal offset of the top-left corner of each mosaic region.

- **u32 mosYOffset** [MAX_MOSAIC_NUM]
The vertical offset of the top-left corner of each mosaic region.
- **u32 mosWidth** [MAX_MOSAIC_NUM]
The width of each mosaic region.
- **u32 mosHeight** [MAX_MOSAIC_NUM]
The height of each mosaic region.
- **u32 osdMapEnable**
- **char * osdMapInput**
- **u32 osdMapStride**
- **u32 osdMapBlockSize**
- **u32 osdMapAlpha** [MAX_OSDMAP_COLOR_NUM]
- **u32 osdMapY** [MAX_OSDMAP_COLOR_NUM]
- **u32 osdMapU** [MAX_OSDMAP_COLOR_NUM]
- **u32 osdMapV** [MAX_OSDMAP_COLOR_NUM]
- **VCEncPrpTileExtra * tileExtra**
The input pre-processing configurations for the second tile to the last tile in multi-tile scenarios.
- **VCEncPictureScanType scanType**

10.62 VCEncPrpTileExtra

Contains input pre-processing configurations for the second tile to the last tile in multi-tile scenarios.

Data Fields

- **u32 origWidth**
The width of the input picture, in pixels.
- **u32 origHeight**
The height of the input picture, in pixels.
- **u32 xOffset**
The horizontal offset, in pixels, of the top-left corner of the encoded picture relative to the input picture.
- **u32 yOffset**
The vertical offset, in pixels, of the top-left corner of the encoded picture relative to the input picture.

10.63 VCEncRateCtrl

Contains bit rate control parameters.

Data Fields

- **i32 crf**
*The constant rate factor for CRF rate control mode.
The value range is from 0 to 51, inclusive.*
- **u32 pictureRc**
*Whether to enable picture-level rate control to adjust QP between pictures.
0 (default): disable.
1: enable.
Set this field to 0 if a target bit rate is specified.*
- **u32 ctbRc**
*The block-level rate control mode for adjusting QP inside a frame.
0: block-level rate control disabled.
1: subjective rate control only.
2: precise rate control only.
3: mixed subjective and precise rate control.*
- **u32 blockRcSize**
*The block size for block-level rate control.
0: 64x64 pixels.
1: 32x32 pixels.
2: 16x16 pixels.*
- **u32 pictureSkip**
*Whether to allow skipping pictures for bit rate control.
0 (default): disallow.
1: allow.*
- **i32 qpHdr**
*The default or initial picture-level QP value.
- If picture-level rate control is disabled, this QP value is used as the default QP value.
- In other cases, this QP value is used as the initial QP value for the first encoded picture.
When block-level rate control is enabled, this QP value is used as the base QP value. The encoder calculates delta values based on this base QP value.
The value range is from -1 to 51, inclusive. Values 0 to 10 are not recommended.
The default value is -1, which indicates that the encoder calculates the default or initial QP value according to the target bit rate.*
- **u32 qpMinPB**
*The minimum QP for P- and B-pictures.
The value range is from 0 to 51, inclusive. The default value is 0. A value less than 10 is not recommended.*
- **u32 qpMaxPB**

The maximum QP for P- and B-pictures.

The value range is from 0 to 51, inclusive.

- **u32 qpMini**

The minimum QP for I-pictures.

The value range is from 0 to 51, inclusive. The default value is 0. A value less than 10 is not recommended.

- **u32 qpMaxI**

The maximum QP for I-pictures.

The value range is from 0 to 51, inclusive.

- **u32 bitPerSecond**

The target bit rate, in bits per second.

Set this field if picture-level rate control is enabled.

The value must be greater than or equal to 10000. If HRD is enabled, the maximum target bit rate is limited by the stream tier and level selected during encoder initialization. For detailed limitations, see ITU-T Rec. H.265.

- **u32 cpbMaxRate**

The maximum bit rate of the coded picture buffer (CPB), in bits per second.

Set this field if the CBR or VBR mode is used.

- **true_e fillerData**

Whether to enable filler data when HRD is disabled.

false: disable.

true: enable.

- **u32 hrd**

Whether to enable the Hypothetical Reference Decoder (HRD) model to restrict the instantaneous bit rate and the total number of bits per coded picture.

0: disable.

1: enable.

Enabling HRD triggers picture-level rate control to be automatically enabled and imposes tight constraints on the operation of the HRD model.

- **u32 hrdCpbSize**

The size of the coded picture buffer (CPB) in bits.

When HRD is enabled, each encoded frame cannot be larger than the specified size.

By default, the video encoder uses the maximum size allowed by the stream level. Setting this field to 0 restores the default value. A value of $(2 * \text{VCEncRateCtrl.bitPerSecond})$ is recommended.

- **u32 bitrateWindow**

The number of frames within which the rate control algorithm tries to achieve the target bit rate.

The value range is from 1 to 300, inclusive. The default value is `VCEncGopConfig.idr_interval`.

Recommended values vary with video use cases.

The rate control algorithm uses the GOP length to match the average bit rate of each GOP to the target bit rate.

- **i32 intraQpDelta**

The delta between QP and intra QP values.

The intra QP delta can be used to reduce intra flashing.

The value range is from -12 to 12, inclusive.

- A smaller QP value increases the relative quality of the intra frame.

- A larger QP value lowers the size of encoded intra frames.

This field is invalid if a fixed QP is specified through `VCEncRateCtrl.fixedIntraQp`.

- **u32 fixedIntraQp**

The fixed QP value for all intra frames.

The value range is from 0 to 51, inclusive. If the field value is 0, the encoder automatically calculates the intra QP value.

- **i32 bitVarRangeI**

Obsoleted.

- **i32 bitVarRangeP**

The permitting variance percentage over average bits per P-frame from the target bit rate.

The value range is from 10 to 10000, inclusive. The default value is 10000, which indicates no limitation on the P-frame size.

- **i32 bitVarRangeB**

The permitting variance percentage over average bits per B-frame from the target bit rate.

The value range is from 10 to 10000, inclusive. The default value is 10000, which indicates no limitation on the B-frame size.

- **i32 tolMovingBitRate**

The tolerance percentage of the maximum moving bit rate over the target bit rate, which means that the average bit rate of the monitored frames is limited to $[VCEncRateCtrl.bitPerSecond * (1 + VCEncRateCtrl.tolMovingBitRate/100)]$.

The value range is from 0 to 2000, inclusive. The default value is 100, which indicates the moving bit rate can tolerate maximal 100% over the target bit rate.

- **i32 monitorFrames**

The number of frames to be monitored for calculating the moving bit rate.

The value range is from 10 to 120, inclusive.

- **i32 targetPicSize**

The target size of the picture output by the block-level rate control algorithm.

- **i32 smoothPsnrInGOP**

Obsoleted.

- **u32 u32StaticScenelbitPercent**

The percentage of I-frame bits in static scenes.

- **u32 rcQpDeltaRange**

The maximum absolute delta between block-level and picture-level QP values.

If the value of `EWLHwConfig_t.CtbRcVersion` is less than 1, the option value range is from 0 to 15, inclusive.

If the value of `EWLHwConfig_t.CtbRcVersion` is greater than or equal to 1, the option value range is from 0 to 51, inclusive.

- **u32 rcBaseMBComplexity**

The MB complexity threshold for subjective block-level rate control.

When the complexity equals the threshold, the block-level QP is not adjusted. The QP value increases when the complexity exceeds the threshold, and vice versa.

- **i32 picQpDeltaMin**

The minimum QP delta in picture-level rate control.

The value range is from -10 to -1, inclusive. The default value is -2.

- **i32 picQpDeltaMax**

The maximum QP delta in picture-level rate control.

The value range is from 1 to 10, inclusive. The default value is 3.

- **i32 longTermQpDelta**

The QP delta of frames using long-term reference (LTR).

The value range is from -51 to 51, inclusive. The default value is 0.

- **i32 vbr**

Whether to enable variable bit rate control based on the minimum QP allowed.

0: disable.

1: enable.

- **rcMode_e rcMode**

The rate control mode.

By default, the CVBR mode is used.

- **float tolCtbRcInter**

The tolerance of block-level rate control for inter frames, which defaults to -1.0.

The rate control algorithm tries to limit inter frame bits within range from $\text{targetPicSize}/(1 + \text{tolCtbRcInter})$ to $\text{targetPicSize} * (1 + \text{tolCtbRcInter})$.

A negative `tolCtbRcInter` value indicates no bit rate limit for inter frames in block-level rate control.

- **float tolCtbRcIntra**

The tolerance of block-level rate control for intra frames, which defaults to -1.0.

The rate control algorithm tries to limit inter frame bits within range from $\text{targetPicSize}/(1 + \text{tolCtbRcIntra})$ to $\text{targetPicSize} * (1 + \text{tolCtbRcIntra})$.

A negative `tolCtbRcIntra` value indicates no bit rate limit for intra frames in block-level rate control.

- **i32 tolRcUnderflow**

- **u32 maxIprop**

The ratio of the maximum size of intra frames to inter frames.

- **u32 minIprop**

The ratio of the minimum size of intra frames to inter frames.

- **i32 changePos**

ratio of bitrate to the maximum bitrate at initial of the qp adjustment

- **i32 ctbRcRowQpStep**

The maximum accumulated QP adjustment step per CTB/MB row allowed by block-level rate control.

- **i32 ctbRcRowQpDeltaRange**

max QP delta adjustment based on picQP by Ctb Rate Control

- **u32 ctbRcQpDeltaReverse**

Whether to reverse the QP adjustment direction for subjective block-level rate control.

0: do not reverse.

1: reverse.

- **u32 frameRateNum**

The numerator for calculating the target frame rate.

The value range is from 1 to 1048575, inclusive. The default value is 30.

It is recommended to set this field to the same value as `VCEncConfig.frameRateNum`.

- **u32 frameRateDenom**

The denominator for calculating the target frame rate.

The value range is from 1 to `VCEncRateCtrl.frameRateNum`, inclusive. The default value is 1.

It is recommended to set this field to the same value as `VCEncConfig.frameRateDenom`.

- **i32 frameRateUpdatePicCnt**

The frame count of frame rate update.

Frame rate will be updated at this frame, counting from start.

Set to -1 to disable frame rate update.

The default value is -1.

- **u32 hieQpDeltaEnable**

GOP Hierarchical frame QP delta Enable.

10.63.1 Field Documentation

10.63.1.1 tolCtbRcIntra

`float tolCtbRcIntra`

Tolerance of frame RC controlling underflow bitrate for better quality

10.64 VCEncReconPara

Contains reconstructed frame information.

Data Fields

- **i32 poc**

The picture order count (POC).

- **i32 frame_num**

The frame number.

- **i32 frame_idx**

The LTR index.

This field is valid only if the frame is used as an LTR frame.

- **i32 flags**

The reference type flag.

1: short-term reference.

2: long-term reference.

- **i32 temporalId**

The temporal layer ID.

- **u32 * mc_sync_va**

The virtual address of the multi-core synchronization word.

- **ptr_t mc_sync_pa**

The bus address of the multi-core synchronization word.

- **ptr_t busReconLuma**

The bus address of the Y plane of the reconstructed frame.

- **ptr_t recon_luma_va**

The bus address of the UV plane of the reconstructed frame.

- **ptr_t busReconChromaUV**

- **ptr_t recon_chroma_va**

- **ptr_t reconLuma_4n**

The bus address of the down-sampled reconstructed frame.

- **ptr_t reconLuma_4n_va**

- **ptr_t compressTblReconLuma**

The bus address of the compression table of the Y plane.

- **ptr_t compressTblReconLuma_va**

- **ptr_t compressTblReconChroma**

The bus address of the compression table of the UV plane.

- **ptr_t compressTblReconChroma_va**

- **ptr_t colBufferH264Recon**

The bus address of the column buffer for H.264 (AVC).

- **ptr_t colBufferH264Recon_va**

- **ptr_t framectx_pa**

- **ptr_t framectx_va**

- **ptr_t culInfoMemRecon**

10.65 VCEncSliceReady

Defines the input to the callback function `VCEncSliceReadyCallBackFunc()`. For details, see Section *Multi-Slice Encoding*.

Data Fields

- **u32 slicesReadyPrev**
The number of slices that have been completed by the previous callback.
This field is given because one or more slices can be completed between the callbacks.
- **u32 slicesReady**
The number of slices that have been completed.
- **u32 nalUnitInfoNum**
The number of information NAL units that have been completed, including all kinds of SEI information.
- **u32 nalUnitInfoNumPrev**
The number of information NAL units that have been completed by the previous callback, including all kinds of SEI information.
- **u32 * sliceSizes**
A pointer to the size of each completed slice, in bytes.
- **VCEncStrmBufs streamBufs**
The output stream buffers.
- **void * pAppData**
A pointer to the application data.
- **EWLLinearMem_t * outbufMem [MAX_STRM_BUF_NUM]**
The output memory.
- **u32 PreDataSize**
The number of bytes in the stream before hardware encoding, excluding the initial stream header data.
- **u32 PreNaluNum**
The number of NAL units in the stream before hardware encoding, excluding the initial stream header data.
- **u32 streamBufChain**
Whether to enable one or more stream buffers.
0: one output stream buffer. Only `outbufMem[0]` is valid.
1: two chained output stream buffers. All entries in the `outbufMem` array are valid.

10.66 VCEncStatisticOut

Contains statistics collected during the encoding of a frame.

Data Fields

- **const void * kEwl**
The EWL instance handle.
- **u32 avg_qp_y**
The average luma QP.
- **u32 frame_target_bits**
The target number of bits estimated for the current frame.
- **u32 frame_real_bits**
The actual number of bits generated for the current frame.
- **double psnr_y**
The luma PSNR.
- **u32 header_stream_byte**
The stream size generated by software, in bytes.
- **const void * kOutputbufferMem [MAX_STRM_BUF_NUM]**
The list of stream output buffers.
- **u32 output_buffer_over_flow**
Whether buffer overflow occurs in the encoding of the current frame.
0: does not occur.
1: occurs.
- **u32 scene_change**
Whether scene change occurs in the encoding of the current frame.
0: does not occur.
1: occurs.
- **u32 frame_min_size**
The minimum target picture size used for block-level rate control during encoding of the current frame.
- **u32 frame_max_size**
The maximum target picture size used for block-level rate control during encoding of the current frame.
- **u32 targetQp**
The slice QP used to encode the current frame.
- **u32 POCTobeDisplayAV1**
(For AV1 only) The POC of frame to be displayed and recovered during re-encoding.
- **u32 secure_mode**
Whether security encoding is enabled.
0: disabled.
1: enabled.
- **i32 encodeTimeout**
Whether timeout.
0: does not occur.
1: occurs.

- **u32 regValueAXIWritePending**
The AXI write pending when timeout.
- **u32 regValueAXIReadPending**
The AXI read pending when timeout.
- **u32 regValueAXITotalPending**
The AXI total pending when timeout.
- **u32 frameNum**
The frame number in decoding order.

10.67 VCEncStrmBufs

Contains output stream buffers.

Data Fields

- **u8 * buf [MAX_STRM_BUF_NUM]**
A pointer to the beginning of each output stream buffer.
- **u32 bufLen [MAX_STRM_BUF_NUM]**
The length of each output stream buffer.
- **u32 bufOffset [MAX_STRM_BUF_NUM]**
The offset of each output stream buffer.

10.68 VCEncStrmCtrl

Defines the stream-level encoding control parameters.

Data Fields

- **u32 width**
*The width of a new stream.
The field value must be less than or equal to `VCEncConfig.width`.*
- **u32 height**
*The height of a new stream.
The field value must be less than or equal to `VCEncConfig.height`.*

10.69 VuiColorDescription

Defines the color description in the VUI, which is coded in the sequence parameter set (SPS).

This structure is valid only if the value of `VCEncCodingCtrl.vuiVideoSignalTypePresentFlag` is 1.

Data Fields

- **u8 vuiColorDescripPresentFlag**

Whether to present the color description in the VUI.

0: do not present.

1: present.

- **u8 vuiColorPrimaries**

The index of chromaticity coordinates.

The value range is from 0 to 9, inclusive.

For details, see Table E.3 in ITU-T Rec. H.265.

- **u8 vuiTransferCharacteristics**

The reference of the opto-electronic transfer function of the source picture.

0: ITU-R Rec. BT.2020.

1: SMPTE ST 2084.

2: ARIB STD-B67.

For details, see Table E.4 in ITU-T Rec. H.265.

- **u8 vuiMatrixCoefficients**

The index of matrix coefficients used for deriving luma and chroma signals from green, blue, and red or Y, Z, and X primaries.

The value range is from 0 to 9, inclusive.

For details, see Table E.5 in ITU-T Rec. H.265.

Appendix: Glossary

The following table describes the terms used in this document. For additional HEVC acronyms, see <http://www.hevcbook.de/acronyms/>.

Term	Description
1080p	high-definition resolution of 1920 x 1080 progressive video
720p	high-definition resolution of 1280 x 720 progressive video
ABR	average bit rate
AIoT	artificial intelligence of things
APB	Advanced Peripheral Bus
API	application programming interface
ARIB	Association of Radio Industries and Businesses
ASIC	application-specific integrated circuit
AV1	AOMedia Video 1
AVC	Advanced Video Coding (H.264)
AXI	Advanced eXtensible Interface
bps	bits per second
CABAC	context-adaptive binary arithmetic coding
CAVLC	content-adaptive variable-length coding
CB	coding block
CBR	constant bit rate
CIF	Common Interchange Format (352 x 288 pixels)
CIR	cyclic intra refresh
CPB	coded picture buffer
CPU	central processing unit
CQP	constant quantization parameter
CRF	constant rate factor
CSC	color space conversion
CTB	coding tree block
CTU	coding tree unit

Term	Description
CU	coding unit
CVBR	constrained variable bit rate
DDR	double data rate
DRM	digital rights management
EFBC	external frame buffer compression
EOS	end of sequence
EWL	encoder wrapper layer
FPS	frames per second
GDR	gradual decoder refresh
GOP	group of pictures
HD	high definition
HDTV	high-definition television
HEIF	High Efficiency Image File Format
HDR	high dynamic range
HEVC	High Efficiency Video Coding, a video coding standard (H.265) developed by JCT-VC
HRD	hypothetical reference decoder
IDR	Instantaneous Decoder Refresh
IPCM	intra pulse code modulation
ITU	International Telecommunication Union
ITU-T	ITU Telecommunication Standardization Sector
ITU-R	ITU Radiocommunication Sector
Kbps	kilobits per second
LCU	largest CU
LTR	long-term reference
MAD	mean absolute difference
MB	macroblock
MMU	memory management unit
MPEG-4	Moving Picture Experts Group 4
MV	motion vector
NAL	network abstraction layer
NALU	NAL unit, the smallest decodable unit of a stream
OBU	open bitstream unit
OSD	on-screen display
PAL	Phase Alternate Line (720 x 576 pixels resolution)
POC	picture order count
PPS	picture parameter set
PSNR	peak signal to noise ratio

Term	Description
PU	prediction unit
QCIF	Quarter CIF (176 x 144 pixels)
QVGA	Quarter Video Graphics Array (320 x 240 pixels)
QP	quantization parameter
RC	rate control
RDO	rate-distortion optimization
RDOQ	rate-distortion optimized quantization
REE	rich execution environment
RFC	reference frame compression
RGB	red, green, blue color space representation
ROI	region of interest
RPS	reference picture set
SAO	sample adaptive offset
SBI	segment buffer interface
SEI	supplemental enhancement information
SMPTE	Society of Motion Picture and Television Engineers
SPS	sequence parameter set
SSIM	structural similarity index measure
STR	short-term reference
sub-QCIF	Sub Quarter CIF (128 x 96 pixels resolution)
SVC-T	temporal scalable video coding
SXGA	Super Extended Graphics Array (1280 x 1024 pixels resolution)
TU	transformation unit
TV	television
VBR	variable bit rate
VBV	video buffer verifier
VGA	Video Graphics Array (640 x 480 pixels resolution)
VLC	variable-length code
VPS	video parameter set
VUI	video usability information
YCbCr	color space with one luma and two chroma components, used for digital encoding
YUV	color space with one luma and two chroma component, used for either digital or analog encoding

Document Revision History

This section describes top level differences in the versions of this document.

Note

This document is not necessarily updated for each patch or minor revision.

Document Revision	Date	Compatible Core	Description
1.11	2024-01-02	VC9000E	Updated the document name and template.
1.10	2023-05-23	VC9000E	- Reconstructed the document and refined descriptions. - Added Section <i>Cyclic Intra Refresh</i>, Section <i>Frame Statistics Output</i>, Section <i>Security Encoding</i>, Section <i>High Efficiency Image File Format</i>, Section <i>HEIF Writer API</i>, Section <i>HDR Information Programming</i>, and Section <i>HDR10+ or Dolby Vision Information Insertion</i>. - Added descriptions about QP/CU information formats in Section <i>ROI Map</i>. - Removed the <i>Video Frame Storage Format</i> section. For the requirements on the input frame buffer, see <i>Hantro VC9000E Series Memory Buffer and Format Organization</i>. - Added the helper API group. For details, see Section <i>Helper API</i>. - Added <code>VCEncConfig.enableFrameInfo</code>, <code>VCEncCodingCtrl.rect0</code> to <code>VCEncCodingCtrl.rect7</code>, and <code>VCEncOut.frameInfo</code>.

Document Revision	Date	Compatible Core	Description
1.01	2021-09-30	VC9000E	First release of the Doxygen format. Added Section <i>Bit Rate Control Features</i> and Section <i>ROI-Based Coding Control</i> .
1.00	2021-03-15	VC9000E	Initial release.